



CART FOR REGRESSION AND CLASSIFICATION

Predicting Medical Costs and Parole Violations

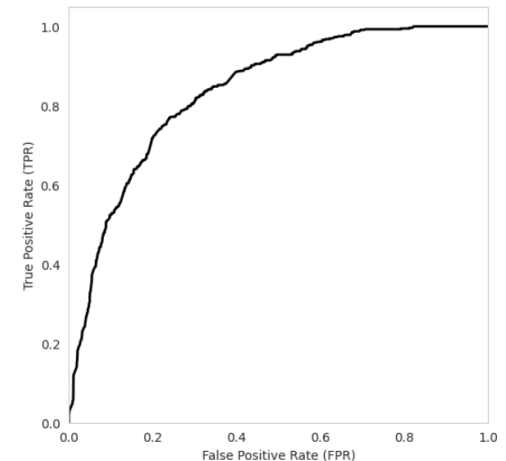
Review of Classification

	Churn	SeniorCitizen	InternetService	tenure	Contract	ElectronicCheck	Pr($Y_i=1$)	ChurnRisk
3	0	0	DSL	45	One year	0	0.062676	0
6	0	0	Fiber optic	22	Month-to-month	0	0.465475	1
13	1	0	Fiber optic	49	Month-to-month	0	0.278666	0
15	0	0	Fiber optic	69	Two year	0	0.042819	0
18	1	0	DSL	10	Month-to-month	0	0.287129	0
...	...	...	...	...	...	...	...	...
7019	0	0	No	2	Month-to-month	0	0.174558	0
7020	0	1	DSL	55	One year	0	0.062797	0
7027	0	0	DSL	24	One year	0	0.111760	0
7029	0	0	DSL	11	Month-to-month	1	0.384410	1
7030	1	1	Fiber optic	4	Month-to-month	0	0.669652	1

Reality

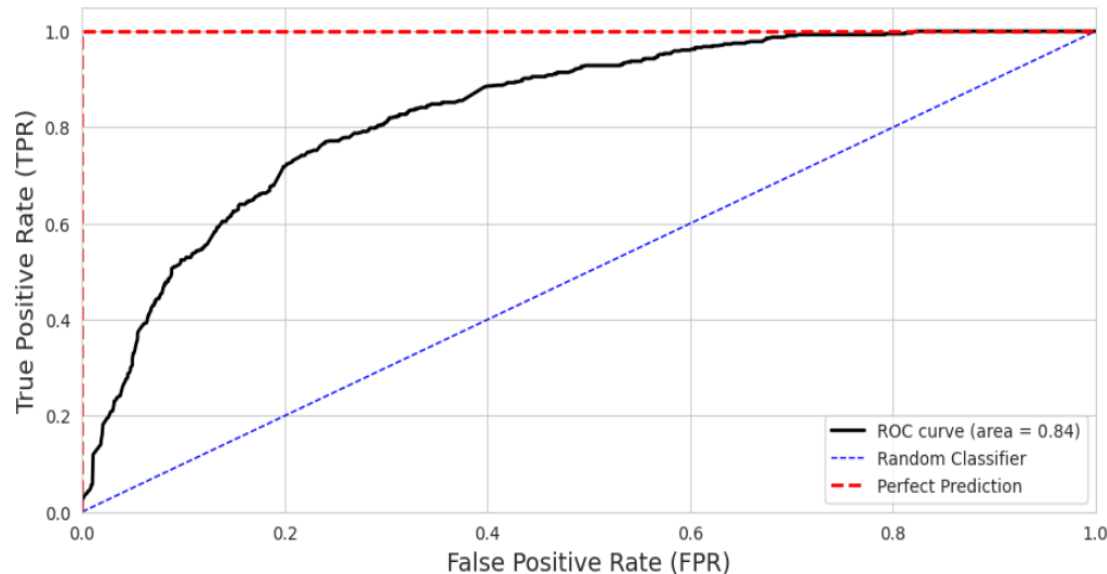
Probabilities

Vary Cutoff



ROC Curve
AUC Score

Area Under the ROC Curve (AUC)



- AUC of perfect prediction is 1.0
- AUC of our Logistic Regression model is 0.841
- AUC of the (very) naïve baseline model that predicts randomly is 0.50
- If the AUC of your model is not much larger than 0.50, then your model is not much better than the (very) naïve baseline model -- which itself is “blind” to the data

How to use AUC-ROC?

- AUC
 - A cutoff-free metric of model
- ROC
 - Instead of a single number (AUC), ROC is the whole curve.
 - Pick a point on ROC = Select a cutoff threshold
 - Tradeoffs between RPR and FPR
- Value(Profit/Cost)-based cutoff
 - Profit/cost for each cell in the confusion matrix
 - Evaluate which cutoff to use based on profit/cost evaluation

Value-based approach

		Model	
Reality		Low-churn risk $\Pr(Y_i = 1) < 0.333$	High-churn risk $\Pr(Y_i = 1) > 0.333$
	No Churn ($Y_i = 0$)	1,217	332
	Churn ($Y_i = 1$)	147	414

Cost		Model	
Reality		Low-churn Risk	High-churn Risk
	No Churn ($Y_i = 0$)	\$0	\$200
	Churn ($Y_i = 1$)	\$400	\$0



PREDICTING HEALTHCARE COSTS

An Introduction to CART for Regression

Healthcare Claims Data

Outline

- Motivating Dataset: Predicting Healthcare Costs
- Introduction to Classification and Regression Tree (CART)
 1. What is CART and how it makes predictions
 2. How to construct a model from data
 3. Parameters that one can control, and the trade-offs induced
- Cross Validation
 - Use cross validation to select parameters without using test set
- Results on Healthcare Dataset
 - Comparing CART and Linear Regression

Predicting Healthcare Costs

- Predicting healthcare future annual cost of patients
 - **Cost:** Lower cost for the healthcare system.
- Historical medical cost data can be used as a proxy for worsening health conditions.
 - **Health Outcomes:** “An ounce of prevention is worth a pound of cure”
- **How might we predict patient medical costs?**

Pre-Analytics Approach

- Limited access to high-quality data
 - Unstructured data—from doctors' and nurses' notes
 - Unavailable data—hard to transfer data across technology platforms
 - Inaccessible—strong privacy laws concerning healthcare data sharing
- Medical knowledge primarily developed through human judgment and clinical trials
 - MDs and other trained staff manually analyzed patient histories
- Costly, inefficient, and not so accurate



Analytics-based Approach

- D2Hawkeye: Healthcare analytics company founded by C. Kryder and R. Pandey, sold to Verisk Health in 2009 (HBS case)
 - Advanced healthcare analytics including risk analysis tools and predictive modeling to identify high-risk patients for intervention.
 - Combines medical management tools (e.g., databases) with analytics to improve patient care and reduce cost.
 - Analyzed 20M patients monthly
- Our public dataset (akin to what D2Hawkeye used)
 - **Claims Data:** Requests for Medicare reimbursement submitted to insurance from doctors, hospitals, and pharmacies. Aggregated.
 - Data on yearly cost, health risk assessments, as well as non-medical data (e.g., demographic)

Healthcare Claims Data

Healthcare Claims Data

k : # of independent variables ($k = 27$); n : # of observations ($n = 1,023,952$)

$n = 1,023,952$

	reimb2010	reimb2008	reimb2009	age2010	male	race	heart.failure	kidney	cancer	copd	depression	diabetes	ihd
0	390	320.0	360	97	0	White	0	0	0	0	0	1	1
1	970	58800.0	2740	79	0	White	1	1	0	1	0	1	1
2	5630	510.0	1580	87	0	White	1	0	0	0	0	1	1
3	3480	2930.0	49330	79	0	White	1	1	0	1	1	1	1
4	920	1500.0	1650	85	1	White	1	0	1	0	0	1	0
...	...	...	...	...	...	...	...	...	...	...	...	...	...
1023947	6620	2090.0	4070	54	1	Black	1	0	0	1	1	1	1
1023948	490	12450.0	1560	70	1	White	1	1	0	0	0	1	1
1023949	690	1370.0	2440	75	0	Other	1	1	0	1	0	0	1
1023950	1140	6530.0	9220	78	0	White	0	1	1	0	0	1	1
1023951	1620	4160.0	18490	67	0	White	1	1	0	1	1	1	1

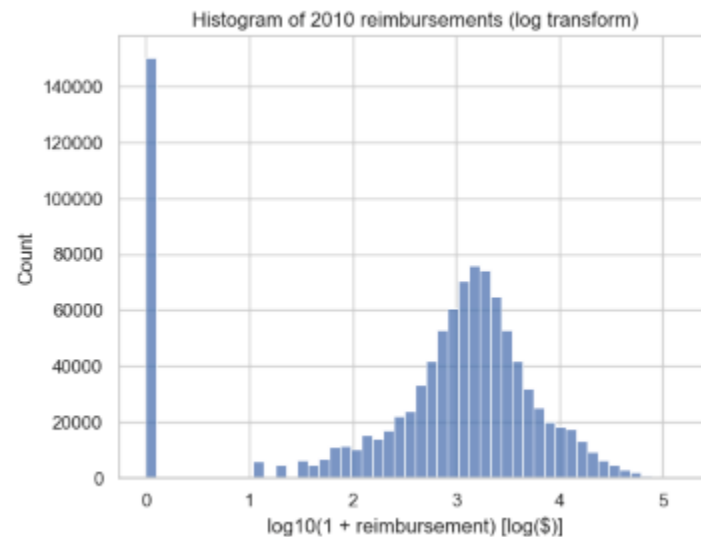
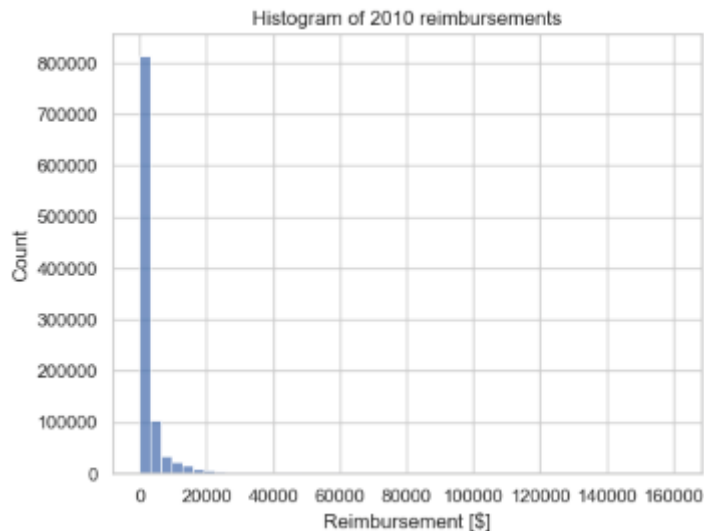
1023952 rows × 28 columns

Healthcare Claims Data

- Data: 1,023,952 patients who had medical expenses in 2008–10
- **Dependent Variable:**
Total amount of expenditures (Medicare reimbursements) in 2010 (log)
- **Independent Variables**
 - Expenditures in 2008, 2009 (log)
 - Demographics: age, gender, race
 - Diseases contracted during the two previous years: Arthritis, Cancer, Depression, Diabetes, Osteoporosis, etc.
 - Number of inpatient/outpatient claims during the previous two years
 - Office visits, eye exams, EKGs, X-rays, CT scans, physical therapy, ambulance use, cost trend, and acuity* (measures monthly cost spikes)
 - * Acuity measures the extent to which annual costs are concentrated in one particular month

Histogram of Dependent Variable

- Highly skewed to the right (expected)
- Log Transformation is common when the dependent variable has values with different *orders of magnitude*.
 - Reimbursements $\rightarrow \text{Log}_{10}(1 + \text{Reimbursements})$
 - E.g., \$0 $\rightarrow$ 0, \$100 $\rightarrow$ 2, \$100,000 $\rightarrow$ 5



Training Set / Test Set

Full Dataset ($n = 1,023,952$)

	log10_reimb2010	log10_reimb2008	log10_reimb2009
0	2.592177	2.506505	2.557507
1	2.987219	4.769385	3.437909
2	3.750586	2.708421	3.198932
3	3.541704	3.467016	4.693120
4	2.964260	3.176381	3.217747
5	3.049606	3.057286	3.385785
6	3.061075	2.654177	4.175541
7	0.000000	0.000000	1.908485
8	3.013259	2.399674	3.637590
9	0.000000	0.000000	0.000000
10	0.000000	0.000000	0.000000
11	0.000000	0.000000	0.000000

70% Training Set ($n = 716,983$)

	log10_reimb2010	log10_reimb2008	log10_reimb2009
0	2.592177	2.506505	2.557507
1	2.987219	4.769385	3.437909
3	3.541704	3.467016	4.693120
5	3.049606	3.057286	3.385785
6	3.061075	2.654177	4.175541

30% Testing Set ($n = 306,969$)

	log10_reimb2010	log10_reimb2008	log10_reimb2009
2	3.750586	2.708421	3.198932
4	2.964260	3.176381	3.217747
8	3.013259	2.399674	3.637590
12	3.893817	2.987219	3.029789
16	3.155640	3.450403	4.750362

Linear regression

- Linear regression model using the training set:

$$\begin{aligned}\log10_reimb2010 &= \beta_0 \\ &+ \beta_1 \times \log10_reimb2008 \\ &+ \dots \\ &+ \beta_{29} \times \text{costTrend}\end{aligned}$$

- Then let us look at the linear regression R^2 and OSR^2

Linear Regression Model

Linear Model Coefficients

	Coef.	Std.Err.	t	P> t	[0.025	0.975]	Signif
const	0.5311	0.0065	81.1582	0.0000	0.5182	0.5439	***
log10_reimb2008	0.3316	0.0017	193.4289	0.0000	0.3282	0.3350	***
log10_reimb2009	0.4482	0.0019	230.2936	0.0000	0.4444	0.4521	***
age2010	0.0005	0.0001	6.7429	0.0000	0.0004	0.0007	***
male	-0.0101	0.0019	-5.3299	0.0000	-0.0137	-0.0064	***
heart.failure	0.0222	0.0024	9.1629	0.0000	0.0175	0.0270	***
kidney	-0.0088	0.0026	-3.4308	0.0006	-0.0138	-0.0038	***
cancer	-0.0167	0.0030	-5.6234	0.0000	-0.0226	-0.0109	***
copd	0.0025	0.0026	0.9786	0.3278	-0.0026	0.0077	
depression	0.0136	0.0022	6.1130	0.0000	0.0093	0.0180	***
diabetes	0.0780	0.0027	28.8473	0.0000	0.0727	0.0833	***
ihd	0.0749	0.0028	26.3190	0.0000	0.0693	0.0805	***
osteoporosis	0.0141	0.0022	6.4420	0.0000	0.0098	0.0184	***
arthritis	0.0113	0.0025	4.5983	0.0000	0.0065	0.0161	***
stroke	-0.0260	0.0035	-7.5341	0.0000	-0.0328	-0.0193	***
InpatientClaims	-0.0960	0.0012	-78.1305	0.0000	-0.0984	-0.0936	***
OutpatientClaims	0.0001	0.0002	0.6853	0.4932	-0.0002	0.0005	
OfficeVisit	-0.0016	0.0002	-7.1429	0.0000	-0.0020	-0.0011	***
EyeExam	-0.0054	0.0007	-8.2516	0.0000	-0.0066	-0.0041	***
EKG	-0.0002	0.0007	-0.3142	0.7534	-0.0015	0.0011	
xray	0.0003	0.0005	0.7036	0.4817	-0.0006	0.0013	
CTScan	-0.0007	0.0008	-0.8841	0.3766	-0.0023	0.0009	
PhysicalTherapy	-0.0005	0.0002	-2.4871	0.0129	-0.0010	-0.0001	*
Ambulance	-0.0021	0.0004	-5.8418	0.0000	-0.0028	-0.0014	***
acuity	-0.5570	0.0043	-129.2047	0.0000	-0.5655	-0.5486	***
costTrend	0.2986	0.0062	48.4398	0.0000	0.2865	0.3106	***
race_Hispanic	-0.0020	0.0069	-0.2978	0.7658	-0.0155	0.0114	
race_Other	-0.0022	0.0056	-0.3982	0.6905	-0.0131	0.0087	
race_White	0.0203	0.0031	6.4611	0.0000	0.0142	0.0265	***

Significance codes: 0 '***' 0.001 '**' 0.05 '*' 0.1 '.' 1

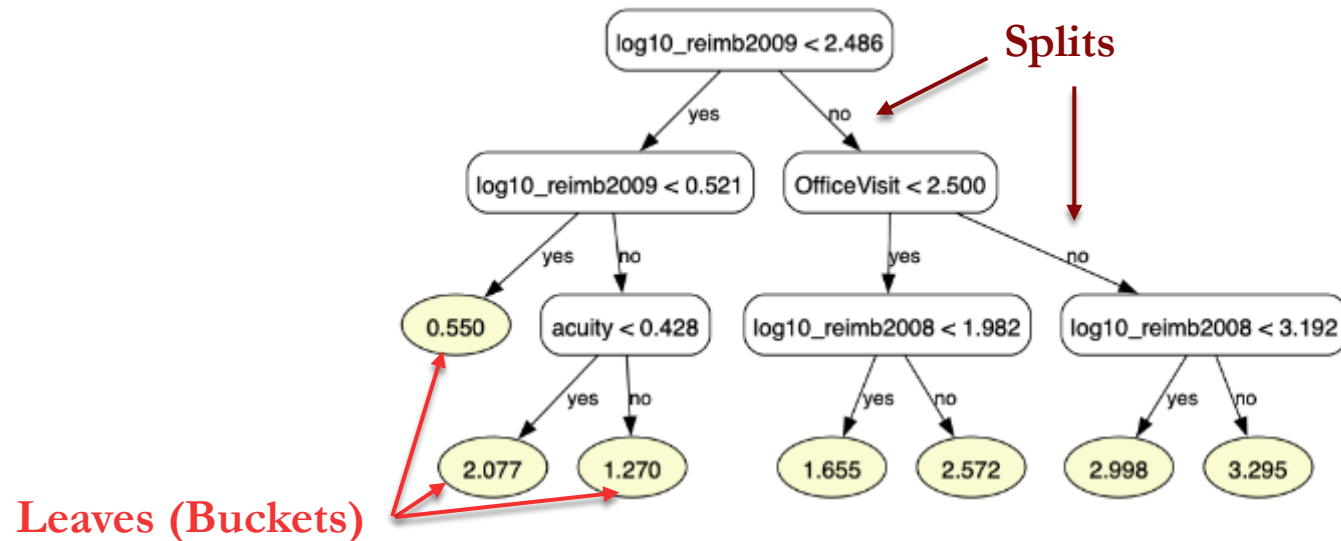
Linear regression performance:

- training set: $R^2 = 0.604$
- test set: $OSR^2 = 0.602$
- 21/29 significant variables

Classification and Regression Tree (CART)

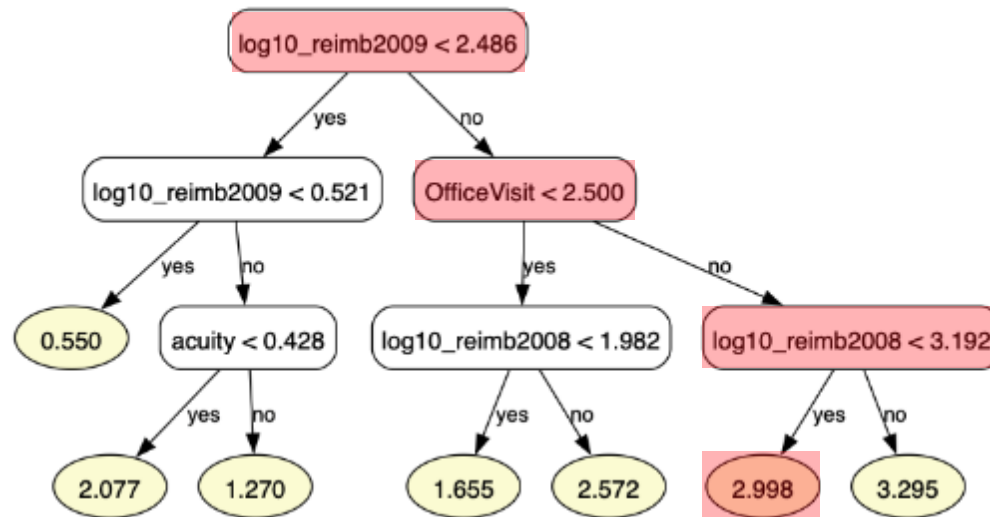
1. What is CART and how does it make predictions
2. How to construct a model from data
3. What are the parameter that you can control, and what are the trade-offs induced?

What is a CART Model?



- CART is a “tree”-based model used to make predictions.
- Given the observed values of the independent variables, just “follow the splits” to the final node
 - At the split, go left if “yes” and go right if “no”
- Follow the splits to the end/bottom of the tree and read off the value. That’s it!

How to Make Predictions



Consider David

- 83 years old, \$379 reimbursements in 2009 (= 2.58 after log transformation), diabetes, ..., 5 office visits, \$290 reimbursements in 2008 (= 2.46 after log transformation), What are David's expected healthcare costs in 2010?
- Answer: $10^{2.998} - 1 = \$994$

Classification and Regression Tree (CART)

- A CART model is a tree that makes predictions by “following the splits” to the final node.
- CART models can be applied to predict either:
 - a numerical outcome – called a regression tree
 - a classification outcome – called a classification tree
- Each split is defined by a single variable
 - **Numerical Variables:** Variable < Threshold or Variable > Threshold
 - **Categorical Variables:** Variable takes a certain value or not
- **Remaining Questions: How does CART build a tree?**
 - How does CART choose each split? (Answer: Minimize SSE *greedily*)
 - How does CART choose when to stop splitting? (Answer: You decide through the parameters! But there are tradeoffs...)

Building a CART Model

How to Choose Each Split

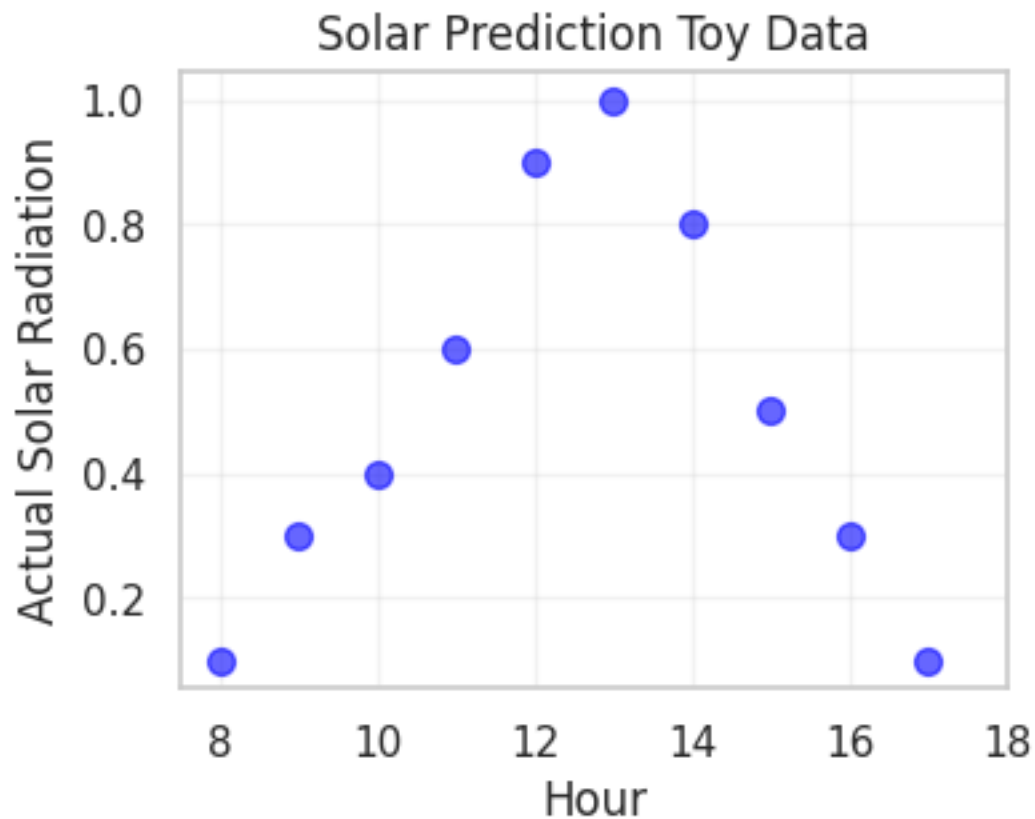
High-Level Ideas

- CART builds up a decision tree step-by-step from a tree stump
- In each step, a leaf is branched (becomes a split) and create two new leaves:
 - Which leaf is chosen to be split?
 - How to define the condition of the split?
 - For a newly-created leaf, it defines a bucket of data points. How to set the predicted values?

Two Toy Examples

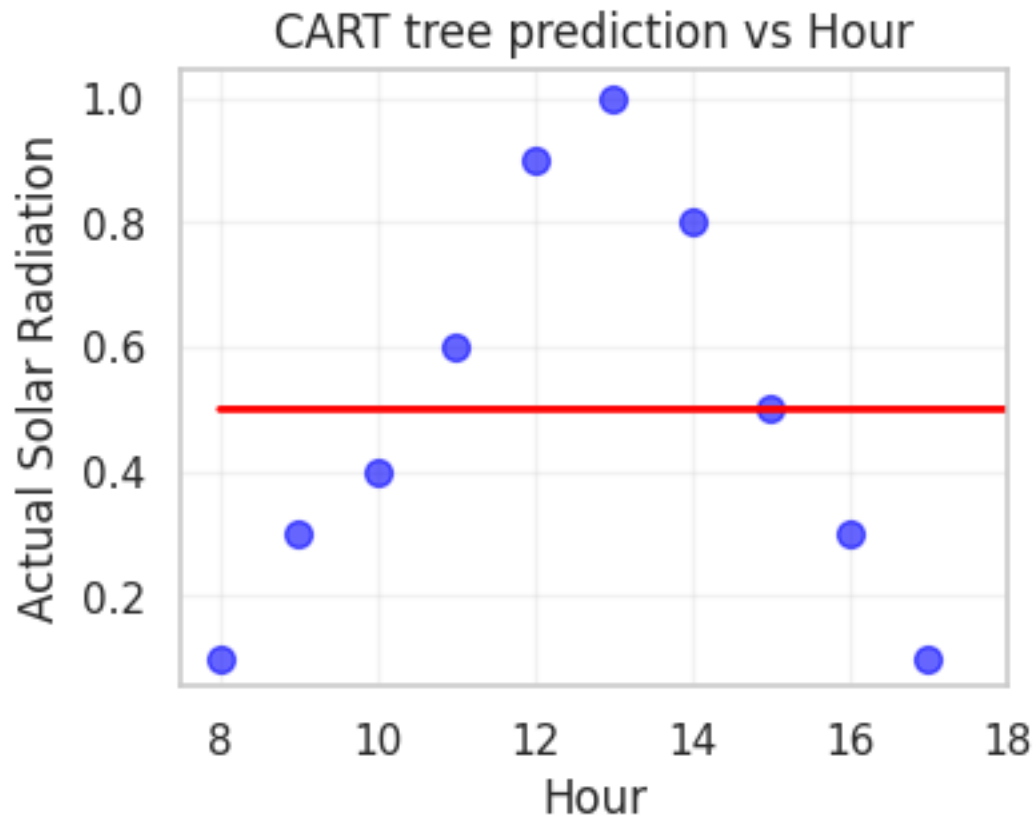
- Toy Example in 1-Dimension (1 Feature)
 - Predicting Solar Radiation with only Hour
- Toy Example in 2-Dimensions (2 Features)
 - Predicting Hourly Salary with GPA and Years of Experience

Toy Example 1



Hour	Actual Solar Radiation
8	0.1
9	0.3
10	0.4
11	0.6
12	0.9
13	1.0
14	0.8
15	0.5
16	0.3
17	0.1

Tree 1

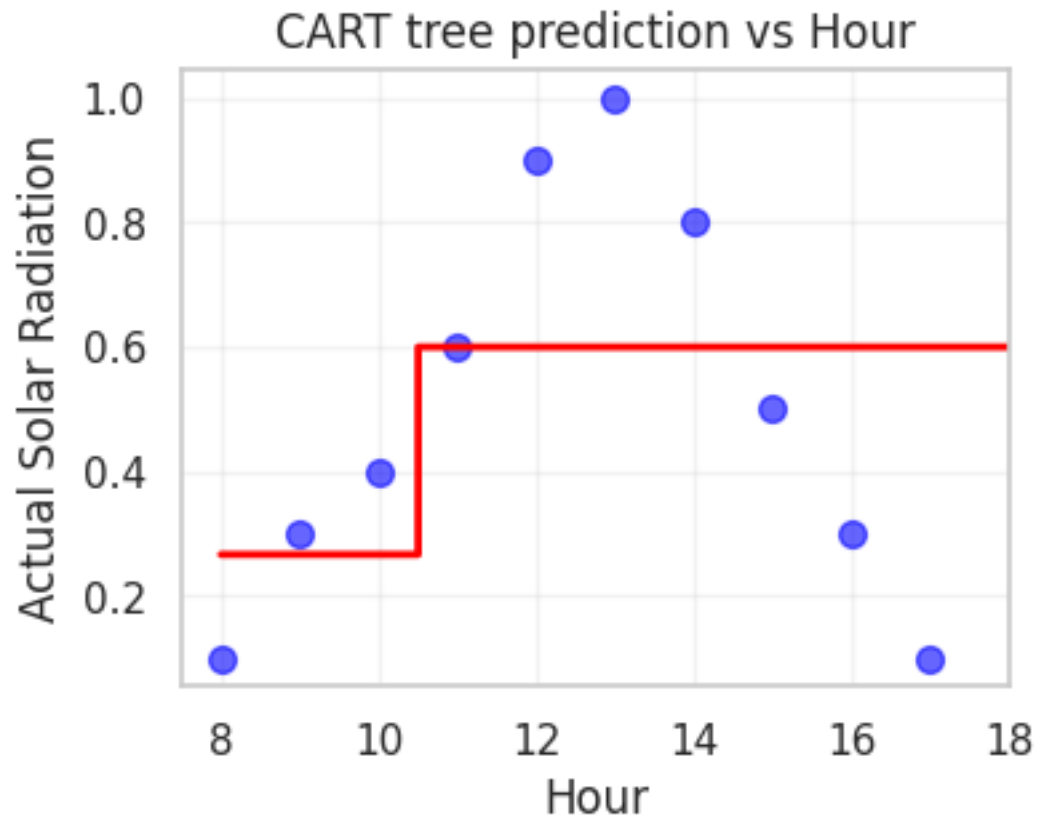


Tree 1

0.500

SSE: 0.920

Tree 2

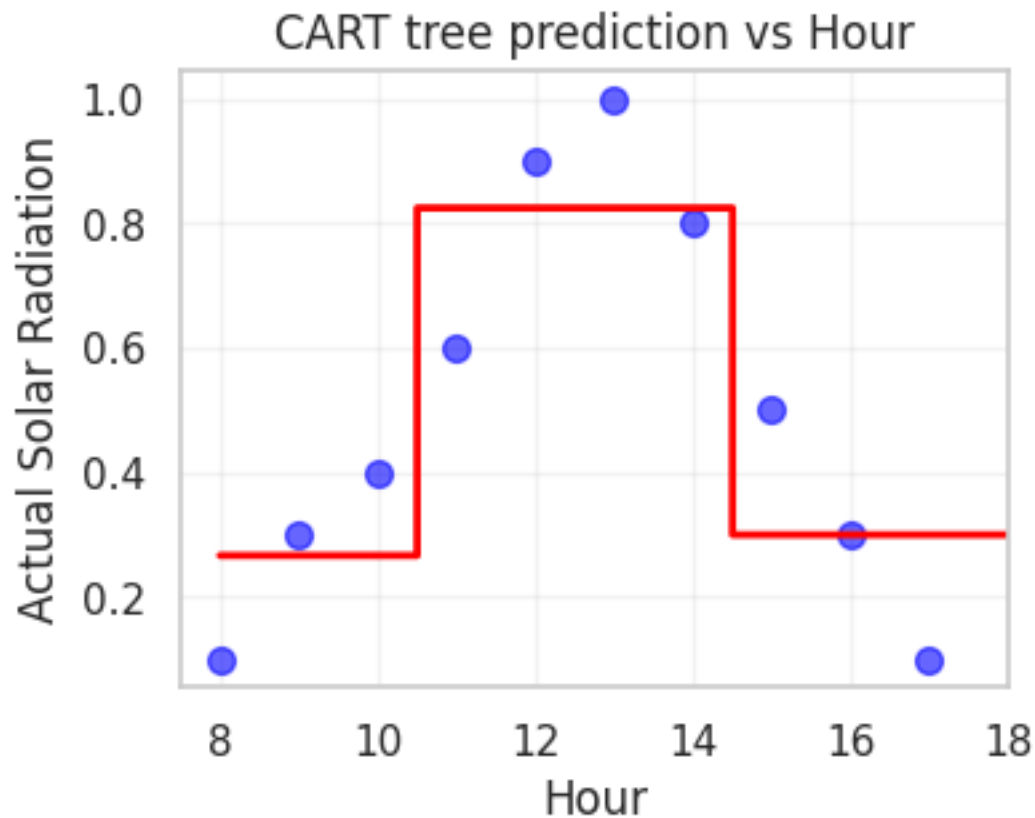


Tree 2

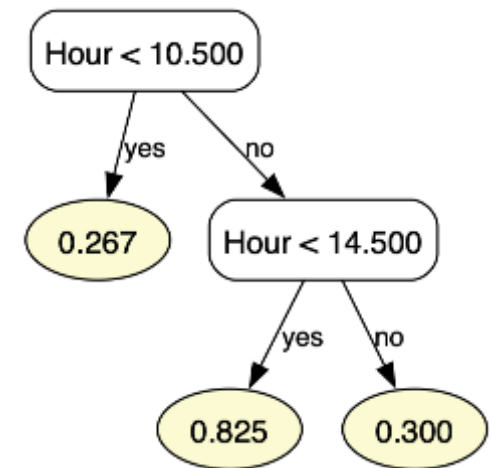


SSE: 0.687

Tree 3

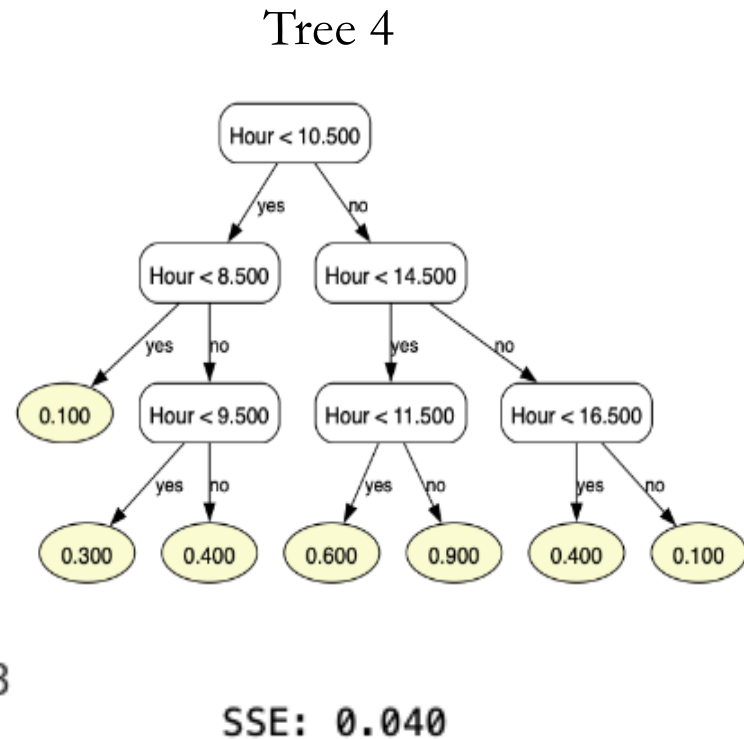
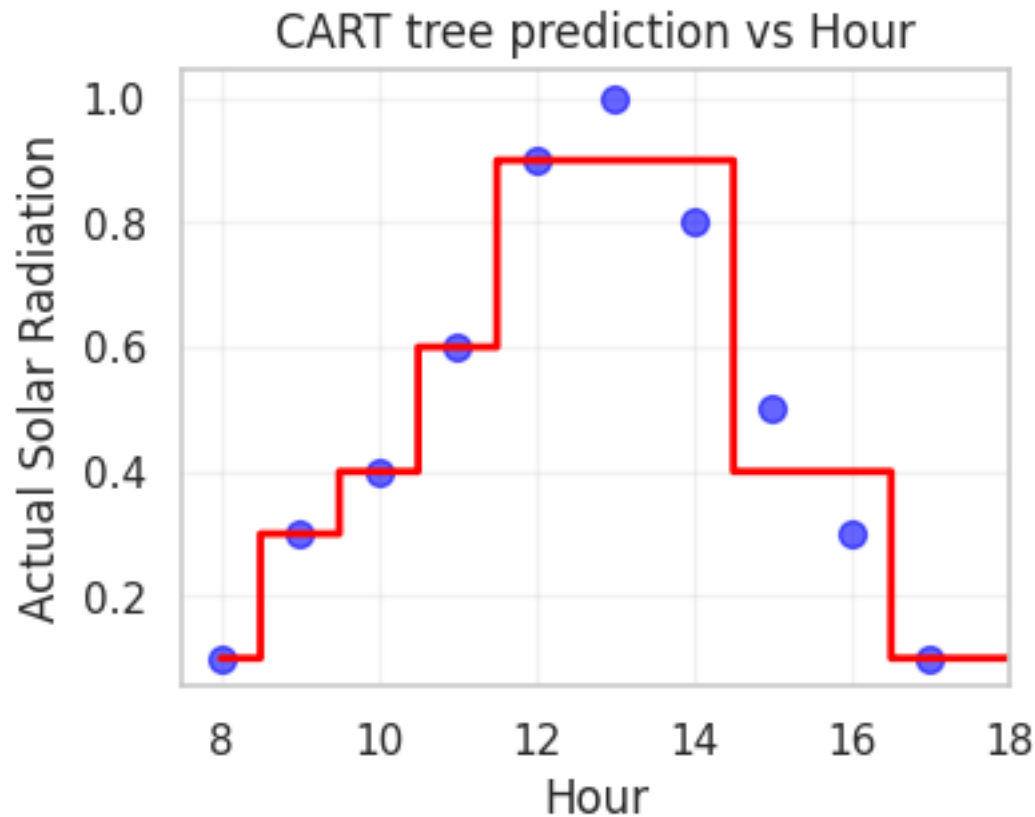


Tree 3

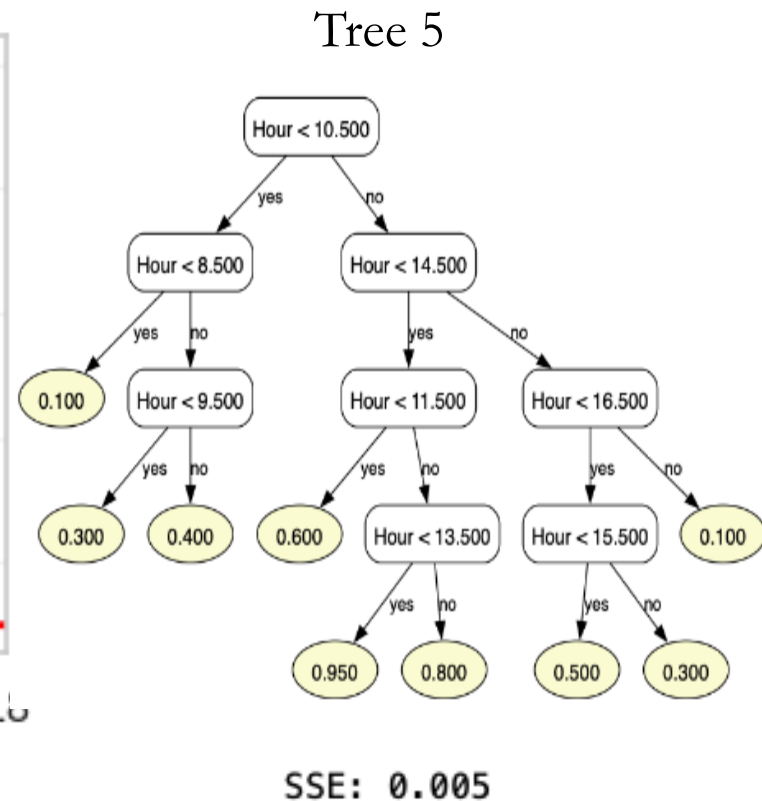
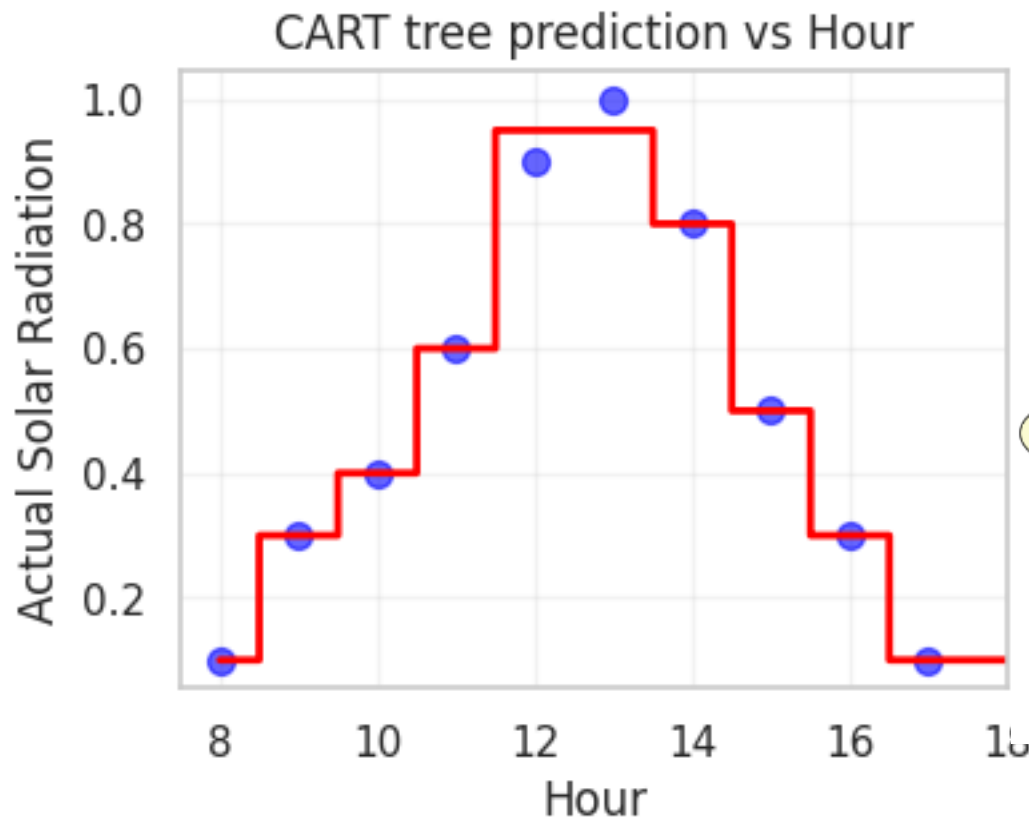


SSE: 0.214

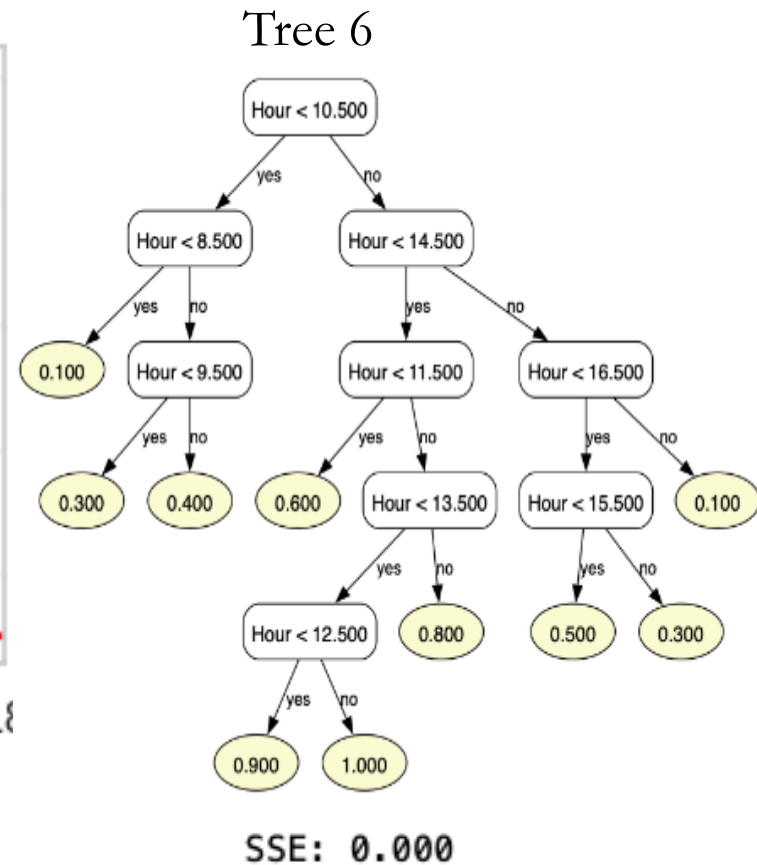
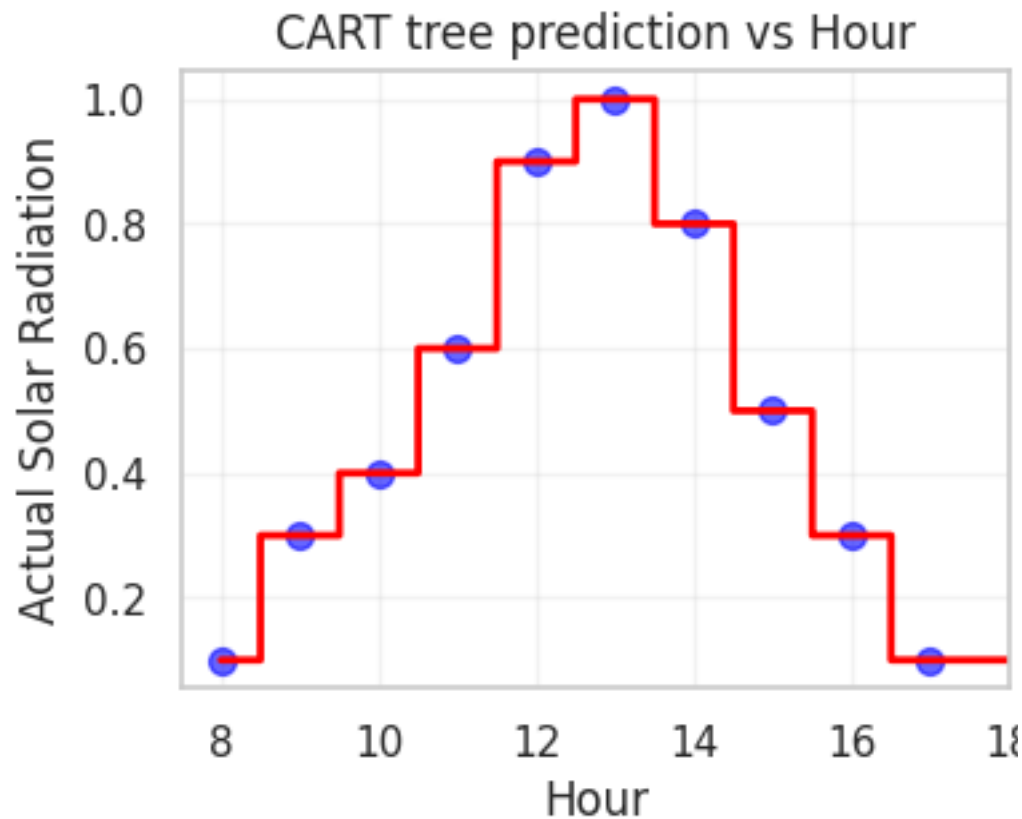
Tree 4



Tree 5

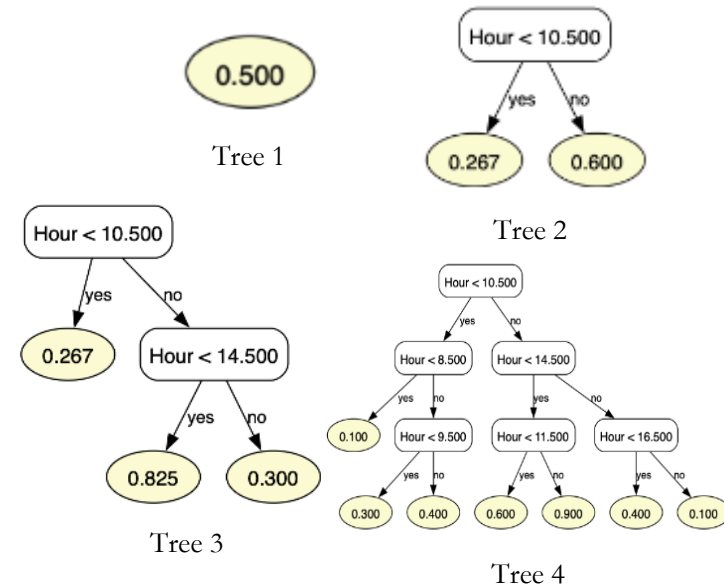


Tree 6



Each Split Decreases SSE and Increases R^2

Tree	Number of Splits	Number of Leaves	SSE	In-Sample R^2
Tree 1	0	1	0.920	0.000
Tree 2	1	2	0.687	0.254
Tree 3	3	3	0.214	0.767
Tree 4	6	7	0.042	0.957
Tree 5	8	9	0.005	0.995
Tree 6	9	10	0.000	1.000



- How is the value at leaf chosen?
 - CART predicts the mean of the data points in that leaf node
- How is the split chosen?
 - At each step, a new split is chosen to *minimize* SSE and equivalently maximize in-sample R^2
 - Splits are chosen in a greedy (myopic) fashion.

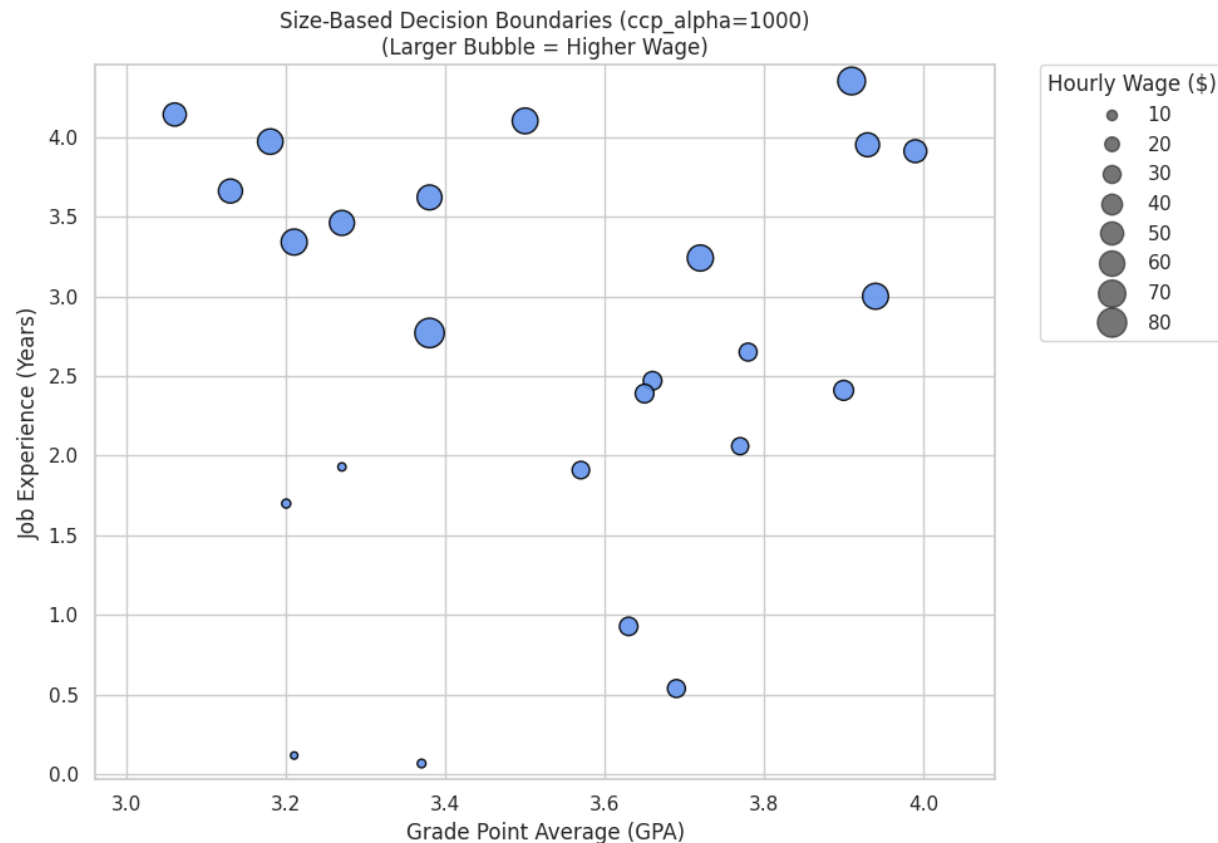
Toy Example 2: Predicting Salaries

k : # of independent variables ($k = 2$); n : # of observations ($n = 25$)

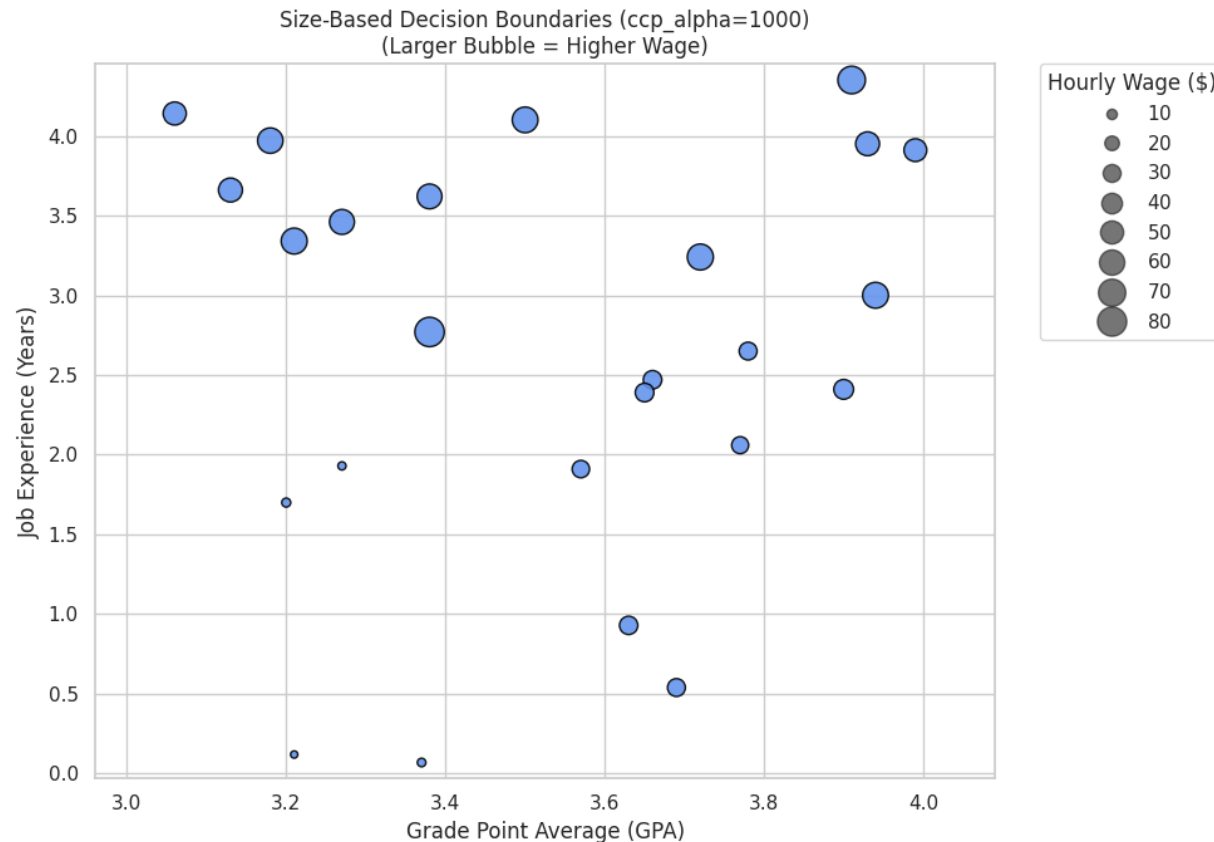
$n = 25$

	Hourly wage	Job Experience (Years)	Grade Point Average
0	6.33	1.93	3.27
1	6.92	0.07	3.37
2	28.50	1.91	3.57
3	71.58	4.35	3.91
4	7.64	1.70	3.20
5	36.89	2.41	3.90
6	62.87	3.00	3.94
...			
22	32.71	2.39	3.65
23	54.18	3.66	3.13
24	58.46	3.46	3.27

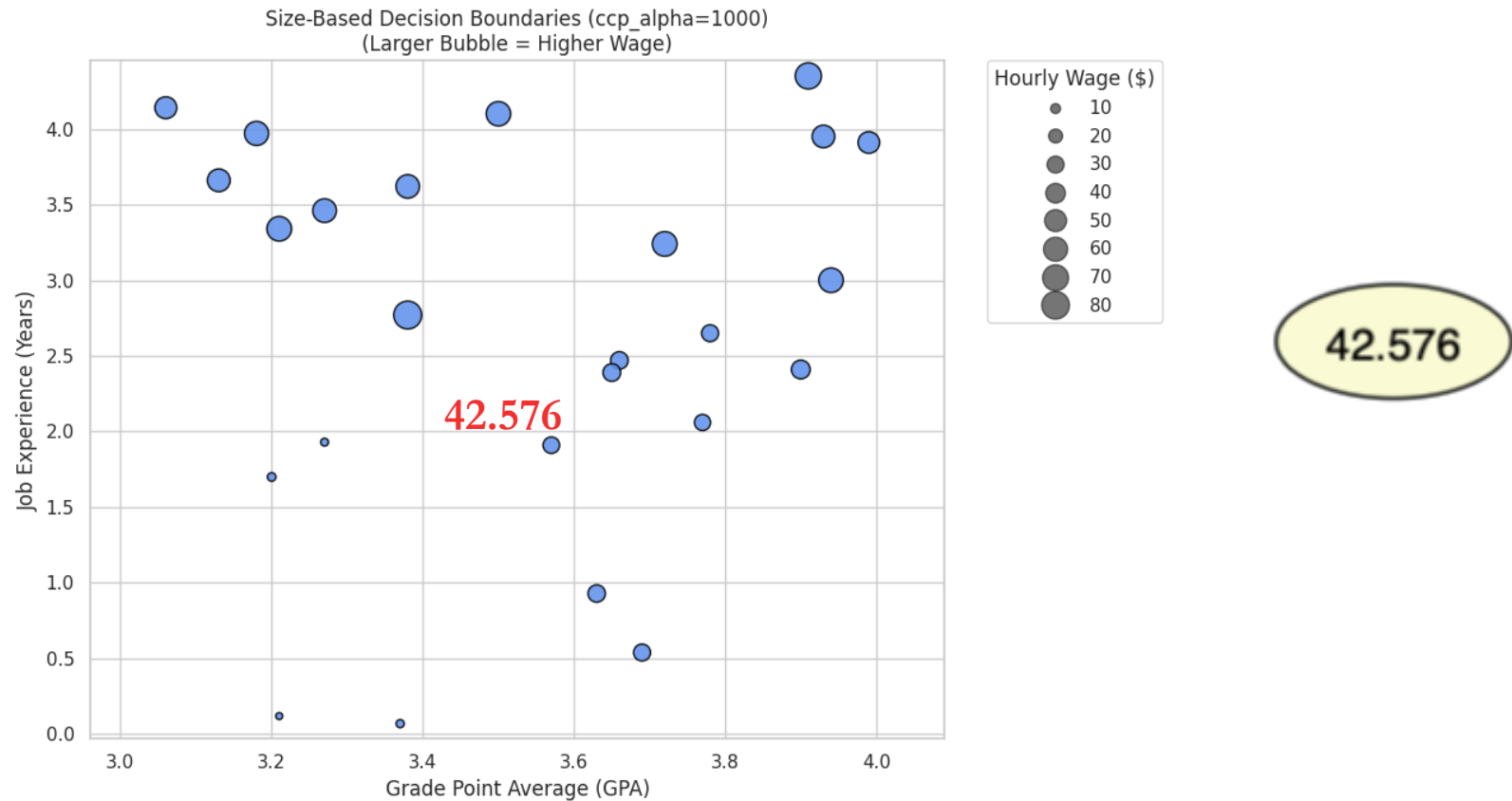
Visualizing the Example Data



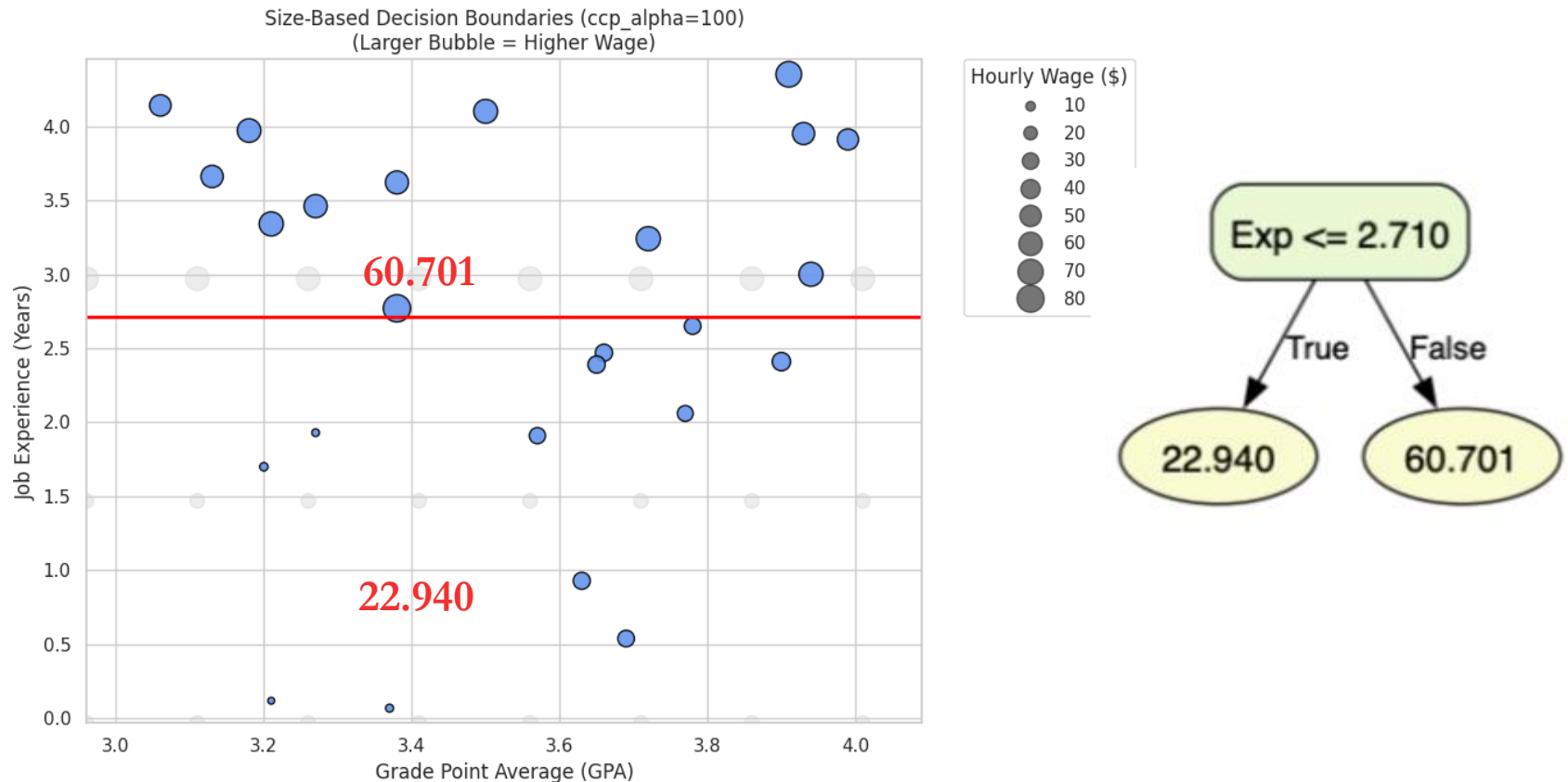
Visualizing the Example Data



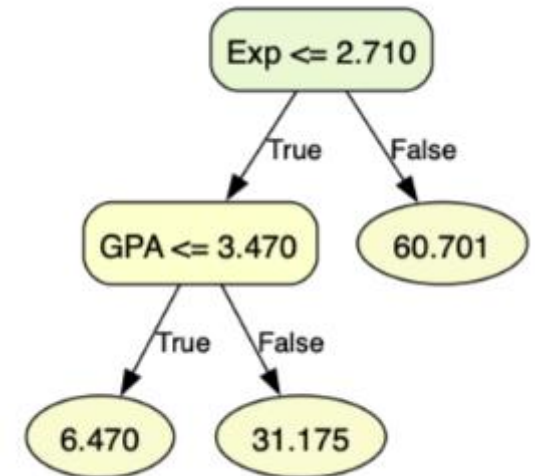
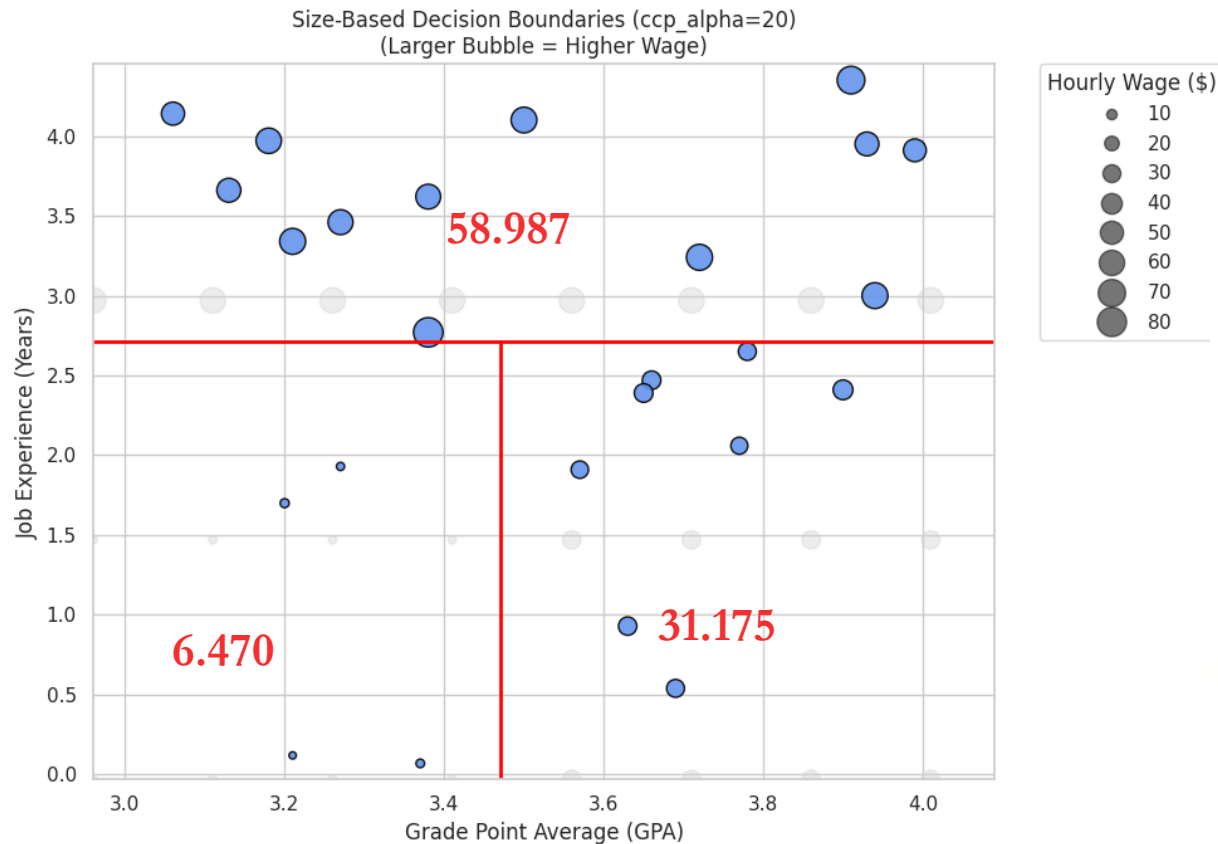
Predicting Salary with CART model



Predicting Salary with CART model



Predicting Salary with CART model, continued



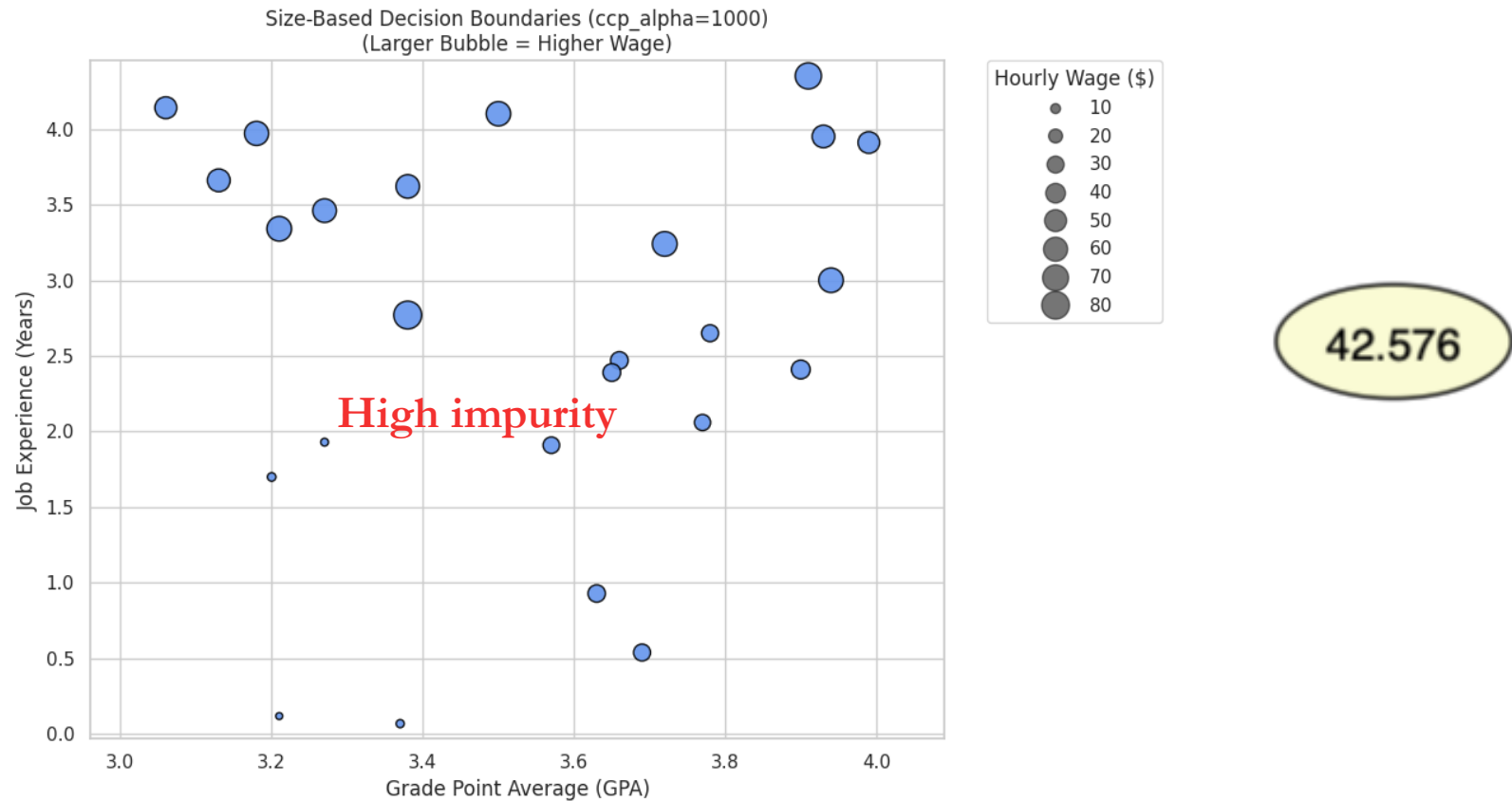
High-Level Ideas

- CART builds up a decision tree step-by-step from a tree stump
- In each step, a leaf is branched (becomes a split) and create two new leaves:
 - Which leaf is chosen to be split?
 - How to define the condition of the split?
 - For a newly-created leaf, it defines a bucket of data points. How to set the predicted values?

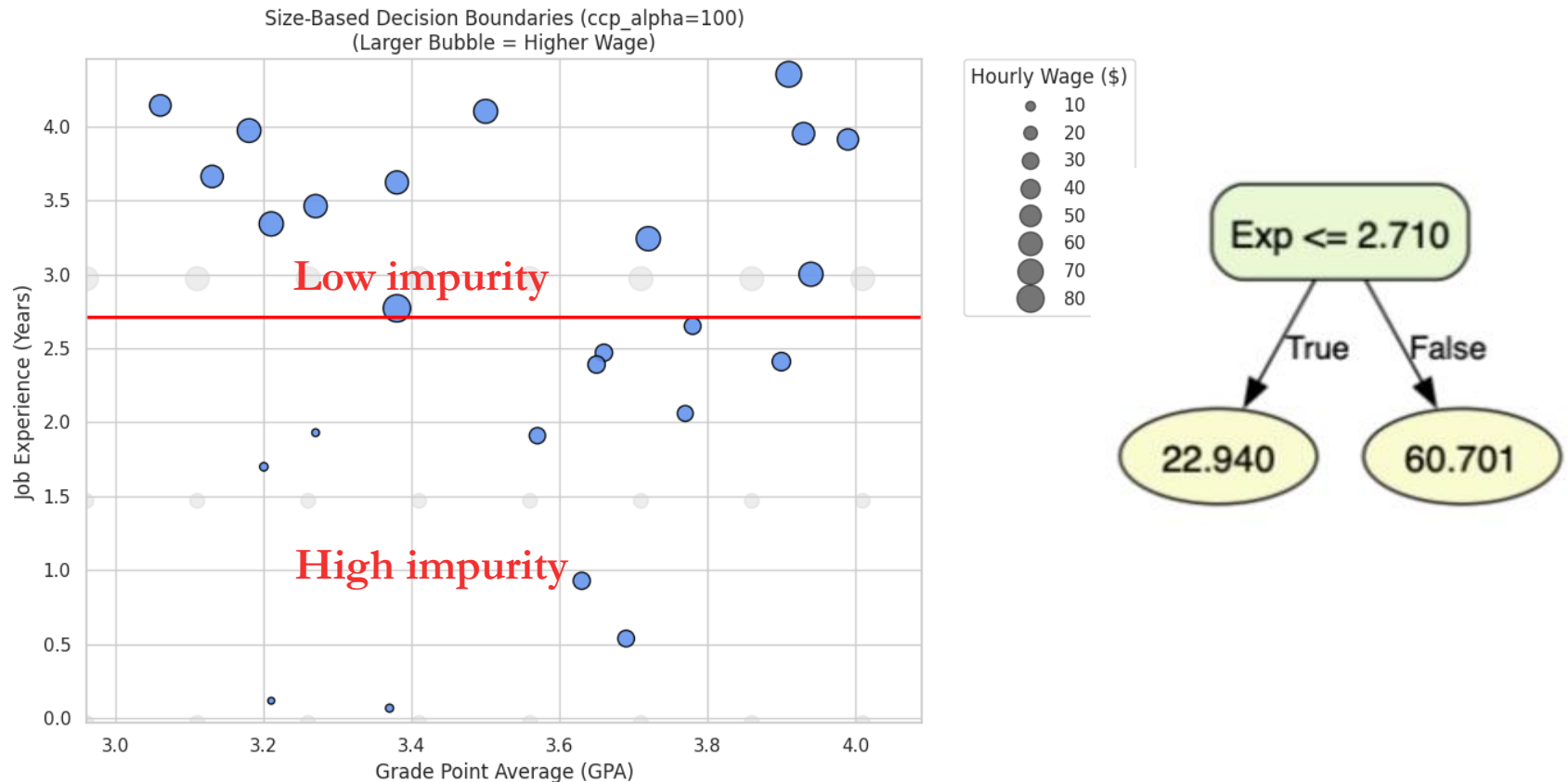
How does CART build a regression tree?

- In general, the splitting criteria (i.e., independent variable and the threshold) is determined by a particular measure of *purity*
 - For **regression tree** problems, purity refers to a bucket that contains a narrow range of values.
 - For **classification** problems, purity refers to a bucket that contains mostly 1s or mostly 0s.
- CART will select the split that reduces the impurity the most.
 - Across all variables and thresholds.

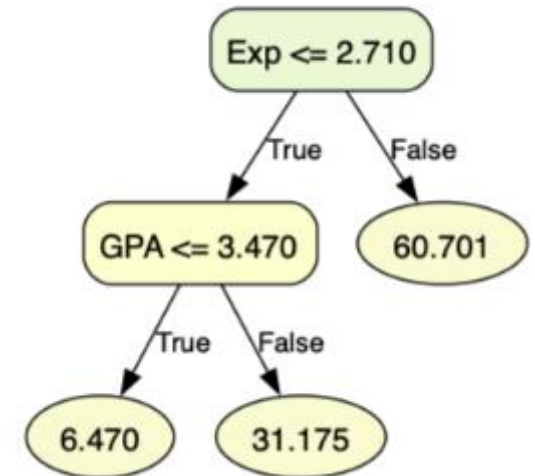
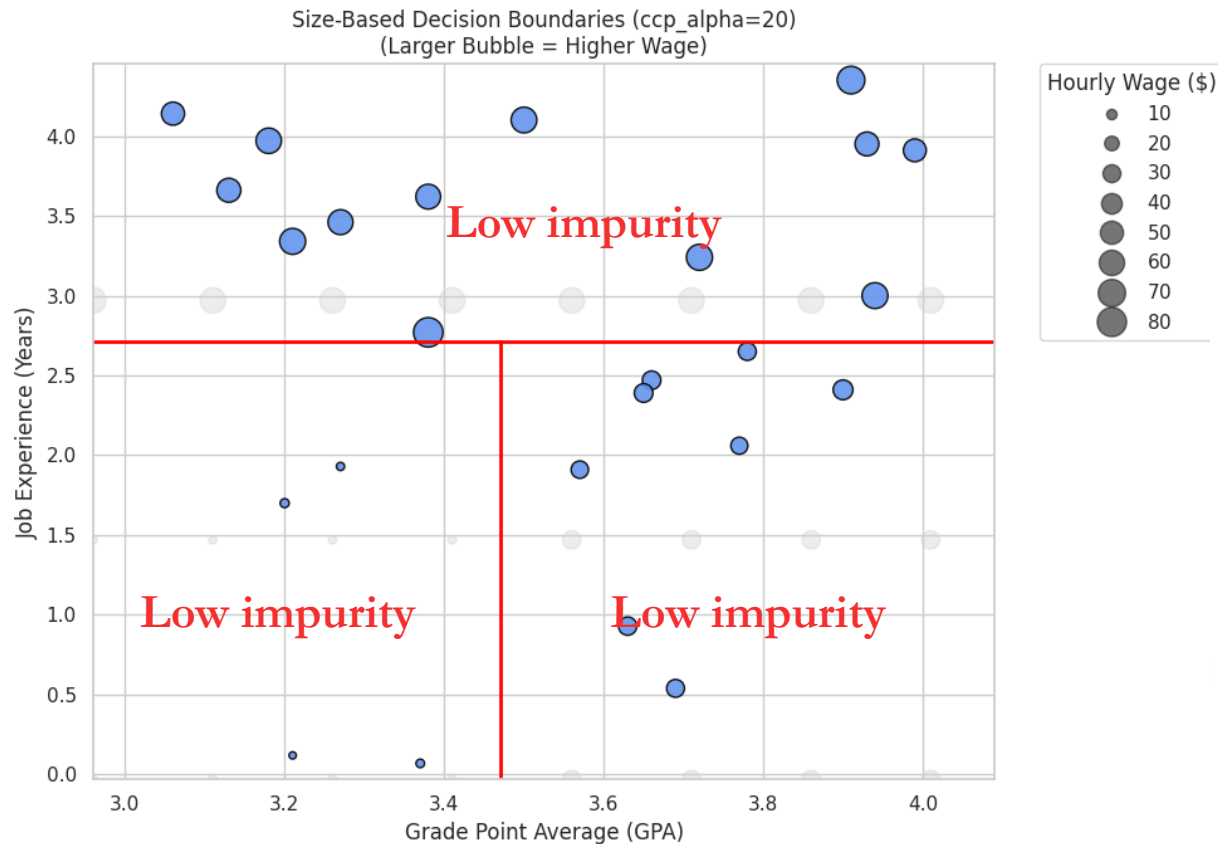
Predicting Salary with CART model



Predicting Salary with CART model



Predicting Salary with CART model, continued

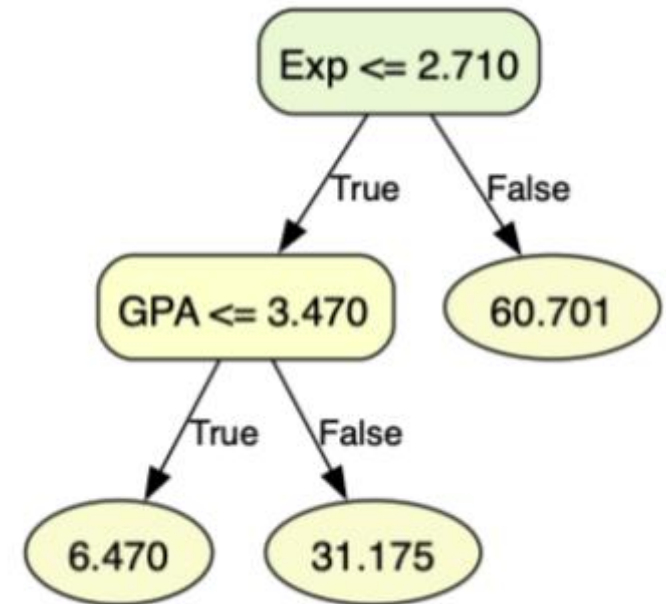


Benefits of CART

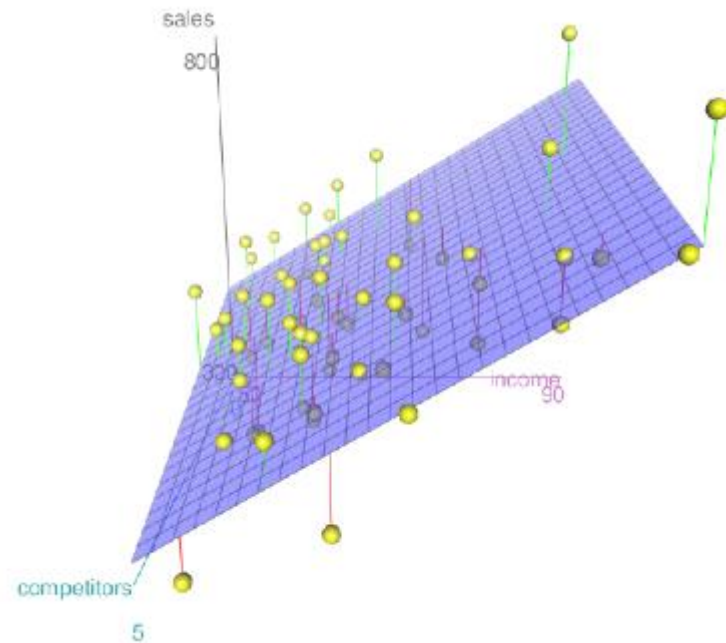
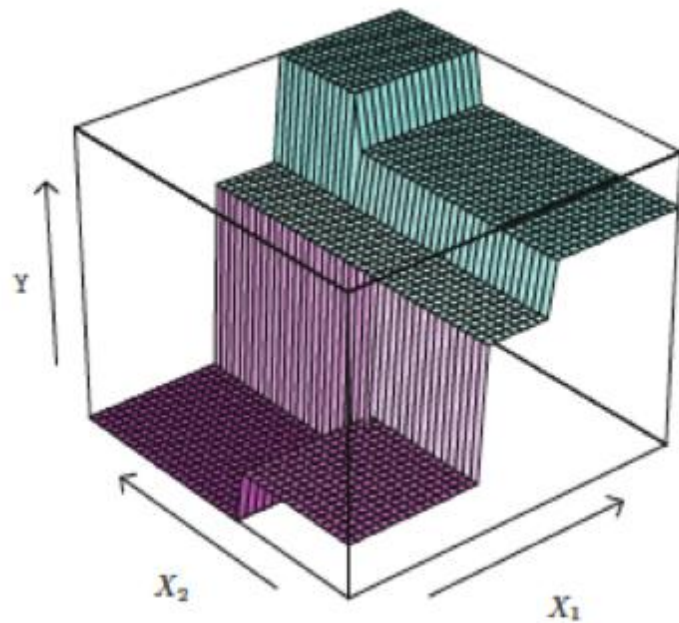
- Decision rule is highly transparent
 - Follows a series of if/else statements; it is even simpler than linear regression!
 - CART can be better explained to non-experts
 - Admits graphical displays through dendrograms (tree diagrams)
- CART may more closely mirror human thought and human decision-making processes (does not rely on linear additive rules)
- CART can uncover/learn nonlinear relationships
- CART can uncover interactions between variables without explicitly incorporating interaction terms
- CART selects the (most relevant) variables for us
 - no need to try adding/removing variables
 - no need to compute significance tests

CART Can Be Incredibly Interpretable

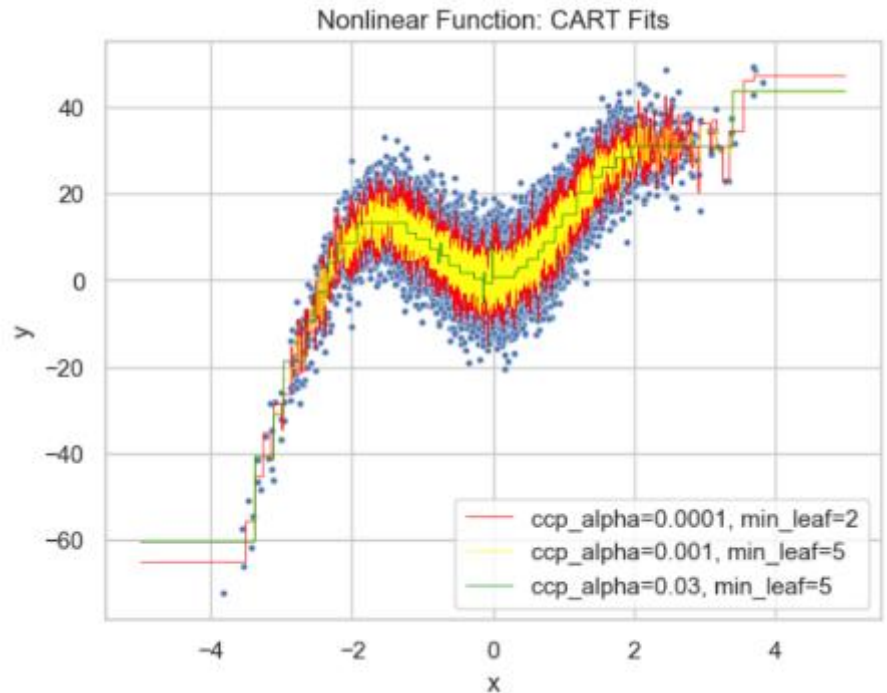
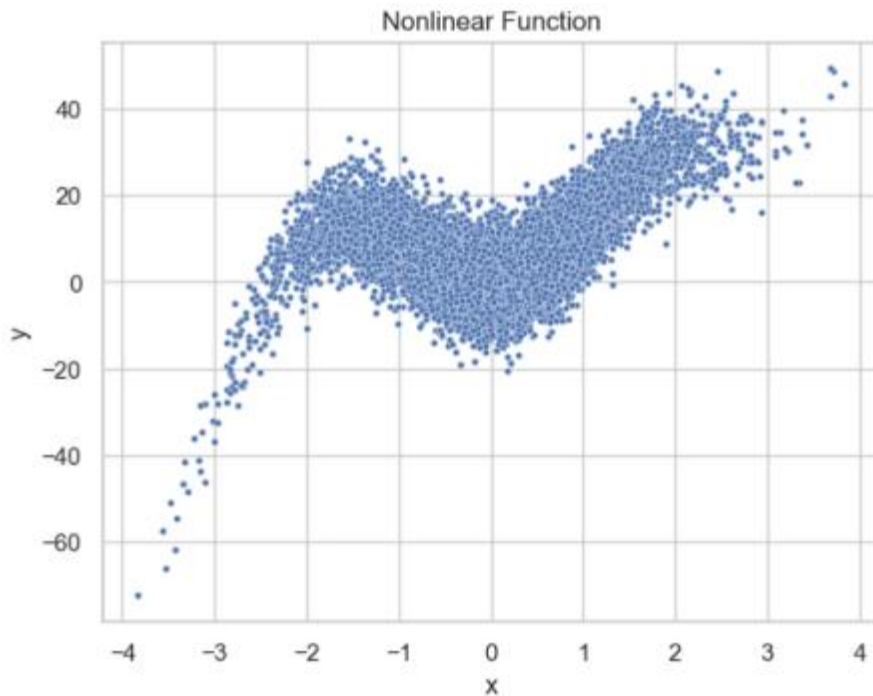
- Model predicts that an employee will receive a salary of \$60.701/hour if they has at least 2.710 years of experience
- For someone with less than 2.710 years of experience, if they had a GPA of 3.47 or less, they would earn \$6.47/hour.
- Otherwise, if they have a GPA of more than 3.47, they will earn \$31.175/hour



CART vs. Regression



Flexibility of CART



Building a CART Model

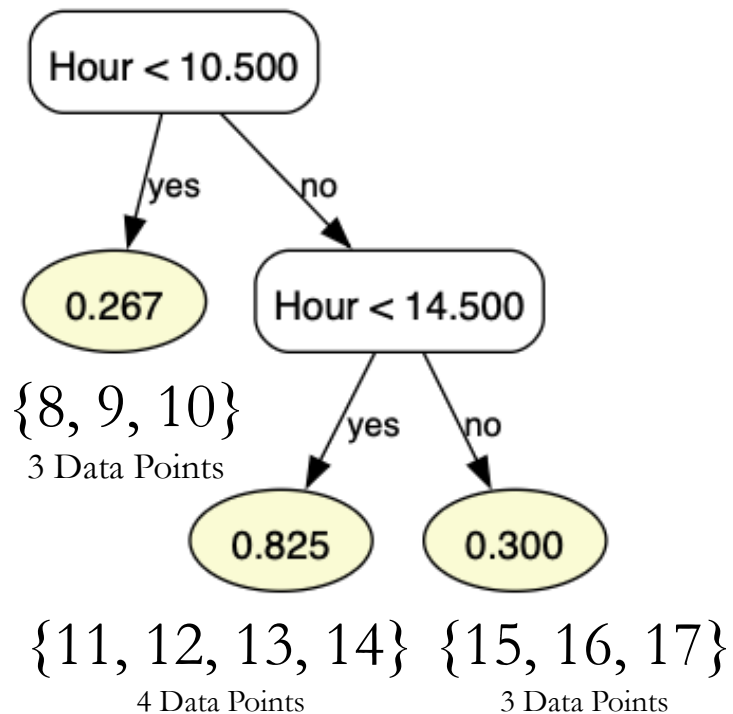
When to Stop? Controlling Model Complexity

Parameters We Control

- When we build a CART model, we can specify some parameters which control the complexity of the resulting tree.

Parameter Name in Python	English Name	Description	Default Value	Higher = Simpler Model?
<code>min_samples_split</code>	Minimum Split Size	CART will only consider splitting a node at all if its size is at least <code>min_samples_split</code> .	2	Yes
<code>min_samples_leaf</code>	Minimum Bucket Size	CART only considers splits if each leaf has <code>min_samples_leaf</code> data points at least.	1	Yes
<code>max_depth</code>	Maximum Depth	CART will build a tree no deeper than <code>max_depth</code> .	None	No
<code>ccp_alpha</code>	Complexity Parameter	CART removes a branch if it reduces impurity (MSE or SSE) by less than <code>ccp_alpha</code> $\times$ (# leaves - 1).	0.0	Yes

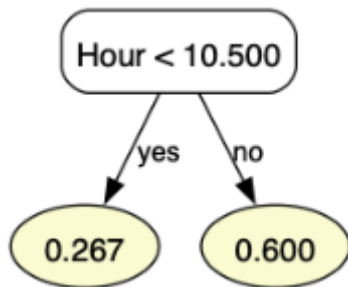
min_samples_leaf and min_samples_split



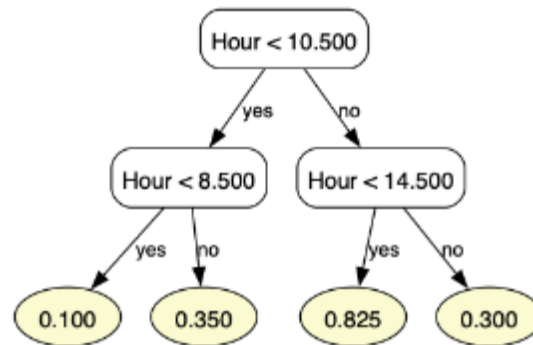
- Suppose
 $\text{min_samples_leaf} = 2$ and
 $\text{min_samples_split} = 4$
- Do not consider splitting $\{8, 9, 10\}$ or $\{15, 16, 17\}$ because they have fewer than 4 data points.
- Only consider splits of $\{11, 12, 13, 14\}$ where each resulting bucket has at least 2 data points.
 - E.g., do not consider the split (hour < 11.5) which would give $\{11\}$ and $\{12, 13, 14, 15\}$

max_depth

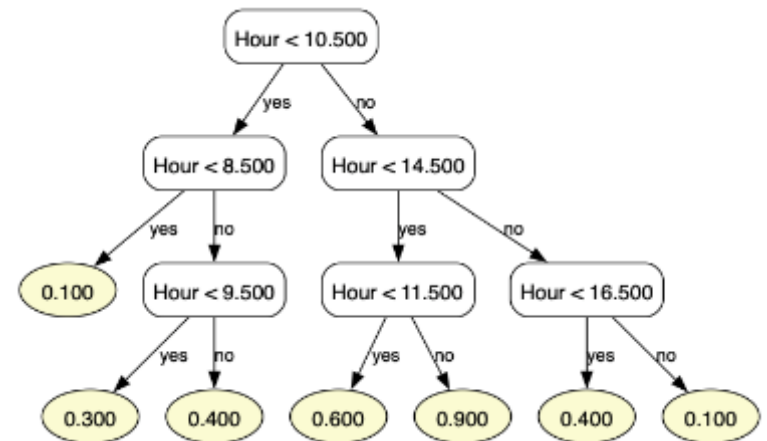
- Depth of a tree is the length of the longest path from the root to a leaf.
- Splits are only considered if the resulting tree has depth no more than maxdepth.



Depth = 1



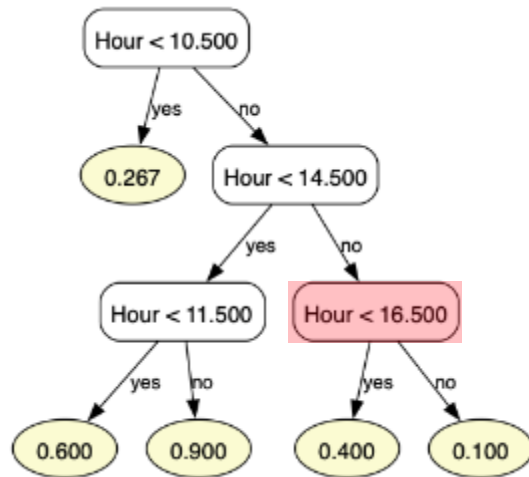
Depth = 2



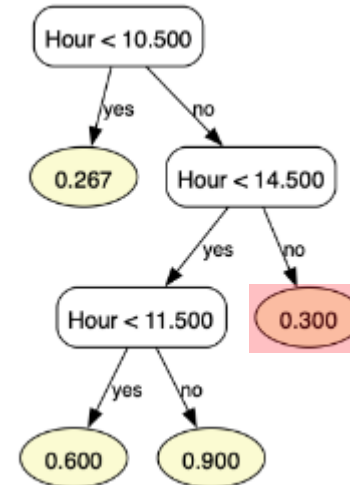
Depth = 3

ccp_alpha (cost-complexity pruning)

- CART removes a branch if it reduces impurity (MSE) by less than $\text{ccp_alpha} \times (\text{its number of leaves} - 1)$.



Tree A: $\text{ccp_alpha} = 0.059$
Impurity = MSE = 0.00867



Tree B: $\text{ccp_alpha} = 0.061$
Impurity = MSE = 0.01467

Here, if ccp_alpha is larger than 0.06, then this split would be removed (since number of leaves - 1 = 1).

Parameters We Control

- When we build a CART model, we can specify some parameters which control the complexity of the resulting tree.

Parameter Name in Python	English Name	Description	Default Value	Higher = Simpler Model?
<code>min_samples_split</code>	Minimum Split Size	CART will only consider splitting a node at all if its size is at least <code>min_samples_split</code> .	2	
<code>min_samples_leaf</code>	Minimum Bucket Size	CART only considers splits if each leaf has <code>min_samples_leaf</code> data points at least.	1	
<code>max_depth</code>	Maximum Depth	CART will build a tree no deeper than <code>max_depth</code> .	None	
<code>ccp_alpha</code>	Complexity Parameter	CART removes a branch if it reduces impurity (MSE or SSE) by less than $\text{ccp_alpha} \times (\# \text{ leaves} - 1)$.	0.0	

Parameters We Control

- When we build a CART model, we can specify some parameters which control the complexity of the resulting tree.

Parameter Name in Python	English Name	Description	Default Value	Higher = Simpler Model?
<code>min_samples_split</code>	Minimum Split Size	CART will only consider splitting a node at all if its size is at least <code>min_samples_split</code> .	2	Yes
<code>min_samples_leaf</code>	Minimum Bucket Size	CART only considers splits if each leaf has <code>min_samples_leaf</code> data points at least.	1	
<code>max_depth</code>	Maximum Depth	CART will build a tree no deeper than <code>max_depth</code> .	None	
<code>ccp_alpha</code>	Complexity Parameter	CART removes a branch if it reduces impurity (MSE or SSE) by less than <code>ccp_alpha</code> $\times$ ($\#$ leaves $- 1$).	0.0	

Parameters We Control

- When we build a CART model, we can specify some parameters which control the complexity of the resulting tree.

Parameter Name in Python	English Name	Description	Default Value	Higher = Simpler Model?
<code>min_samples_split</code>	Minimum Split Size	CART will only consider splitting a node at all if its size is at least <code>min_samples_split</code> .	2	Yes
<code>min_samples_leaf</code>	Minimum Bucket Size	CART only considers splits if each leaf has <code>min_samples_leaf</code> data points at least.	1	Yes
<code>max_depth</code>	Maximum Depth	CART will build a tree no deeper than <code>max_depth</code> .	None	
<code>ccp_alpha</code>	Complexity Parameter	CART removes a branch if it reduces impurity (MSE or SSE) by less than <code>ccp_alpha</code> $\times$ ($\#$ leaves $- 1$).	0.0	

Parameters We Control

- When we build a CART model, we can specify some parameters which control the complexity of the resulting tree.

Parameter Name in Python	English Name	Description	Default Value	Higher = Simpler Model?
<code>min_samples_split</code>	Minimum Split Size	CART will only consider splitting a node at all if its size is at least <code>min_samples_split</code> .	2	Yes
<code>min_samples_leaf</code>	Minimum Bucket Size	CART only considers splits if each leaf has <code>min_samples_leaf</code> data points at least.	1	Yes
<code>max_depth</code>	Maximum Depth	CART will build a tree no deeper than <code>max_depth</code> .	None	No
<code>ccp_alpha</code>	Complexity Parameter	CART removes a branch if it reduces impurity (MSE or SSE) by less than <code>ccp_alpha</code> $\times$ ($\#$ leaves $- 1$).	0.0	

Parameters We Control

- When we build a CART model, we can specify some parameters which control the complexity of the resulting tree.

Parameter Name in Python	English Name	Description	Default Value	Higher = Simpler Model?
<code>min_samples_split</code>	Minimum Split Size	CART will only consider splitting a node at all if its size is at least <code>min_samples_split</code> .	2	Yes
<code>min_samples_leaf</code>	Minimum Bucket Size	CART only considers splits if each leaf has <code>min_samples_leaf</code> data points at least.	1	Yes
<code>max_depth</code>	Maximum Depth	CART will build a tree no deeper than <code>max_depth</code> .	None	No
<code>ccp_alpha</code>	Complexity Parameter	CART removes a branch if it reduces impurity (MSE or SSE) by less than <code>ccp_alpha</code> $\times$ (# leaves - 1).	0.0	Yes

Python Code for CART

To create a CART model:

```
model = tree.DecisionTreeRegressor(  
    ccp_alpha = 0.01,  
    min_samples_leaf = 2,  
    min_samples_split = 5  
)
```

To fit a CART model:

```
model.fit(X_train, y_train)
```

To plot a CART tree:

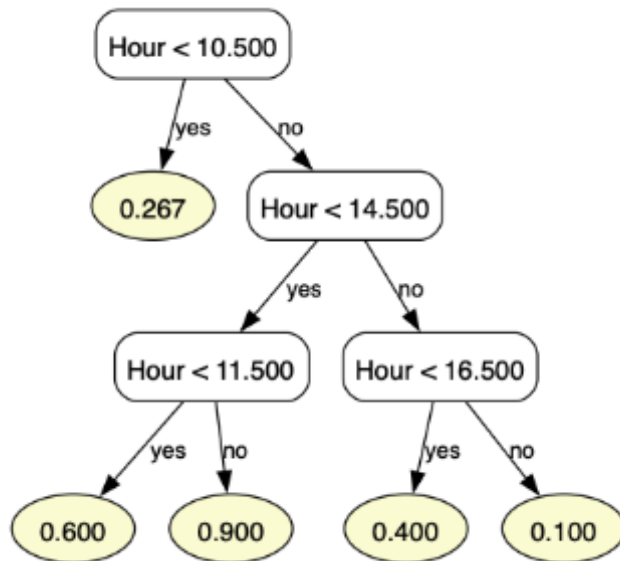
```
plot_tree(model)
```

To predict with a CART tree:

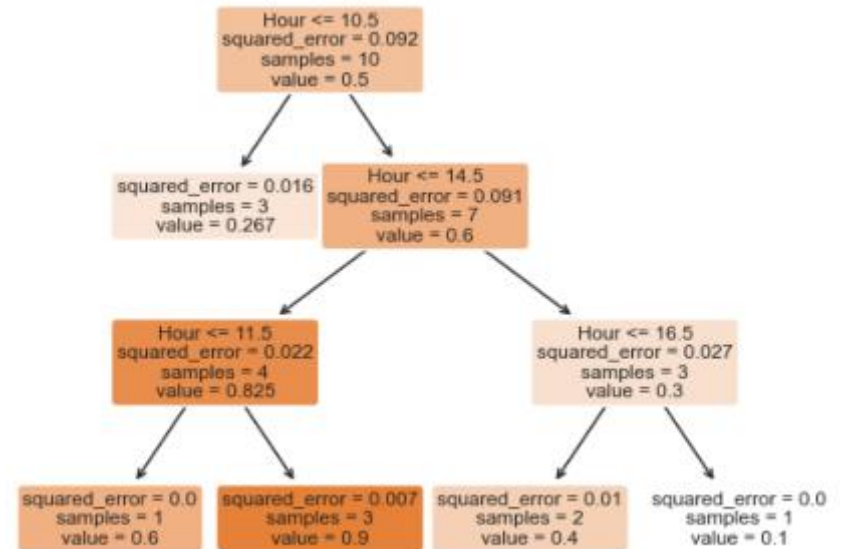
```
model.predict(X_test)
```

Python Code for CART

CART model tree:



In Python, using sklearn:



Back to the Healthcare Claims Dataset

Back to the Healthcare Claims Data Training Set

k : # of independent variables ($k = 27$); n : # of observations ($n = 716,983$)

	log10_reimb2010	log10_reimb2008	log10_reimb2009	age2010	male	race	heart.failure	kidney	cancer
0	2.592177	2.506505	2.557507	97	0	White	0	0	0
1	2.987219	4.769385	3.437909	79	0	White	1	1	0
3	3.541704	3.467016	4.693120	79	0	White	1	1	0
5	3.049606	3.057286	3.385785	90	0	Hispanic	1	0	0
6	3.061075	2.654177	4.175541	47	1	White	1	1	0
7	0.000000	0.000000	1.908485	73	0	White	0	0	0
9	0.000000	0.000000	0.000000	85	0	Block	0	0	0
10	0.000000	0.000000	0.000000	37	1	White	0	0	0
...									
1023946	2.806858	2.506505	3.009026	74	0	White	0	0	0
1023947	3.820924	3.320354	3.609701	54	1	Block	1	0	0
1023951	3.209783	3.619198	4.266960	67	0	White	1	1	0

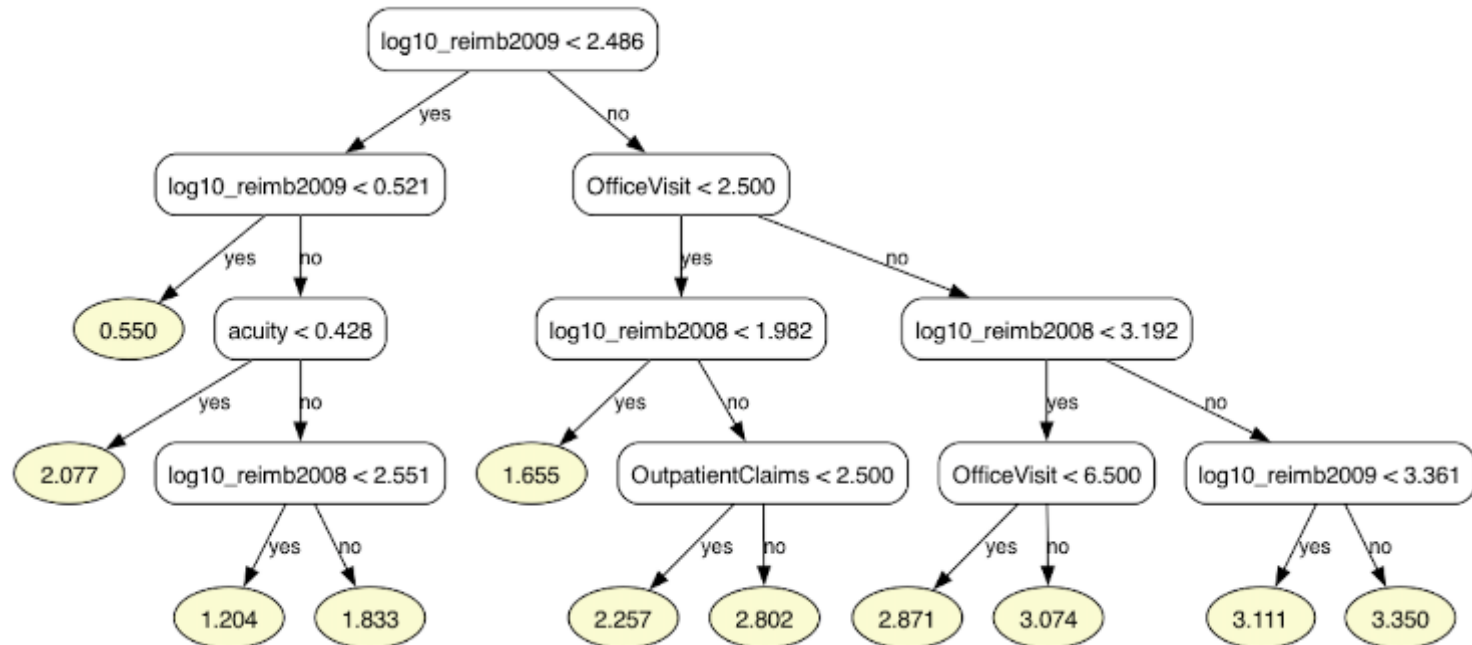
$n = 716,983$

CART Model Parameters for Predicting 2010 Healthcare Costs

- Here is a CART model using:
 - `min_samples_leaf` = 25
 - `ccp_alpha` = 0.002
- Only use 5/27 variables and 11 leaves

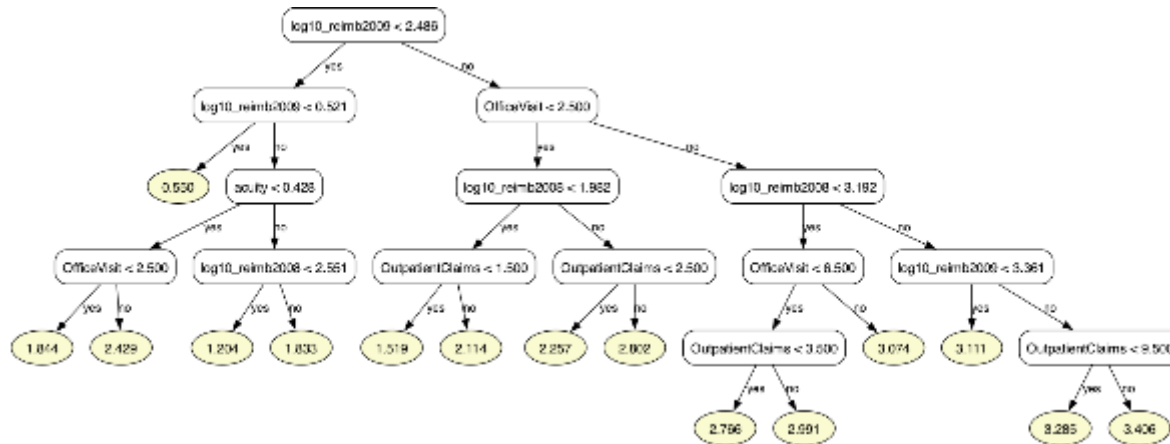
Model Performance

- Training set: $R^2 = 0.599$
- Test set: $OSR^2 = 0.597$



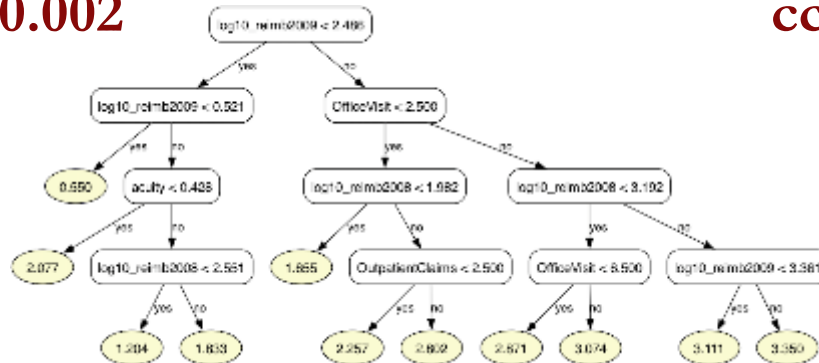
Impact of `ccp_alpha` value on the tree size

ccp=0.001



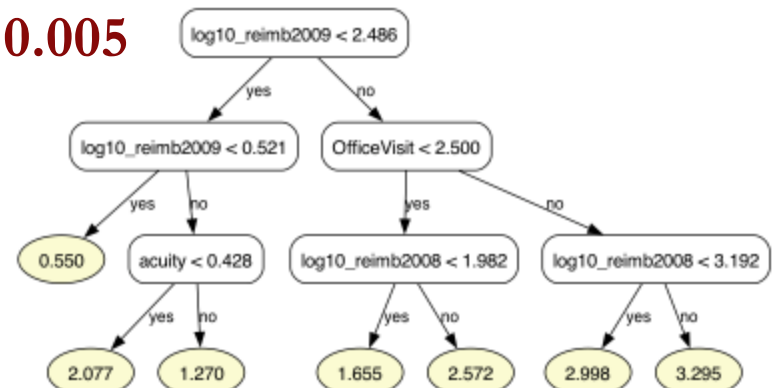
15 leaves

ccp=0.002



11 leaves

ccp=0.005



7 leaves

Model Parameters

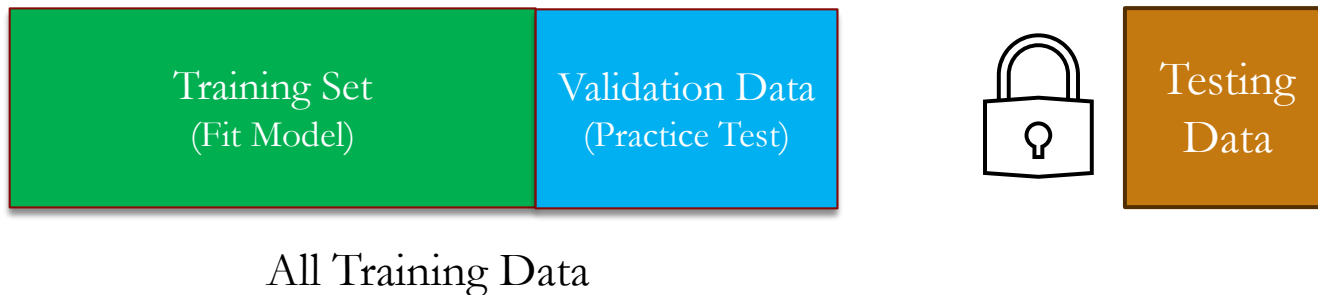
- Model parameters control how the model fit to the data
 - A **setting** chosen by us (the human), not the model itself.
 - Parameters often control some kind of trade-off, like model complexity.
- CART
 - `ccp_alpha`, `min_samples_leaf`, `min_samples_split`, `max_depth` are “parameters”
 - Dictate how CART decides when to split and when to stop splitting
- How to decide these parameter values?

How to decide parameter values?

- Ideally, choose parameters that deliver best performance for the model
- In CART for regression, performance metric is R^2 .
- Directly choosing the parameters that give us the best in-sample R^2 ?
- The “best” may be due to randomness in the training set.
- We do not want to touch the test set when choosing the best parameters.
- Further splitting the training set?

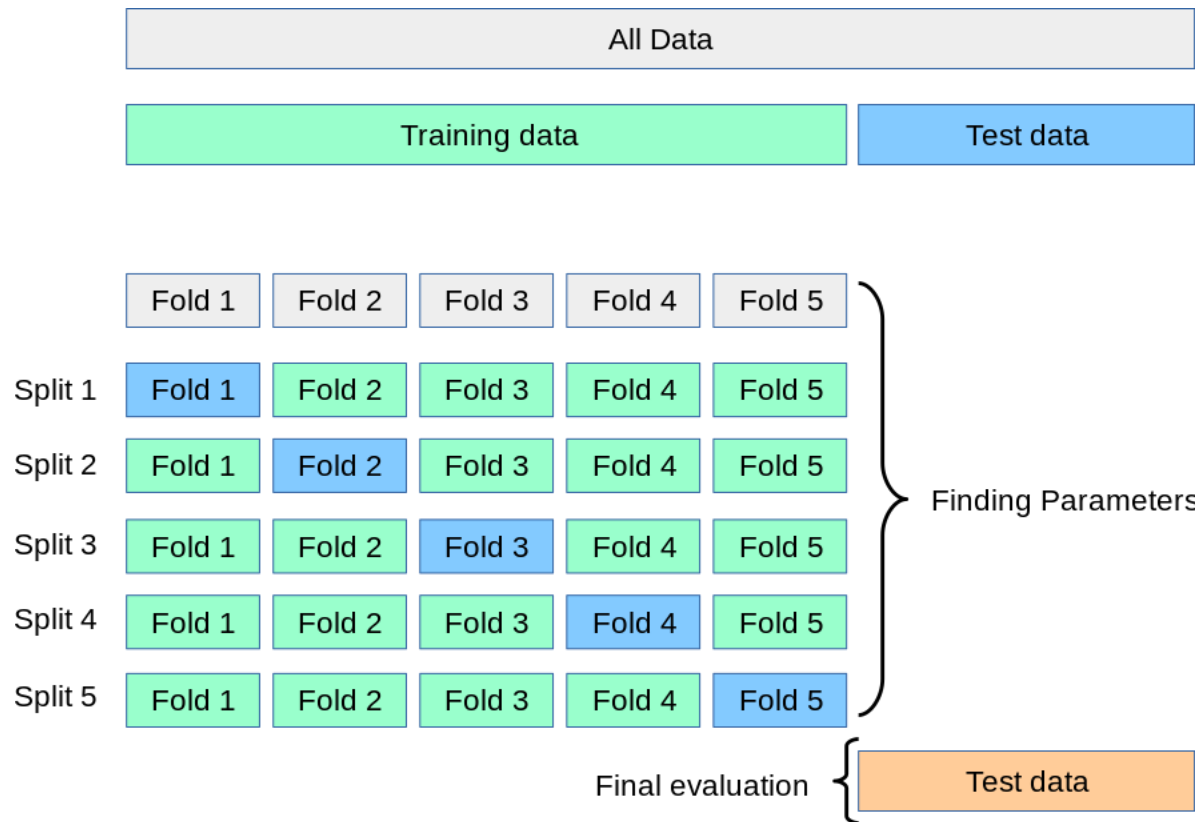
Cross-Validation

- General methodology for finding the “best” parameters for out-of-sample performance without looking at the test set.
 - **Idea:** Give yourself a “practice test”, called a validation set.

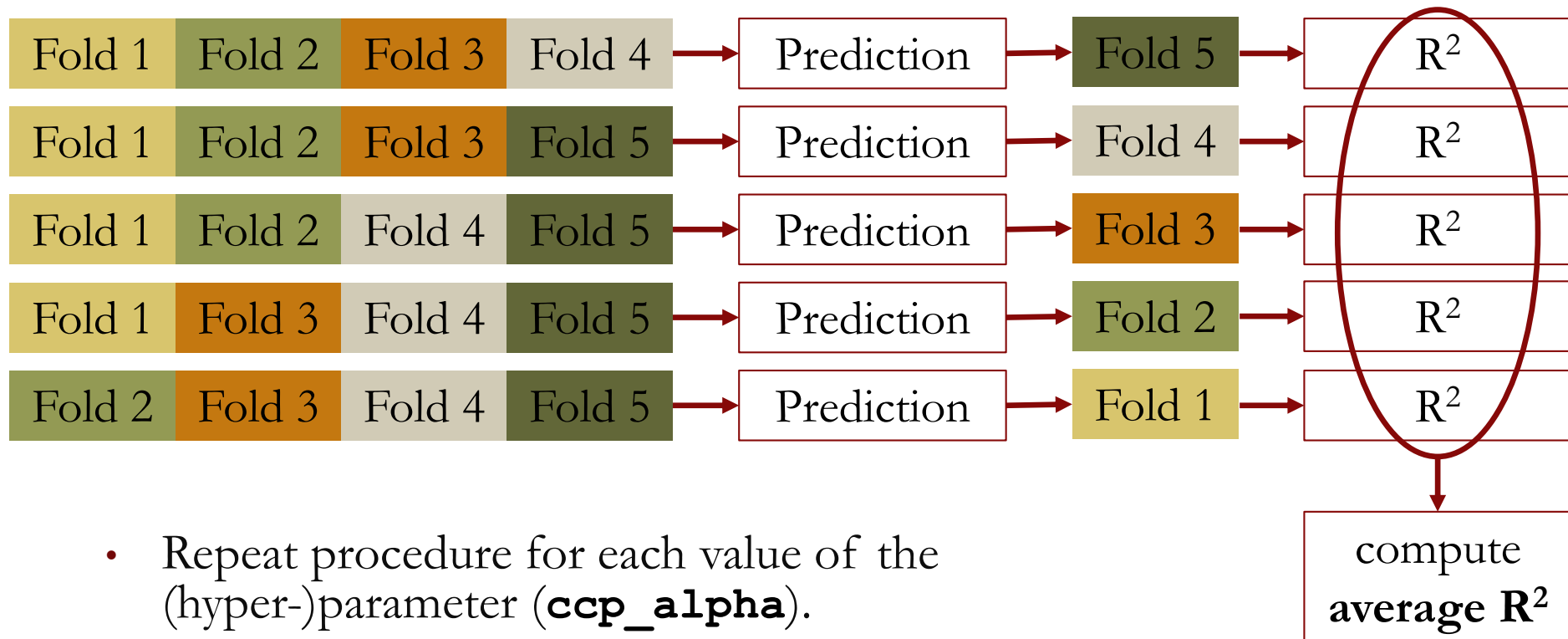


- **Issue:** If you only take one practice test, there is variation.
- **Solution:** K-Fold Cross Validation. For each choice of parameter, run K practice tests.

5-Fold Cross Validation



5-Fold Cross-Validation



- Repeat procedure for each value of the (hyper-)parameter (**ccp_alpha**).
- Choose the hyper-parameter value that maximizes the **average R^2**

k -fold Cross-Validation

Divide

- Divide the training set into k pieces (“folds”)

Fold 1 Fold 2 Fold 3 Fold 4 Fold 5

Train / Test

- For each candidate parameter value...
 - Use $k - 1$ folds to construct a model
 - Test that model on the remaining fold
 - Repeat for each of the k folds

Average

- For each candidate parameter value...
 - compute the average loss over the k folds

Select

- Finally, select the parameter value that maximizes the **average R^2**
 - Uses the entire training set but nothing from the test set

Python Code

```
from sklearn.model_selection import GridSearchCV
from sklearn import tree

# Define the parameter grid
param_grid = {
    'min_samples_leaf': [40, 45, 50, 55, 60, 65],
    'ccp_alpha': [0.000005, 0.00001, 0.00003, 0.00005]
}

# Initialize the Decision Tree Regressor model
cart_model = tree.DecisionTreeRegressor(random_state=123)

# Initialize GridSearchCV with 5-fold cross-validation and R-squared scoring
grid_search = GridSearchCV(
    estimator = cart_model,
    param_grid = param_grid,
    cv = 5, # 5-fold cross-validation
    scoring = 'r2', # Use R-squared as the scoring metric for CV
)

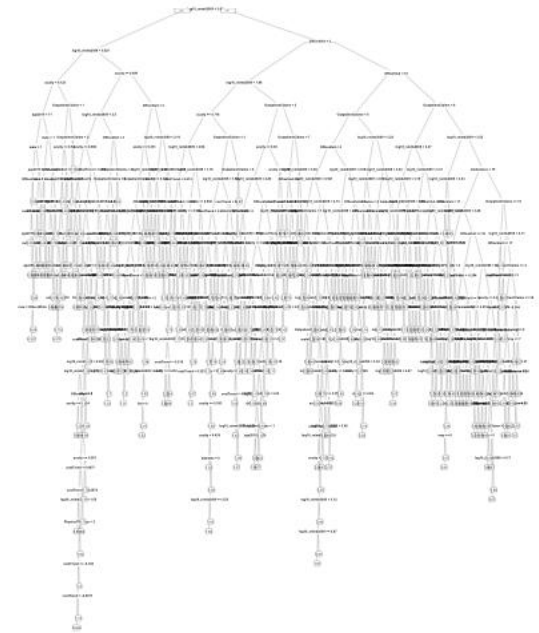
# Fit GridSearchCV to the training data
grid_search.fit(X_train_enc, y_train)

# Get the best estimator
best_cart_model = grid_search.best_estimator_
```

Comparison of Models

Model	Linear Regression	CART (ccp=0.002)	CART (ccp=0.00001)
Fit (R^2)	0.604	0.599	0.631
Predictive Performance (OSR^2)	✓ 0.602	✓ 0.597	✓ 0.610
Interpretability	✗ Linear equation	✓ 11 intuitive rules	✗ Too many rules
Simplicity/Complexity	✗ 21 significant variables	✓ 5 variables	✗ Too many variables

CART tree with
`min_samples_leaf = 50`
`ccp_alpha = 0.00001`

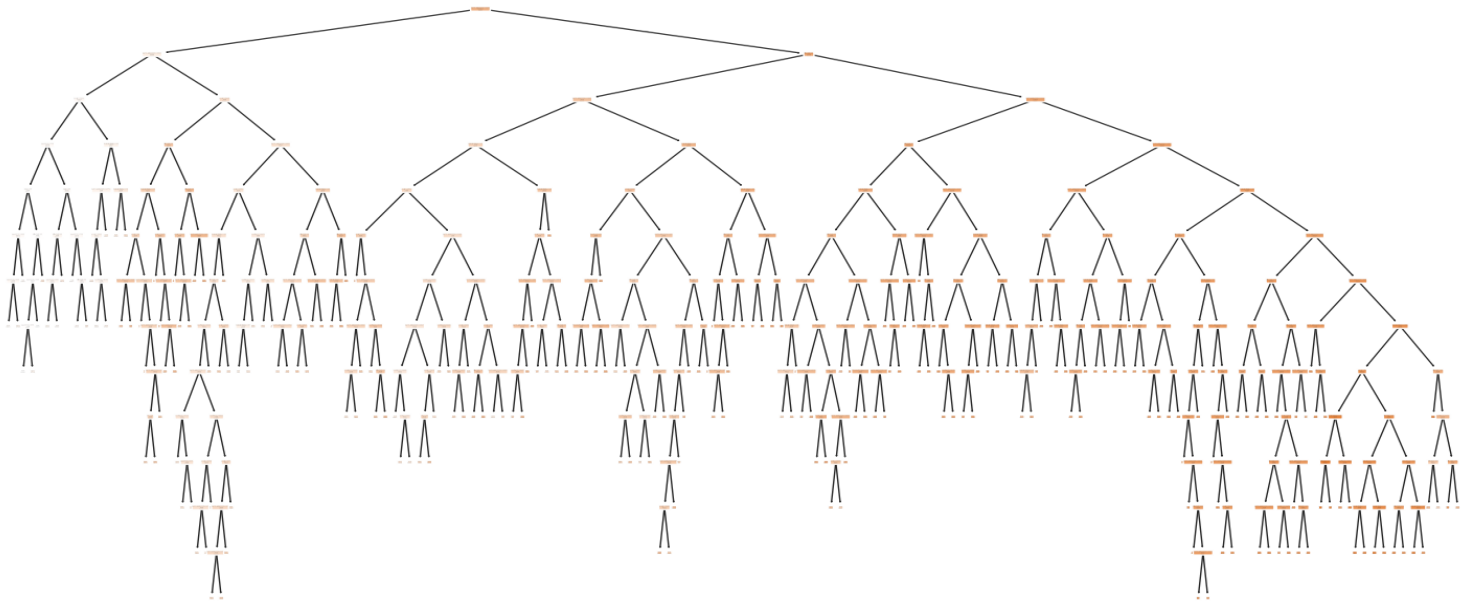


Comparison of Models

Model	Linear Regression	CART (ccp=0.002)	CART (ccp=0.00001)	From Cross Validation (ccp=0.00003, min_samples_leaf=45)
Fit (R^2)	0.604	0.599	0.631	0.619
Predictive Performance (OSR^2)	✓ 0.602	✓ 0.597	✓ 0.610	✓ 0.613
Interpretability	✗ Linear equation	✓ 11 intuitive rules	✗ Too many rules	✗ Too many rules
Simplicity/Complexity	✗ 21 significant variables	✓ 5 variables	✗ Too many variables	✗ Too many variables

Trade off

- Performance (OSR2) and Interpretability



CART for Solar Radiation Prediction

CART for Solar Radiation Prediction

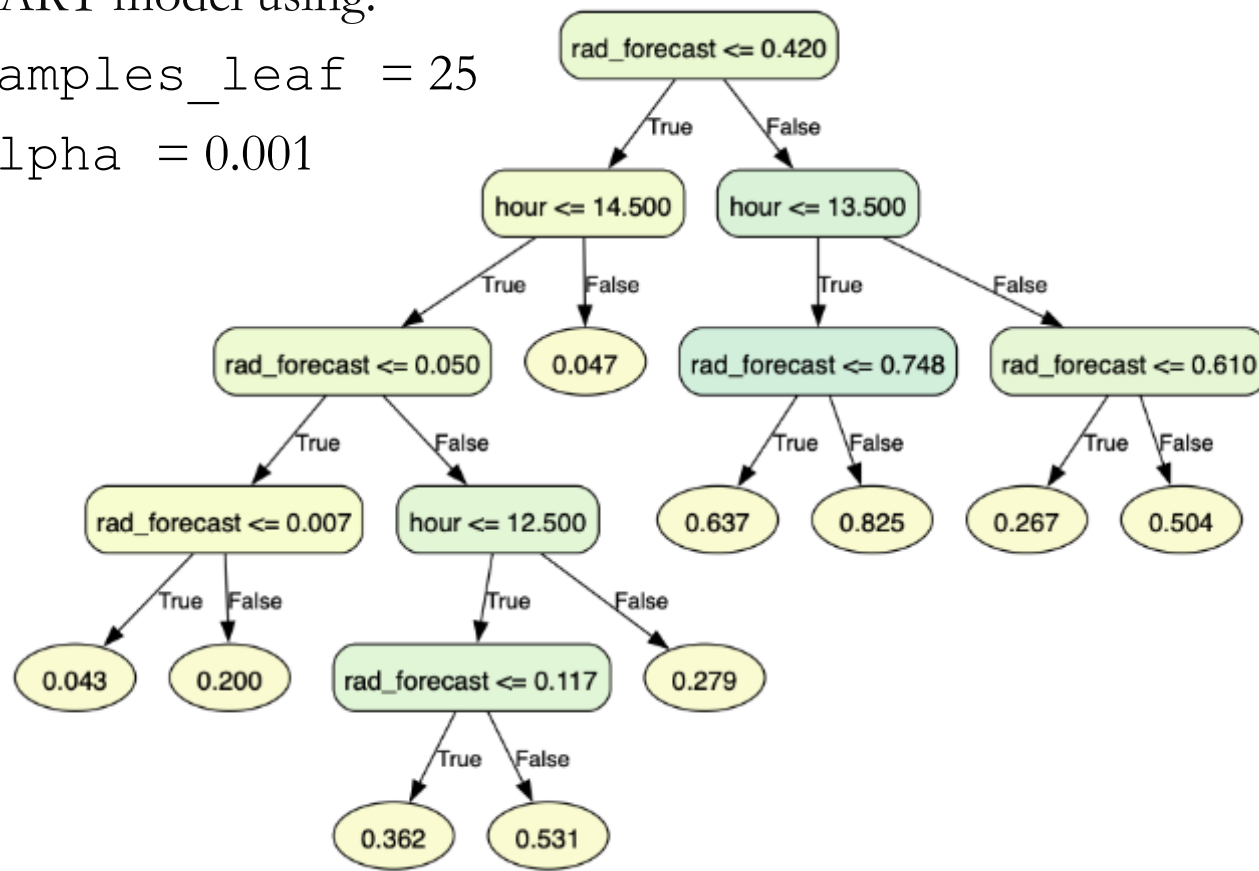
k : # of independent variables ($k = 8$); n : # of observations ($n = 8461$)



	datetime	rad_actual	rad_forecast	temp	humidity	wind_u	wind_v	cloud_cover	month	hour
0	2021-04-02 05:00:00	0.0008	0.0	9.808496	74.800003	-0.919819	-4.182371	16.700001	4	5
1	2021-04-02 05:30:00	0.0339	0.00286	9.781213	75.900002	-1.034073	-4.163641	14.800001	4	5
2	2021-04-02 06:00:00	0.1173	0.00572	9.753931	77.0	-1.148326	-4.144912	12.900001	4	6
3	2021-04-02 06:30:00	0.2312	0.02689	10.15394	76.0	-1.35613	-4.19131	28.300001	4	6
4	2021-04-02 07:00:00	0.2898	0.04806	10.553949	75.0	-1.563933	-4.237708	43.700001	4	7
5	2021-04-02 07:30:00	0.4046	0.08219	10.883157	73.25	-1.750634	-4.420197	71.85	4	7
6	2021-04-02 08:00:00	0.4303	0.11632	11.212366	71.5	-1.937335	-4.602685	100.0	4	8
...	...	...	...	...	...	...	...	...	...	...
8458	2022-01-31 17:30:00	0.0	0.11187	2.963556	55.200001	5.902472	-4.098351	4.65	1	17
8459	2022-01-31 18:00:00	0.0	0.0895	2.777094	55.0	5.642742	-3.866916	4.4	1	18
8460	2022-01-31 18:30:00	0.0	0.07831	2.535776	55.15	5.504078	-3.453133	17.75	1	18

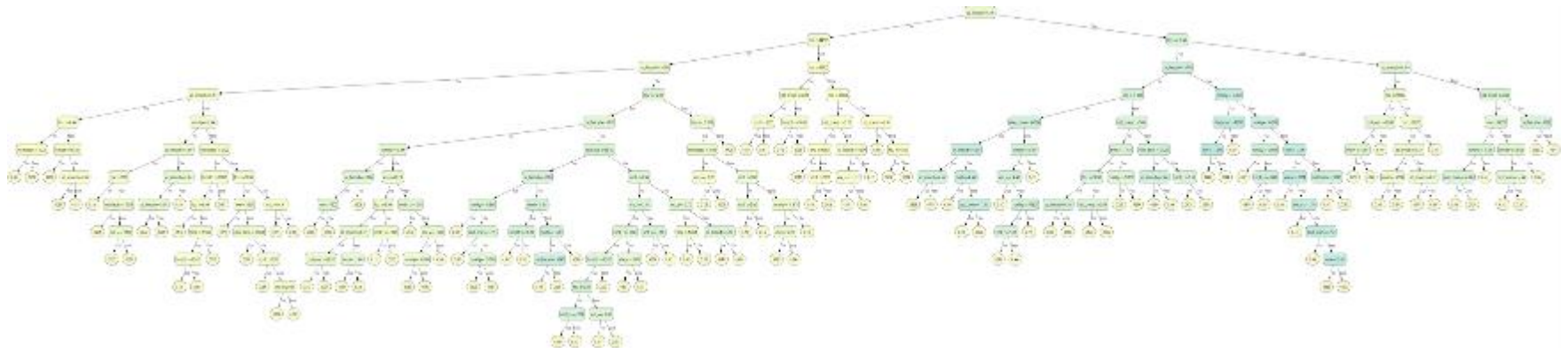
CART for Solar Radiation Prediction

- Here is a CART model using:
 - `min_samples_leaf = 25`
 - `ccp_alpha = 0.001`



CART for Solar Power Prediction

- Here is a CART model using:
 - `min_samples_leaf = 25`
 - `ccp_alpha = 0.00001`



CART vs. Linear Regression for Solar Power Prediction

Model	Linear Regression	CART (ccp=0.001)	CART (ccp=0.00001)
Fit (R^2)	0.73 ✓	0.70 ✓	0.81 ✓
Predictive Performance (OSR^2)	0.73 ✓	0.69 ✓	0.77 ✓
Interpretability	Linear regression ~	Tree ✓	Huge Tree ✗
Simplicity/ Complexity	8 ~ significant variables	3 ~ variables in all	✗

CART Model Limitations

- CART predictions can be “jumpy”
 - Consider a patient with $\log_{10}\text{reimb2009} = 2.6$ and $\text{OfficeVisit} = 2$
 - If $\log_{10}\text{reimb2008} = 1.98$, then we predict $\log_{10}\text{reimb2010} = 1.655$
 - If $\log_{10}\text{reimb2008} = 1.99$, then we predict $\log_{10}\text{reimb2010} = 2.572$
 - Oh, and if $\log_{10}\text{reimb2009} = 2.4$, then all this would not matter anyway, and we would predict 1.270 or 2.077 based on “acuity”.
- A CART model can exhibit instability
 - Fitting a CART model on two similar non-identical datasets may lead to different types of splits and cutoff values
- A CART model (or any model for that matter) may not necessarily lead to the strongest out-of-sample predictive performance—depending on the structure of the data and the problem context

Conclusion on CART for Regression

- Impact on Predicting Healthcare Costs
 - Using CART, D2Hawkeye substantially improved their ability to predict healthcare costs in a wide variety of settings
 - Analytics gave D2Hawkeye a critical edge over competitors who were using more traditional methods
- Benefits of CART: predictive power, interpretability and flexibility to balance performance and simplicity
- Basis for ensemble models (next class)



PREDICTING PAROLE VIOLATIONS

CART for Classification

Lecture Outline

1. Parole and Parole Decisions

- What is parole and how can analytics help make decisions?
- What is a loss matrix?

2. CART for Classification

- How is CART for classification different than CART for regression?

3. Overfitting and Cross Validation

- Why are complex trees more likely to overfit?
- Use cross validation to select “optimal” cp values without looking at the test set.

1. Parole and Parole Decisions

What is parole and how can analytics help make decisions?

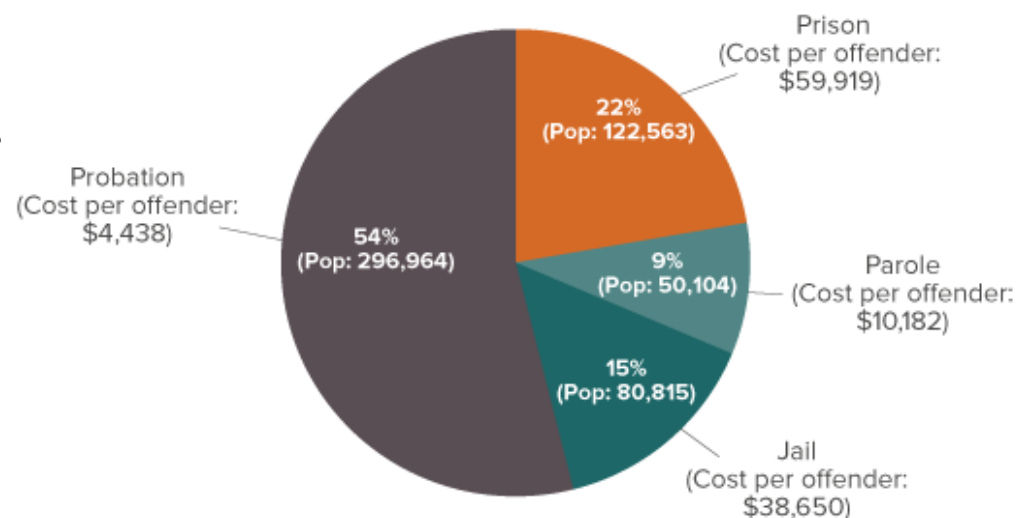
What is a loss matrix?

What is Parole?



Parole: release of a prison inmate before the completion of their sentence, on the promise/anticipation of good behavior

- Provides flexibility in the criminal justice system to enhance justice, ensure equity, incentivize good behaviors, and empower parolees to contribute to society
- Manage prison populations due to overcrowding
- Saves money in the prison system



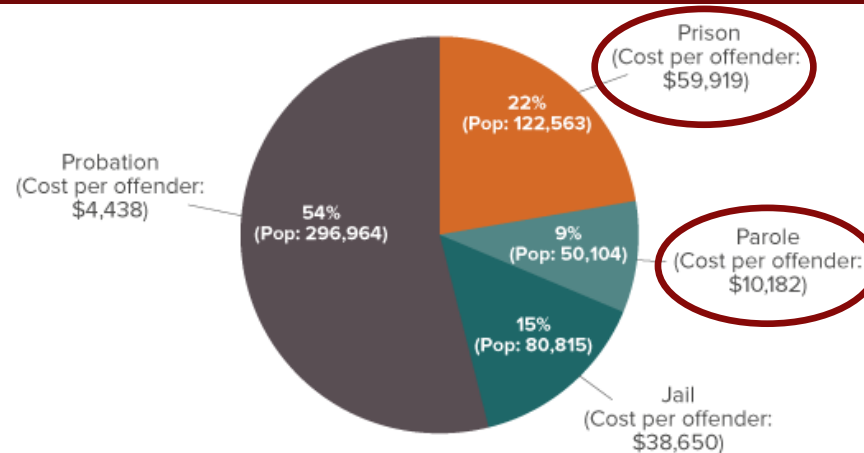
Prison costs in California

Source: Public Policy Institute of California

Parole Decisions

- What are the factors underlying parole decisions?
 - Grant parole to inmates who are likely to adhere to parole terms
 - Do not grant parole to inmates who are likely to violate parole terms
- Two types of possible errors in parole decisions:
Y = 1 is if the person will violate parole. If prediction = 1, do not grant parole.
 - Granting parole to an inmate who will violate the terms of parole —“false negative”
 - Not granting parole to an inmate who would have adhered to the terms of parole —“false positive”
- What are the implications of these two possible errors?
- So ... how to make parole decisions?

Differential Losses of Parole Decision “Errors”



- Using the California data as a guide, let us set \$50,000 to be the cost of not granting parole to an inmate who would have adhered to the terms of parole (“false positive”)
- Let us set \$1,000,000 to be the cost of granting parole to an inmate who will violate the terms of parole (“false negative”)
- Rather than consider these as actual “cost” numbers, let us simply assert that the “losses” of a false negative decision are 20 times the “losses” of a false positive decision

The Loss Matrix

		Prediction/Decision	
		No Violation (Grant Parole)	Violation (Deny Parole)
Reality	No Violation	0	\$50,000
	Violation	\$1,000,000	0

- \$1,000,000 denotes the losses associated with a false negative
- \$50,000 denotes the losses associated with a false positive

The Loss Matrix, continued

		Prediction/Decision	
		No Violation (Grant Parole)	Violation (Deny Parole)
Reality	No Violation	0	1
	Violation	20	0

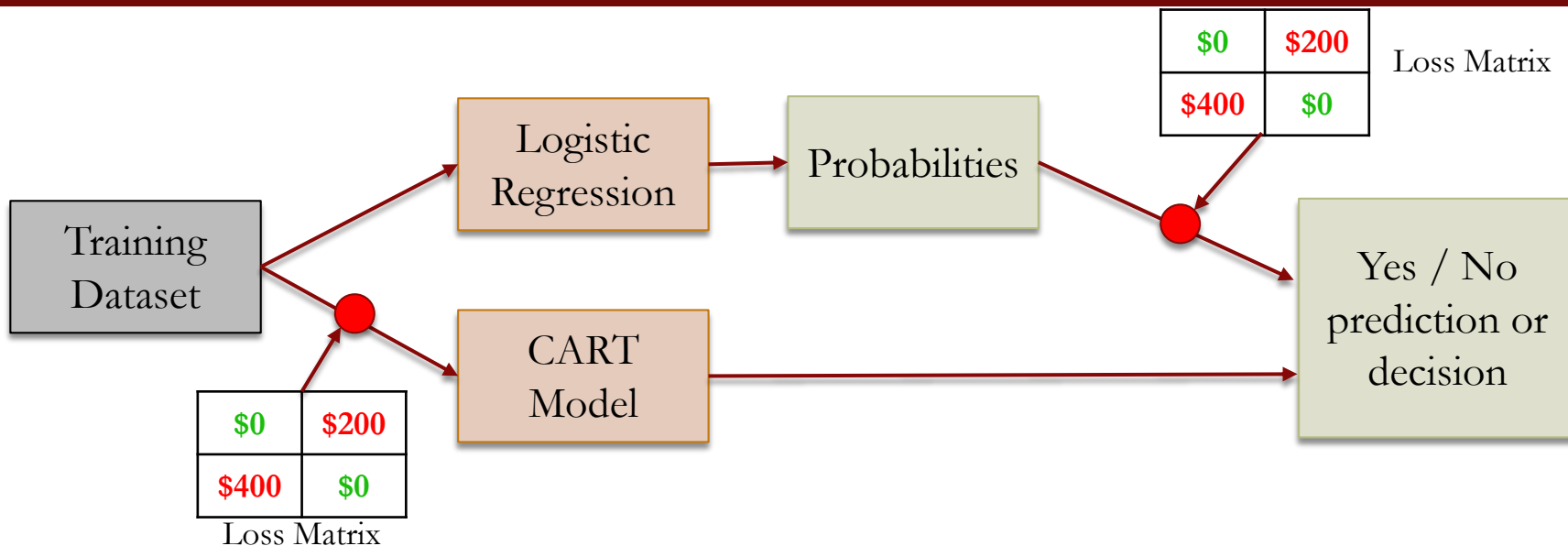
- 20 denotes the losses associated with a false negative
- 1 denotes the losses associated with a false positive

The Loss Matrix, continued

		Prediction/Decision	
		No Violation (Grant Parole)	Violation (Deny Parole)
Reality	No Violation	0	L_{FP}
	Violation	L_{FN}	0

- L_{FN} denotes the losses associated with a false negative
- L_{FP} denotes the losses associated with a false positive

Loss Matrices in Log. Reg. vs. CART



- A logistic regression model generates probabilities. Based on your cost matrix, you can choose the appropriate cutoff to predict Churn / No Churn.
 - The model itself is unaware of the cost matrix.
- In CART, we specify a cost matrix when we train the model. The model's splits are chosen specifically for that cost matrix. We'll see this shortly...

Analytics in Parole Decisions

- Parole decisions are made by a Parole Board
- Typical factors in a parole decision
 - Conduct in prison
 - Inmate interview
 - Psychological exam
- Increasing use of analytics for parole decisions
 - More objective decisions
 - More consistent outcomes
 - More fair decisions?



GCN The Technology that Drives Government IT
AI & Automation Cybersecurity Cloud & Infrastructure Data & Analytics
State & Local More ▾

SHARE... E-MAIL THIS PAGE PRINTABLE FORMAT



Prisons turn to analytics software for parole decisions

BY KEVIN MCCANEY | NOV 01, 2013

Parole Violation Data

Parole Violation Data

- 6,102 parolees from New York state from 2012-2015
- **Dependent Variable:**
 - Whether parole was revoked (1 if revoked, 0 if not revoked)
- **Independent Variables:**
 - Gender (1 if male, 0 if female)
 - Age of the parolee
 - Number of years served by the parolee
 - Class of the most serious conviction
 - A~Murder, ..., E~Theft of \$1,000-\$3,000
 - Was the parolee convicted of multiple crimes? (1 or 0)
 - Was the parolee convicted in New York City? (1 or 0)

Parole Violation Data

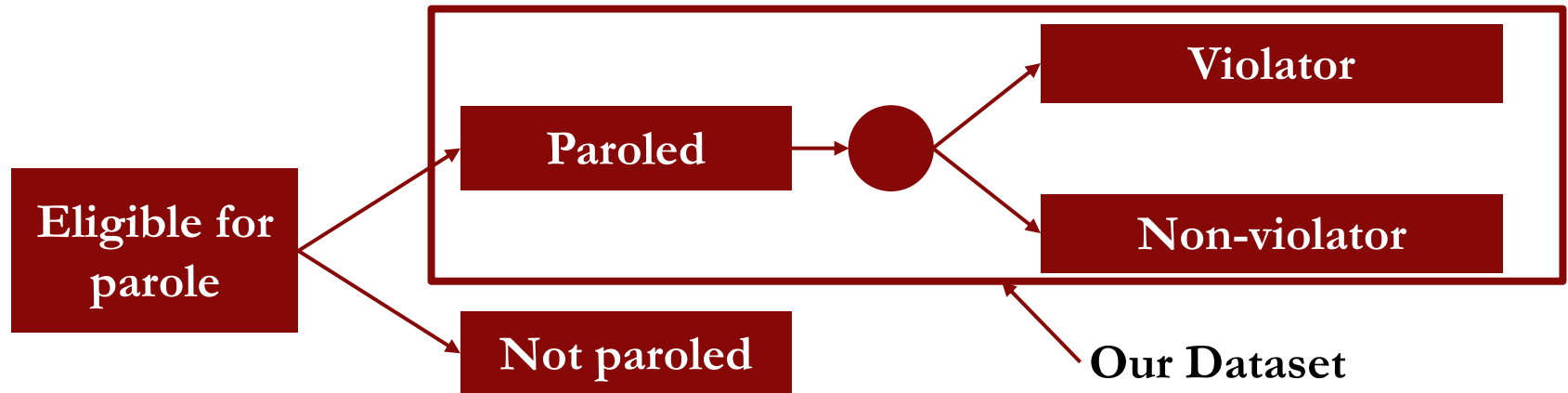
k : # of independent variables ($k = 6$); n : # of observations ($n = 6,102$)

$n = 6,102$

	Violator	Male	Age	TimeServed	Class	Multiple	InCity
0	0	1	49.4	3.15	D	0	1
1	1	1	26.0	5.95	D	1	0
2	0	1	24.9	2.25	D	1	0
3	0	1	52.1	29.22	A	0	0
4	0	1	35.9	12.78	A	1	1
...	...	...	...	...	...	...	...
6097	0	1	55.0	0.72	E	0	0
6098	0	1	49.6	29.88	A	0	1
6099	0	1	22.4	2.85	D	0	1
6100	0	1	44.8	1.76	D	1	0
6101	0	0	45.3	1.03	E	0	0

Parole Decision-Making Dataset

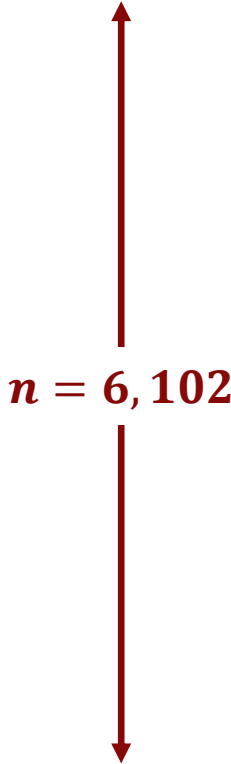
- 6,102 parolees from New York state from 2012-2015
- This is “censored” data on parolees — since we do not know the data on inmates that were not granted parole



→ How would you use a prediction model for decision support?

Parole Violation Data

k : # of independent variables ($k = 6$); n : # of observations ($n = 6,102$)



	Violator	Male	Age	TimeServed	Class	Multiple	InCity
0	0	1	49.4	3.15	D	0	1
1	1	1	26.0	5.95	D	1	0
2	0	1	24.9	2.25	D	1	0
3	0	1	52.1	29.22	A	0	0
4	0	1	35.9	12.78	A	1	1
...	...	...	...	...	...	...	...
6097	0	1	55.0	0.72	E	0	0
6098	0	1	49.6	29.88	A	0	1
6099	0	1	22.4	2.85	D	0	1
6100	0	1	44.8	1.76	D	1	0
6101	0	0	45.3	1.03	E	0	0

Training/Testing (stratified sampling)

Full Dataset ($n = 6,102$)

	Violator	Male	Age	TimeServed	Class	Multiple	InCity	
	0	0	1	49.4	3.15	D	0	1
	1	1	1	26.0	5.95	D	1	0
	2	0	1	24.9	2.25	D	1	0
	3	0	1	52.1	29.22	A	0	0
	4	0	1	35.9	12.78	A	1	1
	...	...	...	...	...	...	...	...
	6097	0	1	55.0	0.72	E	0	0
	6098	0	1	49.6	29.88	A	0	1
	6099	0	1	22.4	2.85	D	0	1
	6100	0	1	44.8	1.76	D	1	0
	6101	0	0	45.3	1.03	E	0	0

70% Training Set ($n = 4,271$)

	Violator	Male	Age	...	TimeServed	Multiple	InCity	
	0	0	1	49.4	...	3.15	0	1
	1	1	1	26.0	...	5.95	1	0
	2	0	1	24.9	...	2.25	1	0
	...	...	...	...	...	...	...	...
	6099	0	1	22.4	...	2.85	0	1
	6100	0	1	44.8	...	1.76	1	0
	6101	0	0	45.3	...	1.03	0	0

30% Testing Set ($n = 1,831$)

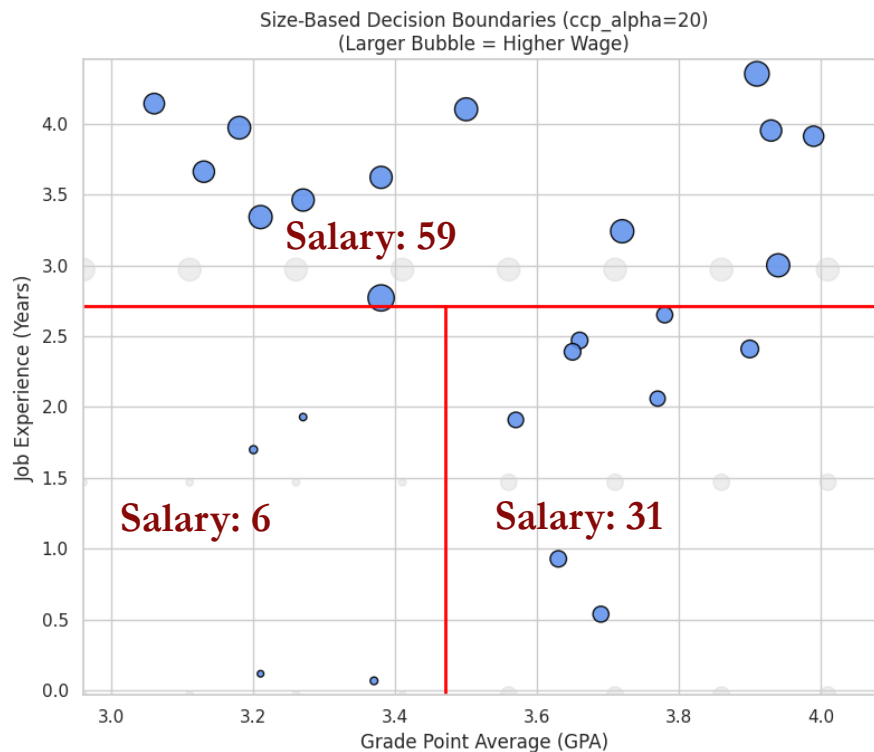
Violator	Male	Age	...	TimeServed	Multiple	InCity	
4	0	1	35.9	...	12.78	1	1
7	0	1	43.2	...	1.07	0	1
12	0	1	36.1	...	3.71	0	1
...	...	...	...	...	...	...	...
6092	0	1	47.2	...	19.57	1	1
6097	0	1	55.0	...	0.72	0	0
6098	0	1	49.6	...	29.88	0	1

2. CART for Classification

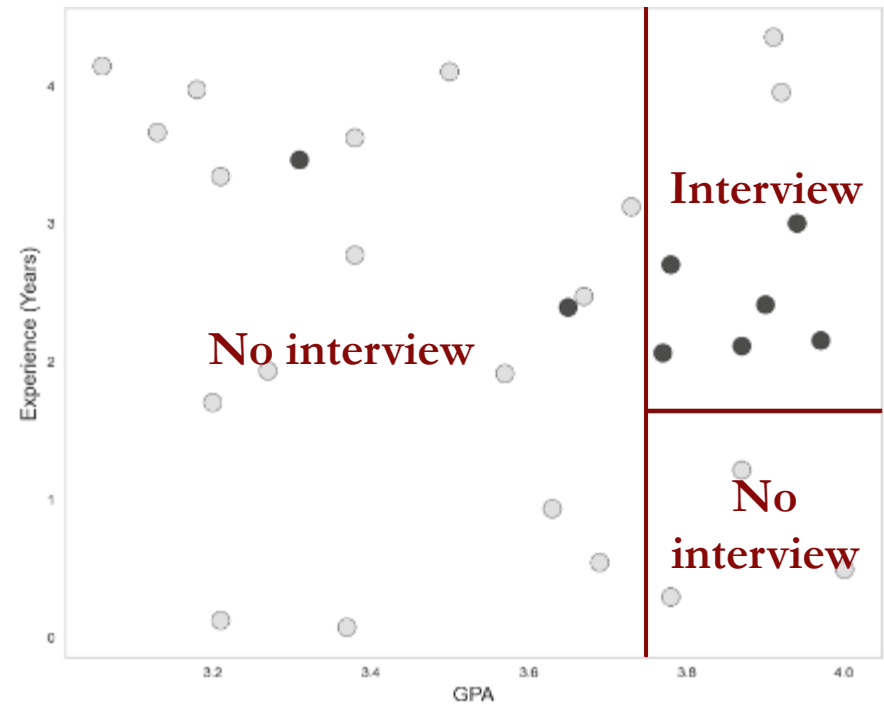
How is CART for classification different than CART for regression?

CART for Regression vs. Classification

Regression: Salary (\$/hour)

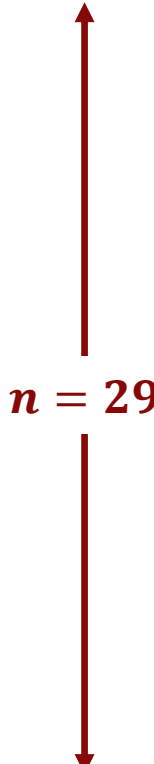


Classification: Interview or not



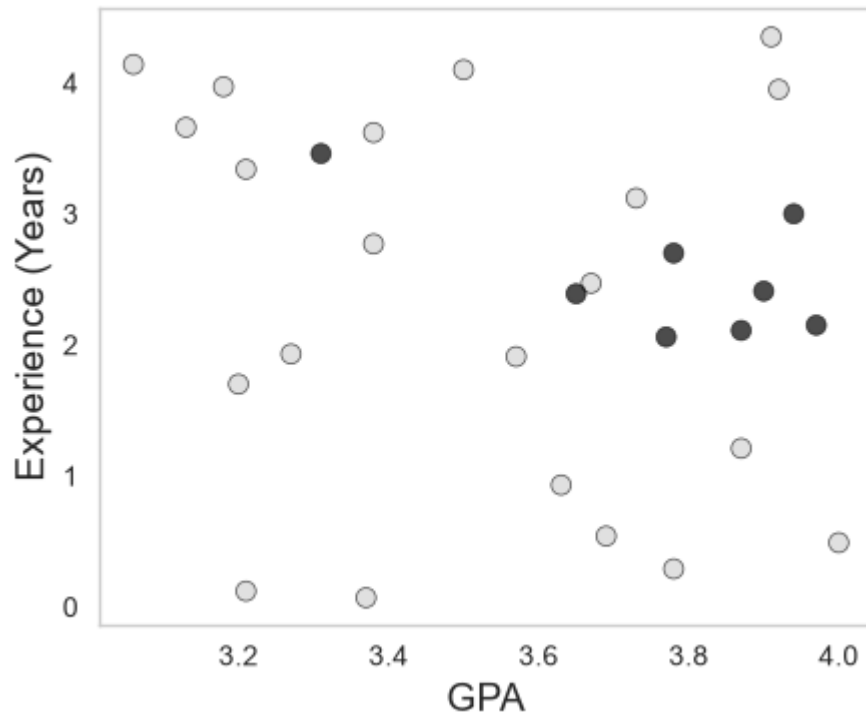
Toy Example: Job Interviews

k : # of independent variables ($k = 2$); n : # of observations ($n = 29$)



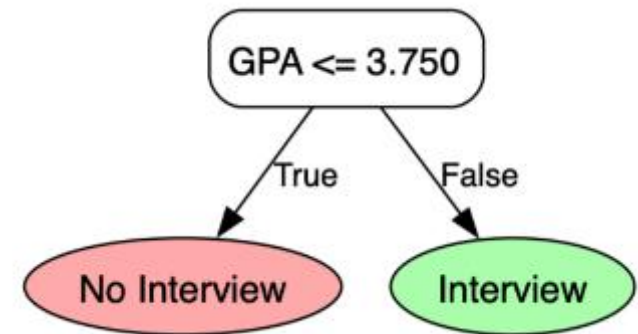
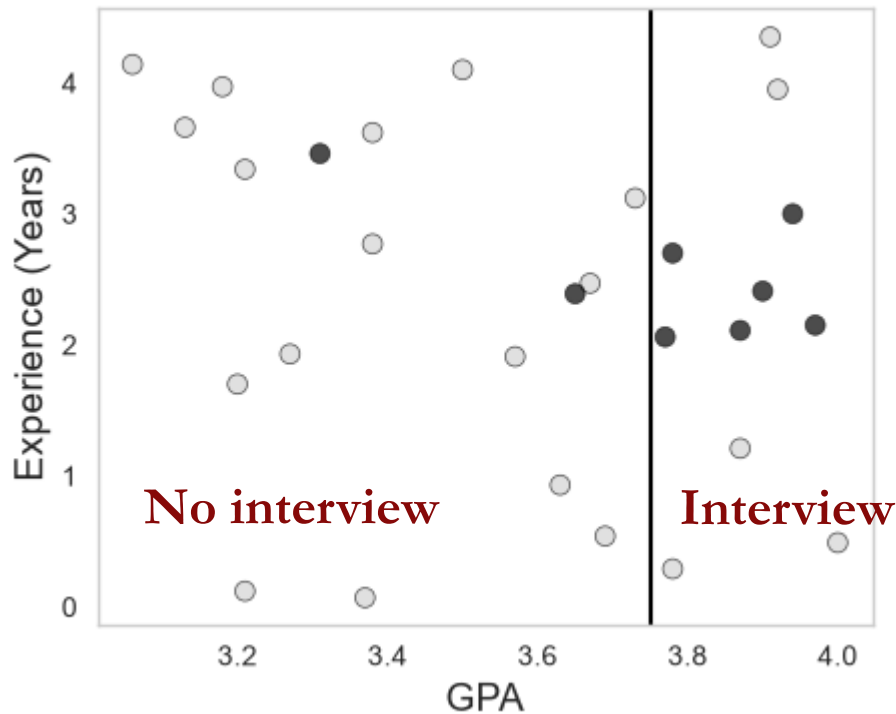
	Interview	GPA	Experience
0	0	3.27	1.93
1	0	3.37	0.07
2	0	3.57	1.91
3	0	3.91	4.35
4	0	3.2	1.7
5	1	3.9	2.41
6	1	3.94	3.0
...	...	...	...
26	1	3.97	2.15
27	1	3.78	2.7
28	0	3.73	3.12

Who Will Receive a Job Interview?

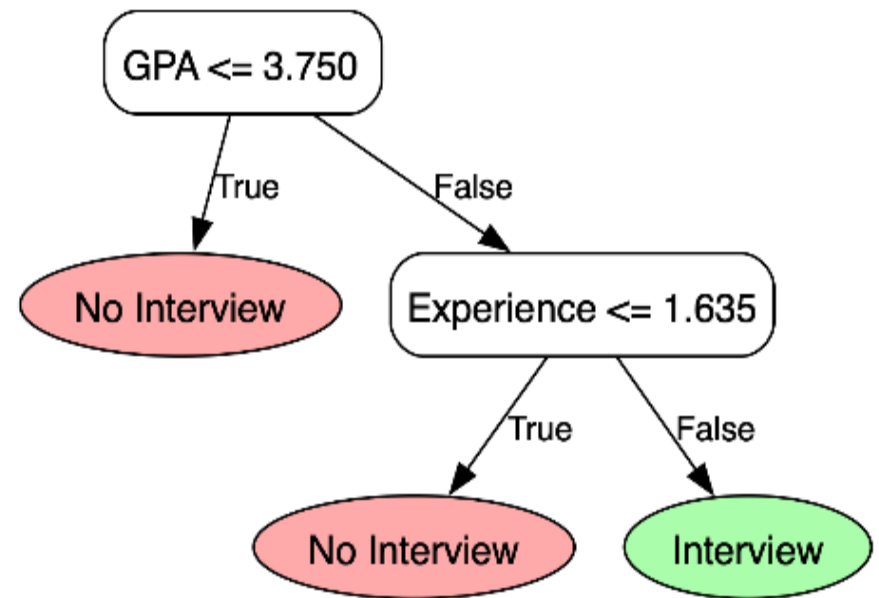
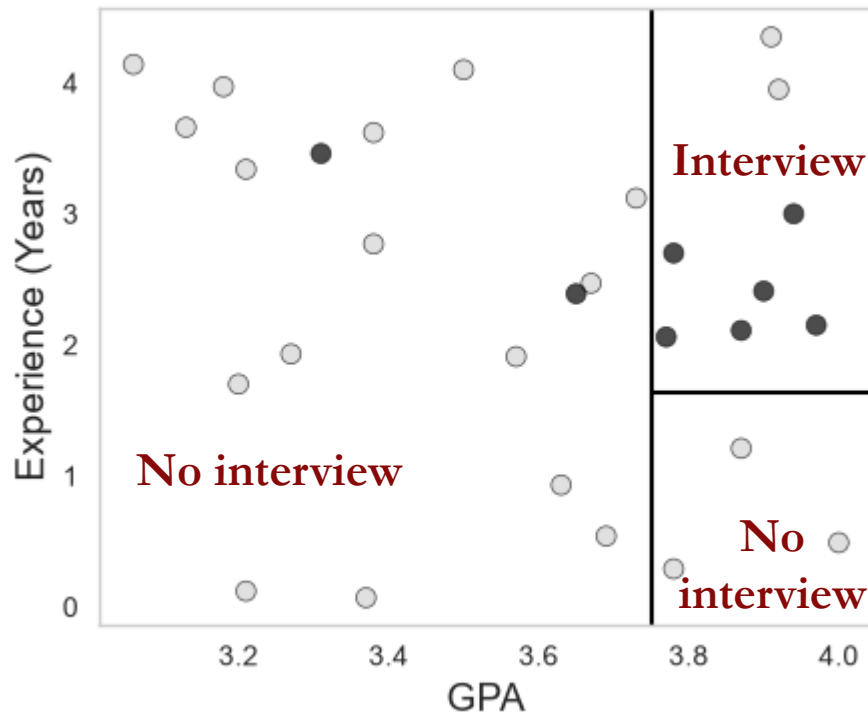


No Interview

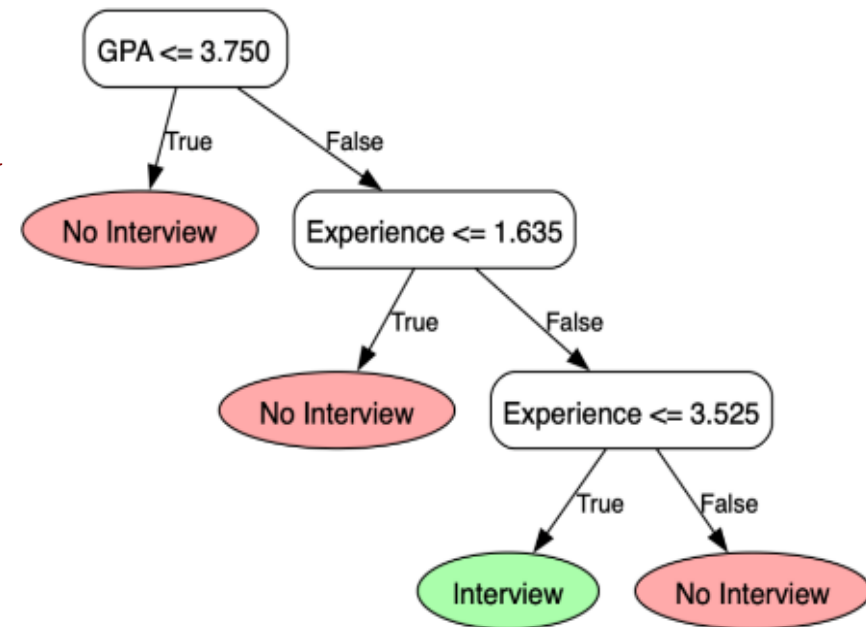
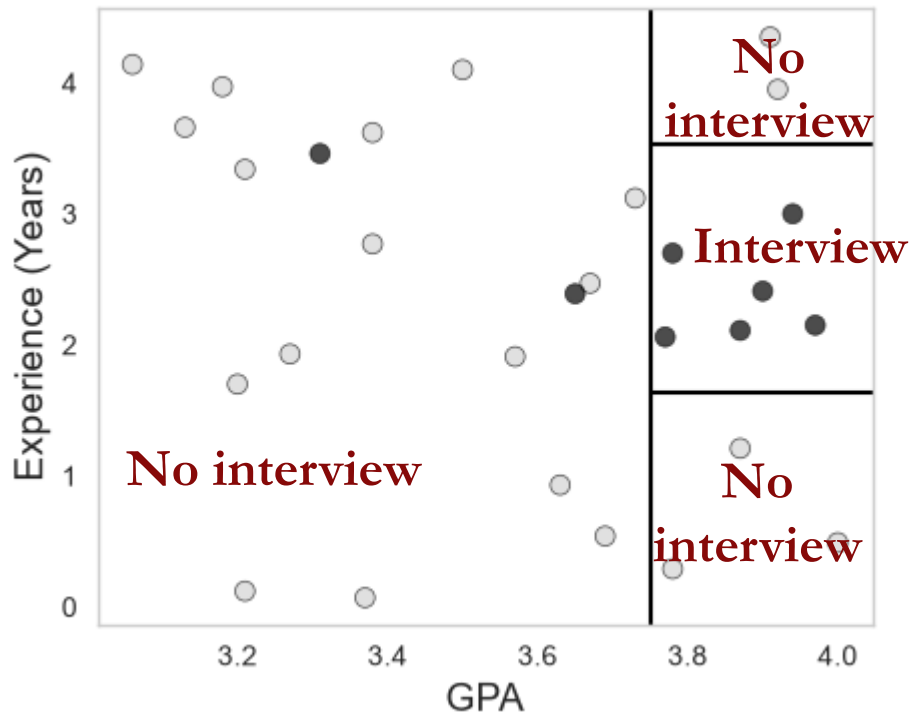
Who Will Receive a Job Interview?



Who Will Receive a Job Interview?

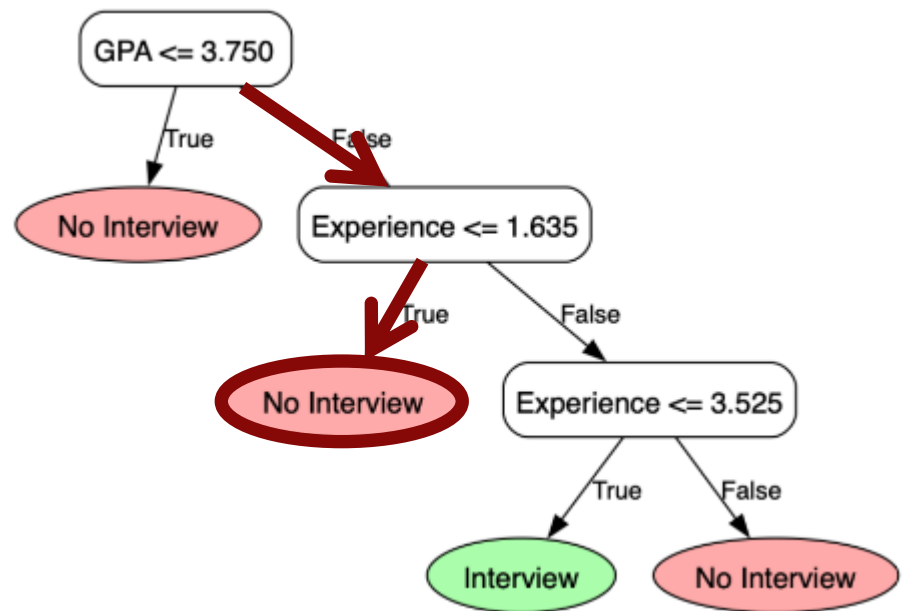


Who Will Receive a Job Interview?



Predicting with the CART Model

- Consider Stephanie, who has a 3.91 GPA and 0.8 years of experience
- Is she predicted by the model to receive a job interview?



Constructing CART for Classification

- How does CART decide:
 - The variable and cutoff for each split
 - What to predict at each leaf node
 - When to stop splitting
- Similar to CART for regression, but minor differences due to differences between regression and classification.

How to predict: Total Loss (Cost)

		Prediction/Decision	
Reality		No Violation (Grant Parole)	Violation (Deny Parole)
	No Violation	0	L_{FP}
	Violation	L_{FN}	0

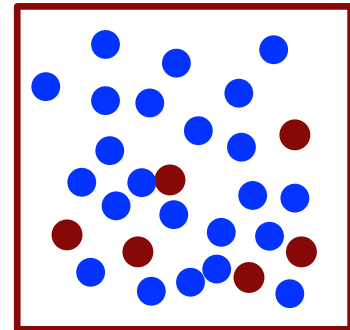
- In CART for Classification, we measure the quality of the model using:

$$\text{Total Loss} = (\text{number of FN}) * \overset{\text{Cost of a single false negative (FN)}}{L_{FN}} + (\text{number of FP}) * \overset{\text{Cost of a single false positive (FP)}}{L_{FP}}$$

$$\text{Average Loss} = \frac{\text{Total Loss}}{\text{Number of Data Points}}$$

Prediction at a Leaf

- For each leaf bucket, CART selects the class that has the lowest **Total Loss** across all the training observations in that leaf bucket.
- From the training set observations, suppose the bucket has:
 - 6 parole violators ●
 - 23 non-violators ●
- $L_{FN} = 20$ and $L_{FP} = 1$
- If predict this leaf as “**Violator**” Total Loss is
 $23 * 1 = 23$
- Total Loss from predicting “**Non-violator**” for this leaf
 $6 * 20 = 120$
- “**Violator**” has the lower loss



Gini index: leaf gini

- Assume a CART tree has L leaves numbered from 1 to L .
- For each $l \in \{1, \dots, L\}$, let p_l and q_l respectively denote the *proportions* of loss-weighted samples sizes of violators and non-violators at leaf l . Note that $p_l + q_l = 1$.
 - E.g. Leaf l has 6 violators and 23 non-violators. Then:

$$p_l = \frac{20 \times 6}{20 \times 6 + 1 \times 23} = 0.84, \quad q_l = \frac{1 \times 23}{20 \times 6 + 1 \times 23} = 0.16.$$

- The expression of the *Gini index* of leaf l is:

$$\text{Gini (Leaf } l) = 1 - p_l^2 - q_l^2.$$

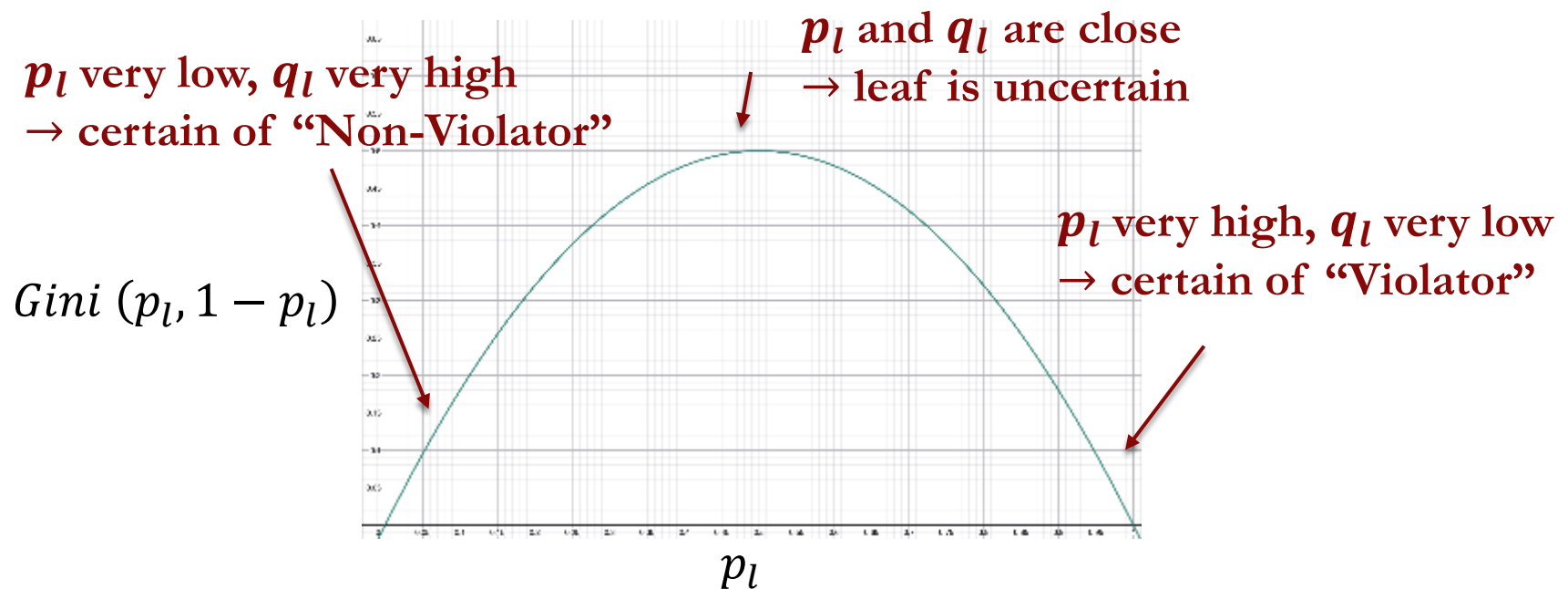
- It quantifies how *uncertain* the leaf is of its decision.

Gini index: Leaf Gini

- The expression of the *Gini index* of leaf l is:

$$\text{Gini}(p_l, q_l) = 1 - p_l^2 - q_l^2.$$

- It quantifies how *uncertain* the leaf is of its label.

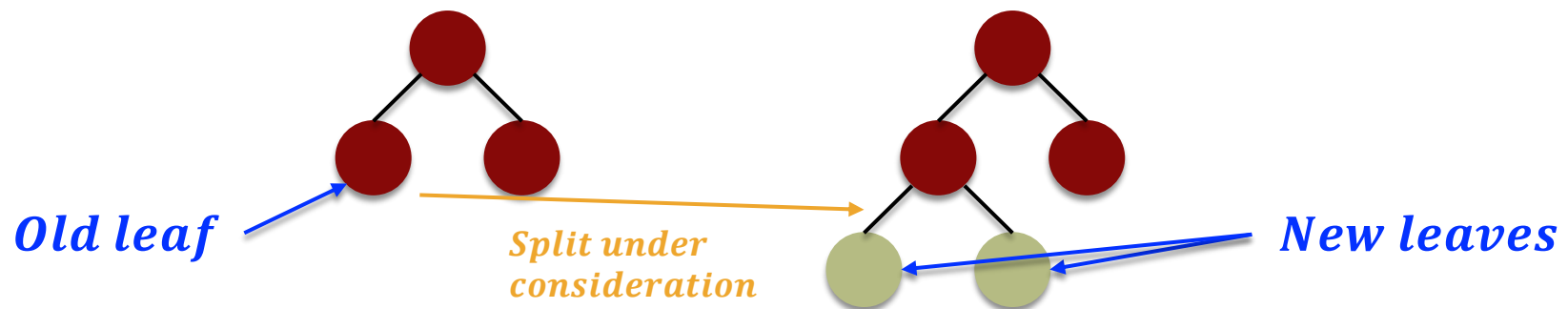


Split criterion: Gini reduction

- CART decides which split to make by computing the *Gini reduction* of each potential split:

$$\begin{aligned} &\text{Gini reduction} \\ &= \\ &\text{weighted sum of Ginis of leaves} - \text{Gini of original leaf} \end{aligned}$$

- Iteratively, CART chooses the split with the *highest Gini Reduction*.



Why use Gini instead of Total Loss for splitting?

- Consider the following example for classification in 1 dimension:

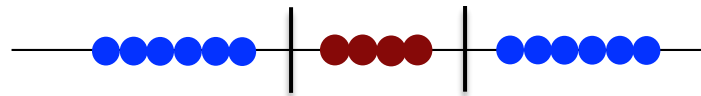


with $L_{FP} = L_{FN} = 1$.

- Assume we use **Total Loss** as a metric to decide where to split. Note that in this case, no split happens since no split has lower loss than predicting ● everywhere.

→ No split

- However, if we use **Gini** as a metric to decide where to split, then the following sequence of two splits is done since each reduces Gini:



CART Model Parameters

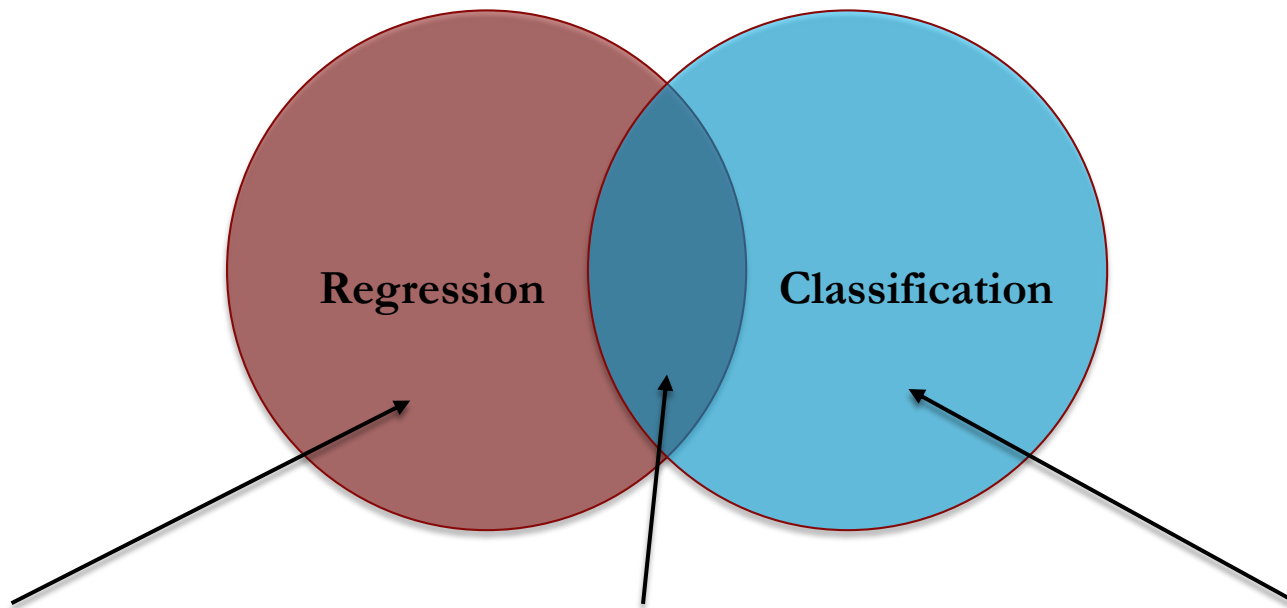
- Parameters `ccp_alpha`, `min_samples_leaf`, `min_samples_split`, `max_depth`.
- The complexity parameter (**ccp_alpha**) removes splits that do not sufficiently lower the losses. `sklearn`'s algorithm automatically performs a two-step process:
 - Step 1: Grow the Full Tree.** The model first grows a complete, unpruned, and overfit tree (within other constraints like `max_depth`).
 - Step 2: Prune the weakest splits.** The algorithm then looks at every branch and calculates its “effective alpha”:

$$\alpha_{eff} = \frac{\textit{Total Gini reduction from this branch}}{\textit{Number of new leaves in this branch} - 1}.$$

It iteratively prunes the branch with the lowest α_{eff} until all remaining branches have an α_{eff} greater than **ccp_alpha**.

Trade-off: loss (Gini) reduction vs. model complexity

CART for Regression vs. Classification

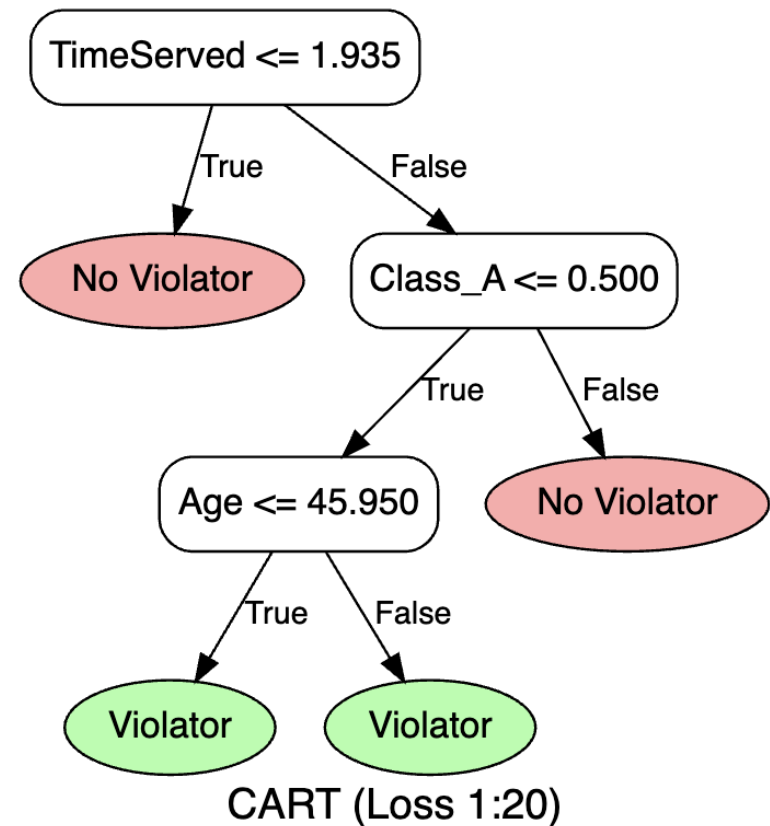


- Each leaf predicts a numerical value (e.g., \$17.20).
- Minimizes **sum of squared error** at each split.
- Constructs splits greedily, minimizing impurity or cost of errors.
- Complexity can be controlled explicitly with `ccp_alpha`, `min_samples_leaf` ...
- Each leaf predicts a categorical value (e.g., interview or not).
- Minimizes **Gini** at each split (weighted by loss matrix).

CART for Parole Data

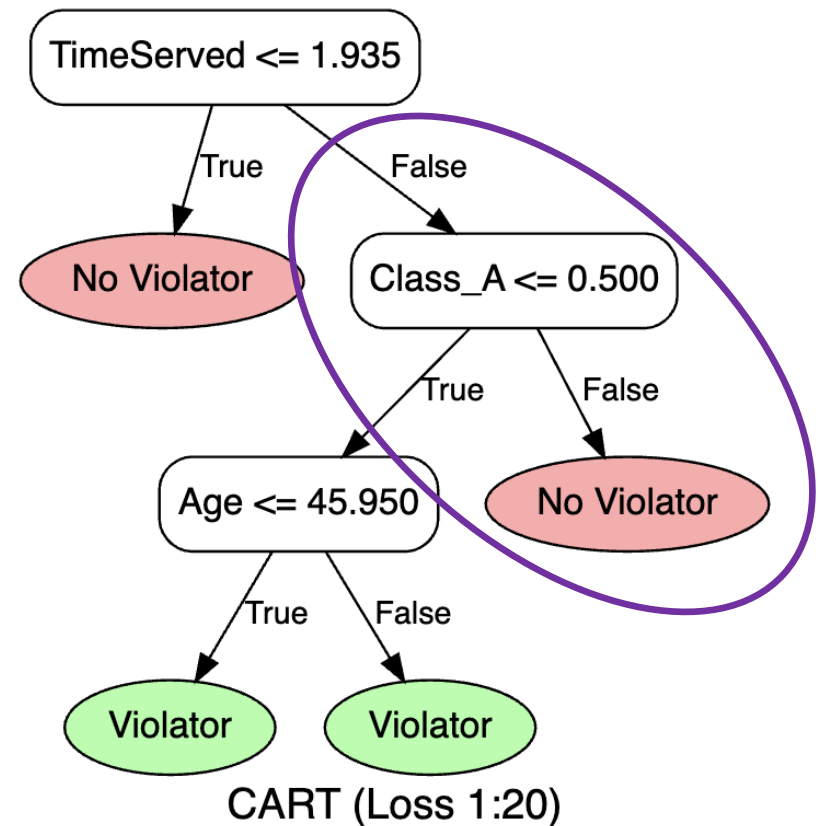
CART Model for Parole Data

- Here is the CART model we built for this classification problem
- $L_{FP} = 1$ and $L_{FN} = 20$
- `min_samples_leaf` = 25
- `ccp_alpha` = 0.01



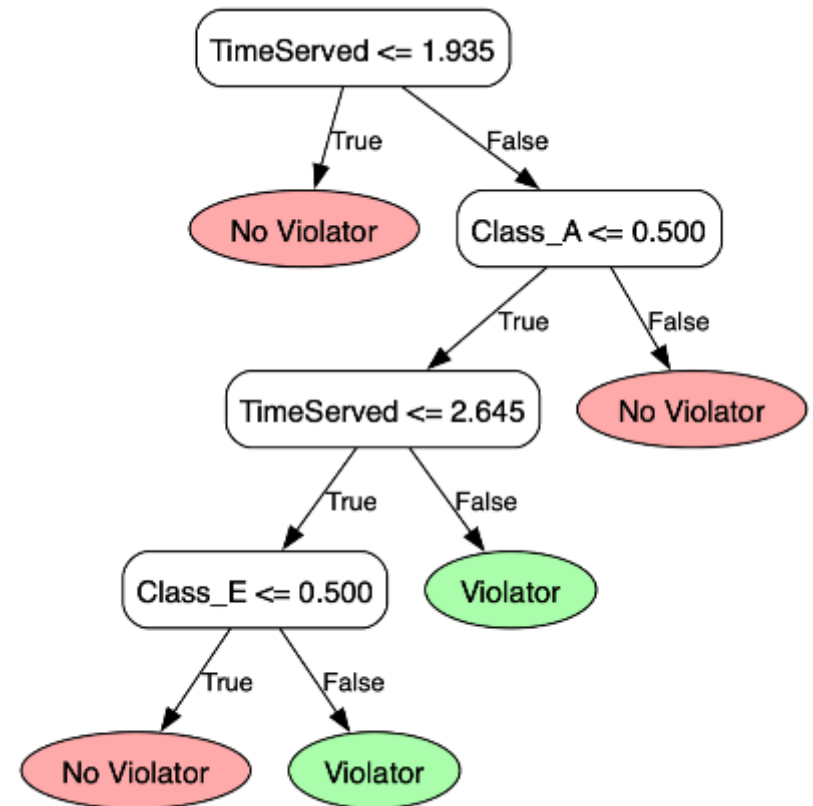
CART Model for Parole Data

- Here is the CART model we built for this classification problem
- What does it mean? Why?



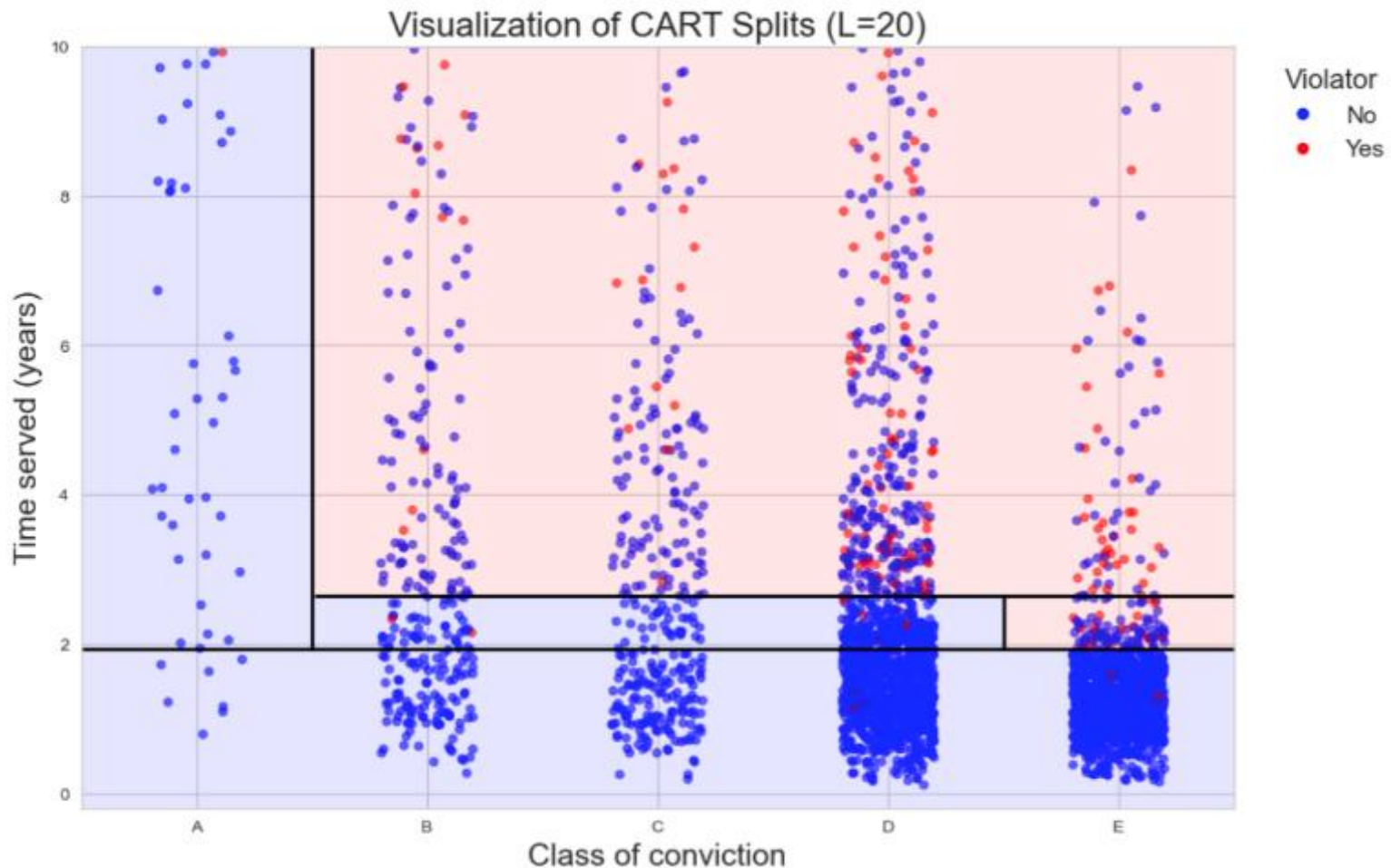
Restricting to “Time served” and “Class”

- To visualize the tree on a 2D graph, we restrict the model to the two features “Time served (Years)” and “Class of conviction” (A,B,C,D,E).
- $L_{FP} = 1$ and $L_{FN} = 20$
- **min_samples_leaf** = 25
- **ccp_alpha** = 0.01



CART using 'Time Served' and 'Class'

Visualizing the Tree using TimeServed and Class



Python Code to Generate Classification CART

To create a CART model:

```
model = DecisionTreeClassifier(  
    min_samples_leaf = 25,  
    ccp_alpha = 0.01,  
    class_weight = {0: 1, 1: 20}  
)
```

To fit a CART model:

```
model.fit(X_train, y_train)
```

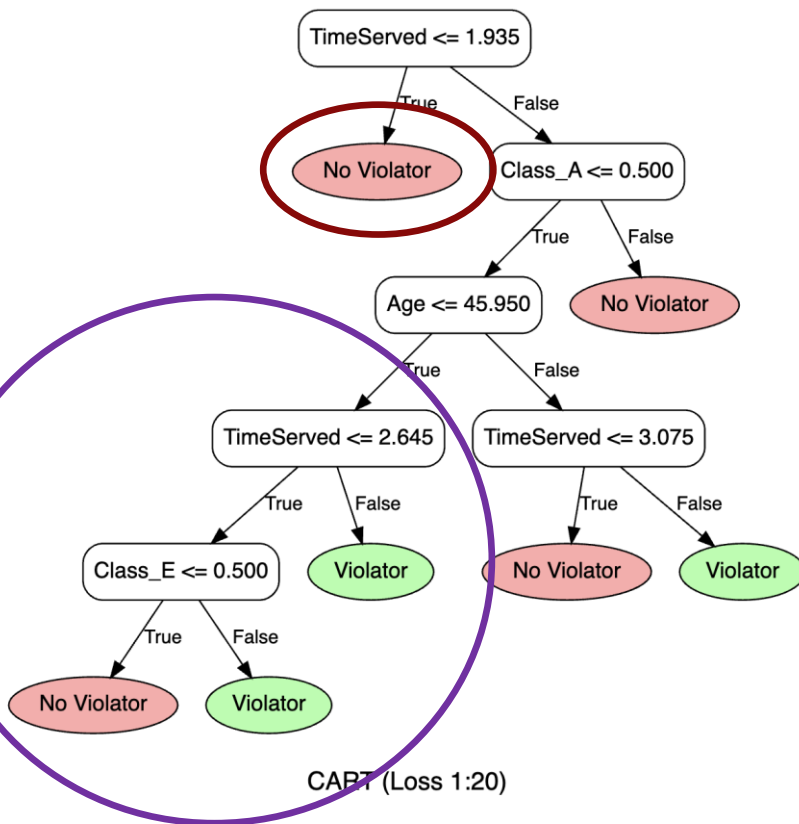
To plot a CART tree:

```
plot_tree(model)
```

Aside: CART trees for different L_{FN} values, cont.

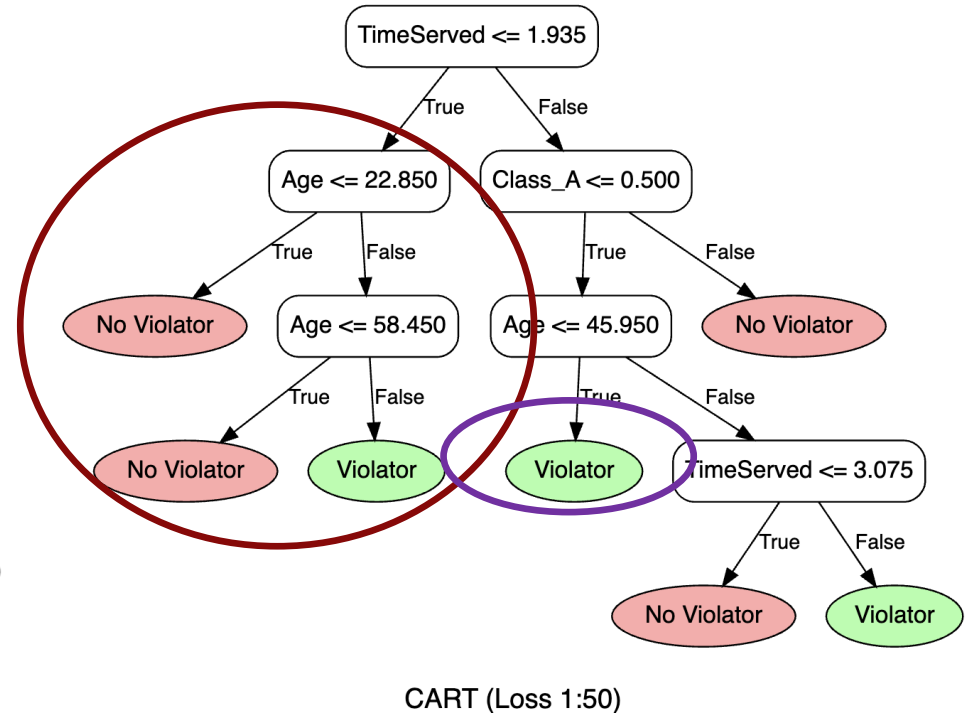
$L_{FN} = 20$

$L_{FP} = 1$



$L_{FN} = 50$

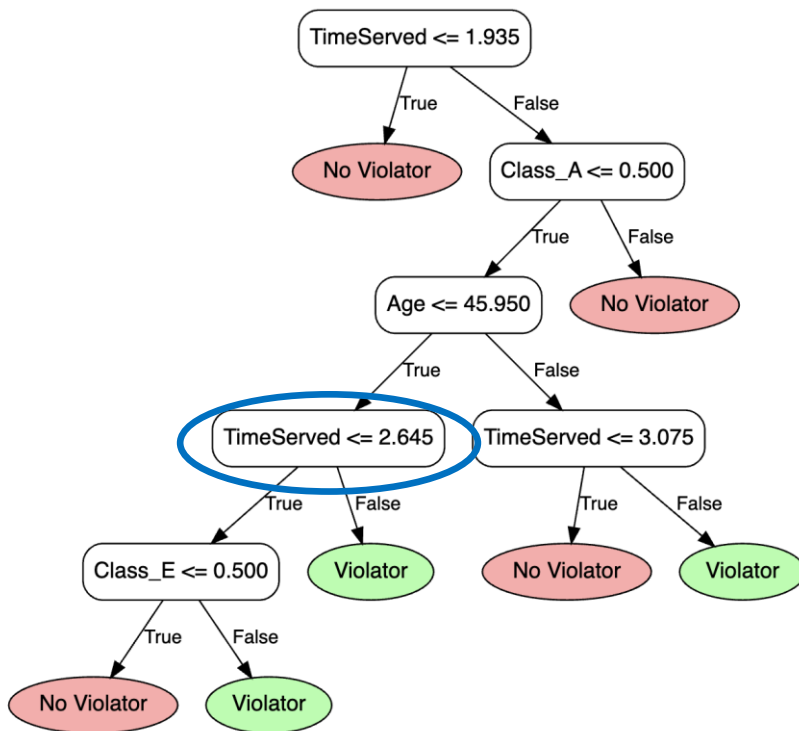
$L_{FP} = 1$



Aside: CART trees for different L_{FN} values, cont.

$L_{FN} = 20$

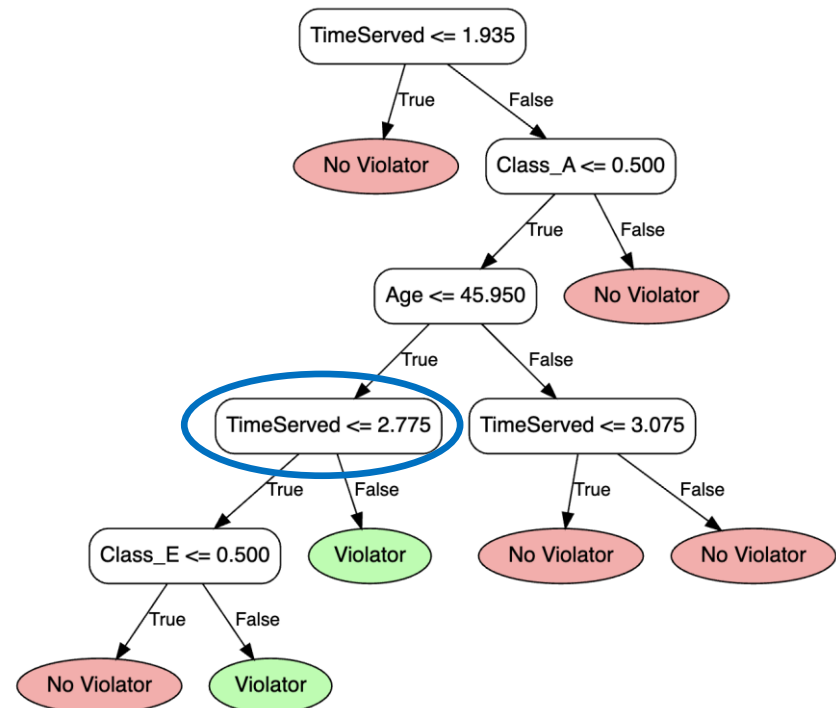
$L_{FP} = 1$



CART (Loss 1:20)

$L_{FN} = 10$

$L_{FP} = 1$



CART (Loss 1:10)

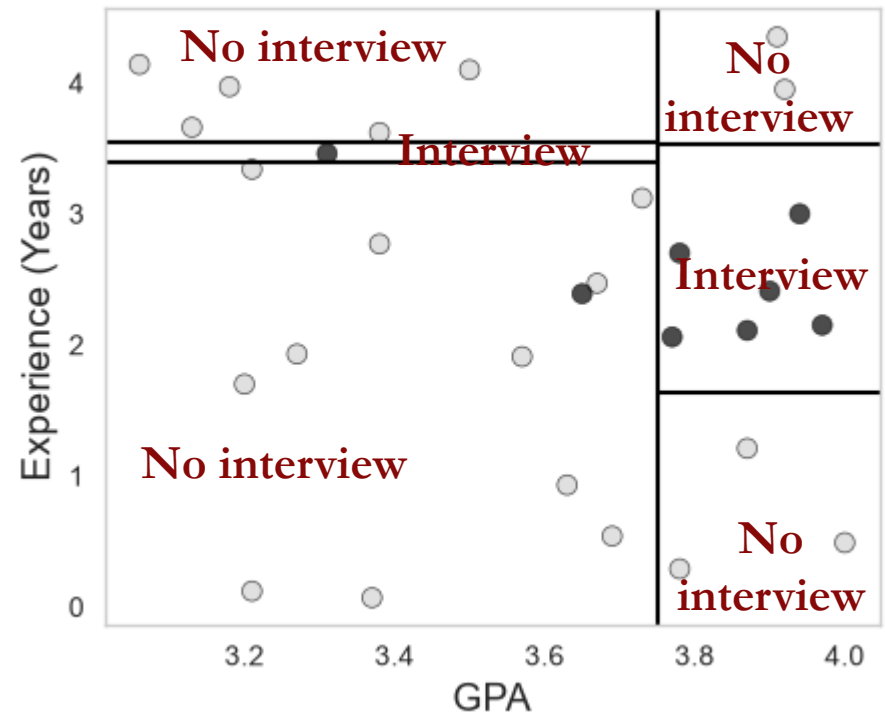
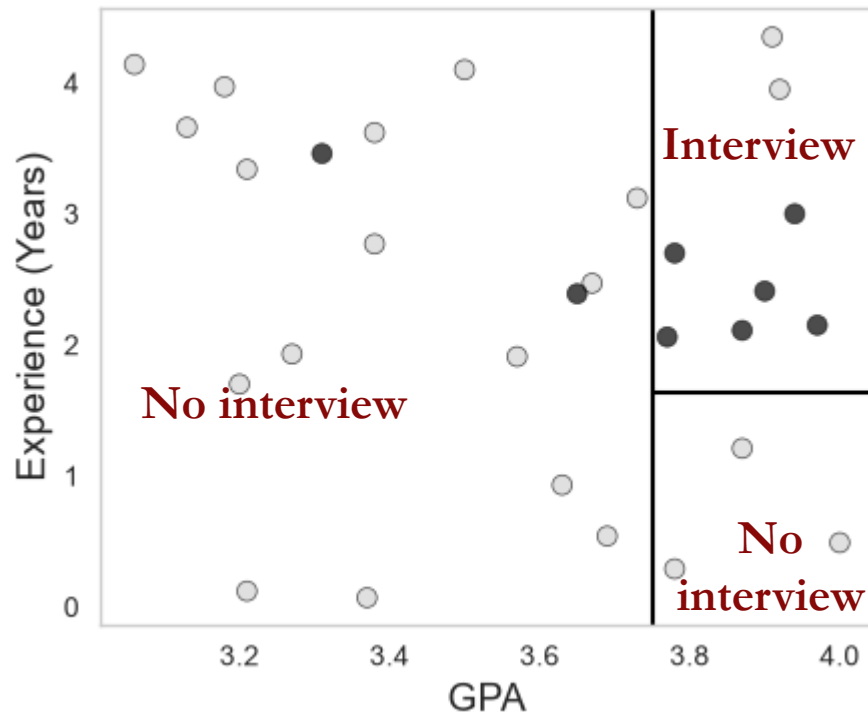
3. Cross Validation

What is overfitting and why are complex trees more likely to overfit?

How does cross validation allow us to select “optimal” parameters (like `cp`) without looking at the test set?

Overfitting

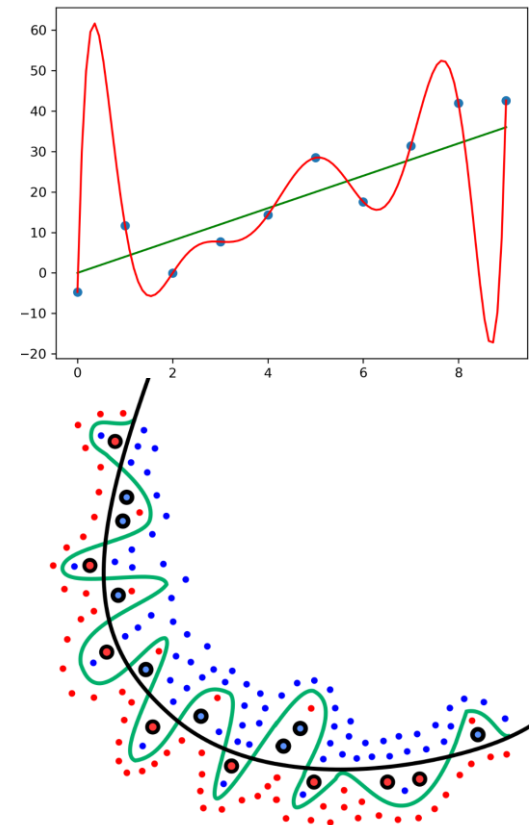
Two different CART models



- Which model is more accurate on the training data?
- Which model will be more accurate on the test data?

Complex Models and Overfitting

- Definition of Overfitting:
 - Finding patterns in the training dataset that do not generalize to the test data.
 - Idiosyncratic Noise
- Higher model complexity is both a blessing and a curse.
 - **Blessing:** Can find complex (real) patterns (e.g., non-linearities)
 - **Curse:** Prone to overfitting to noise
 - Ignorance is bliss? Simple models cannot “overthink” it.



<https://en.wikipedia.org/wiki/Overfitting>

Overfitting the (Training) Data

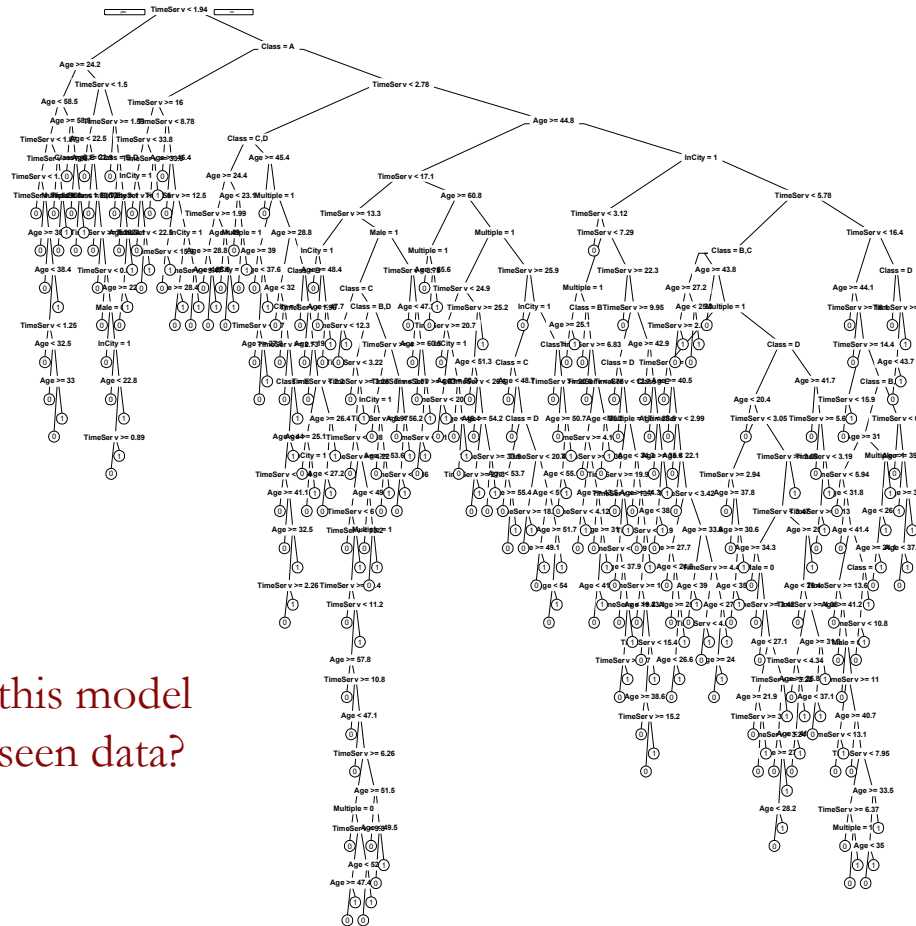
- **Model overfitting** occurs when a model performs very well on the training set in a way that will not translate to good performance on the test set—or on new data in general
- Overfitted models are often more complicated and do not necessarily accord with intuition/judgment
 - Why should GPA falling in the narrow range 3.28-3.30 yield a different result than values slightly above or below?
- Overfitted models often capture idiosyncrasies (or even errors) in the training data, instead of capturing actual relationships that will hold for new data

Addressing Overfitting in CART

- Overfitting stems from differences between how models are constructed (to maximize fitting) and how they are intended to be used (to make prediction on new data)
 - Model fitting maximizes accuracy on a **known** set of training data
 - Models are used to make predictions on **new** (previously unseen) data
- **Challenge:** How to address overfitting in CART?

Solution strategy to address overfitting: Deliberately build a model with fewer splits. Such a simpler model is more likely to not overfit the data.

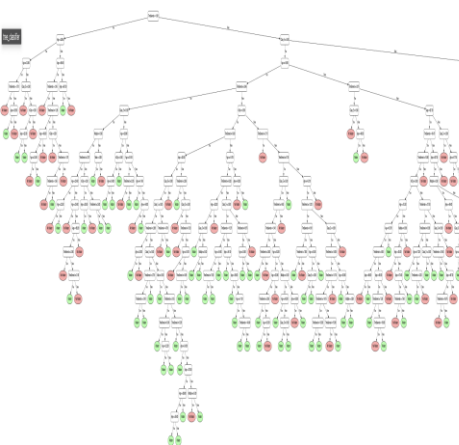
CART that perfectly fits the Training data



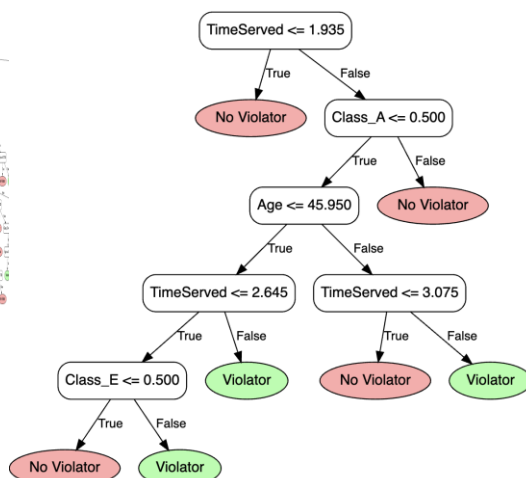
How do you think this model will perform on unseen data?

How do we find the best ccp_alpha?

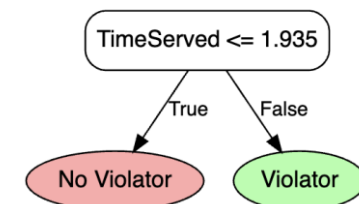
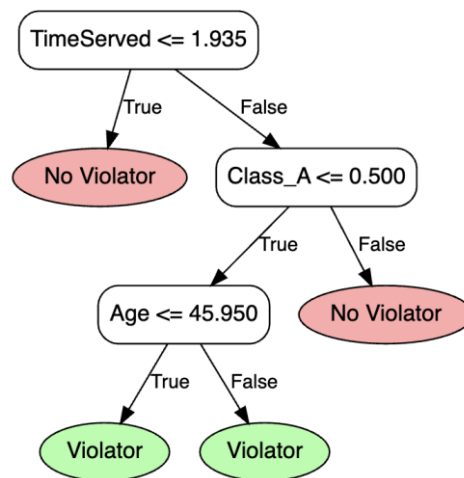
ccp_alpha = 0 ccp_alpha = 0.005 ccp_alpha = 0.01 ccp_alpha = 0.1



overfitting
the training
data?



sweet spot?



underfitting
the training
data?

Model Parameters

- A model parameter controls how the model how to fit to the data by controlling the coefficients, splits...etc.
 - A **setting** chosen by us (the human), not the model itself.
 - Parameters often control some kind of trade-off, like model complexity.
- CART
 - `ccp_alpha`, `min_samples_leaf`, `min_samples_split`, `max_depth` are parameters
 - Dictate how CART decides when to split and when to stop splitting

k -fold cross validation for choosing
model parameters in CART (or any
model)

k -fold Cross-Validation

Divide

- Divide the training set into k pieces (“folds”)

Fold 1 Fold 2 Fold 3 Fold 4 Fold 5

Train / Test

- For each candidate parameter value...
 - Use $k - 1$ folds to construct a model
 - Test that model on the remaining fold
 - Repeat for each of the k folds

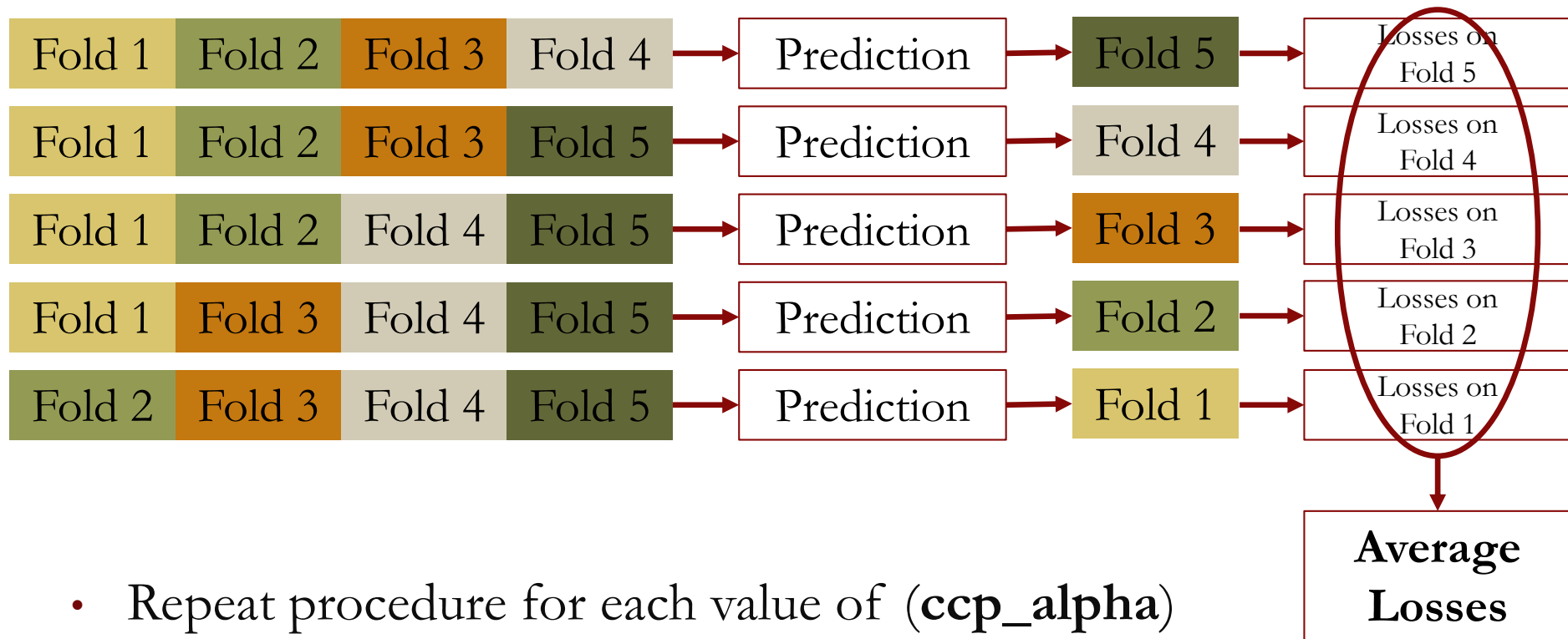
Average

- For each candidate parameter value...
 - compute the average loss over the k folds

Select

- Finally, select the parameter value that minimizes the **average loss**
 - Uses the entire training set but nothing from the test set

5-Fold Cross-Validation

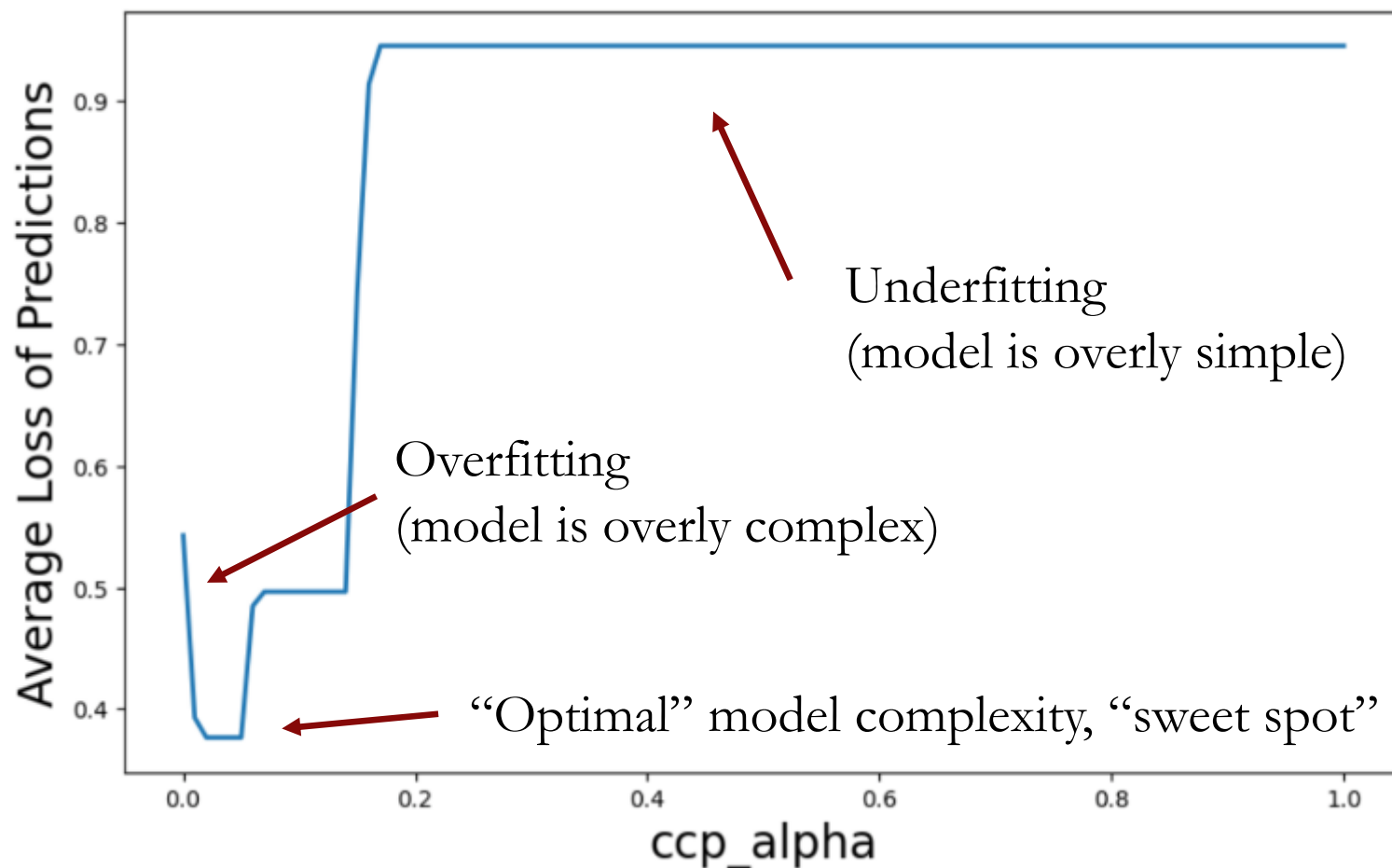


- Repeat procedure for each value of (**ccp_alpha**)
- Select the value of **ccp_alpha** that maximizes the **average loss**

Cross-Validation to choose **ccp_alpha**

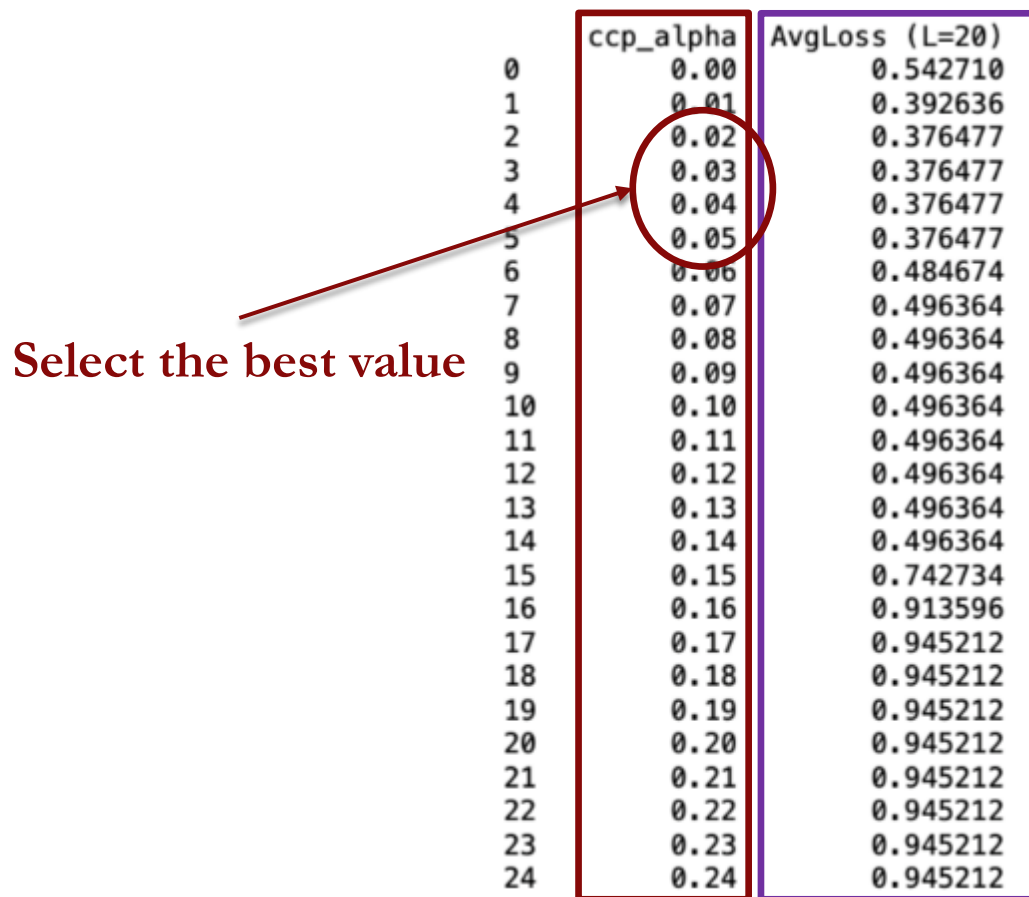
- It is common to perform 5-fold or 10-fold cross-validation (we will perform 10-fold cross-validation here)
- One parameter is varied (**ccp_alpha**). **min_samples_leaf** is fixed throughout.
- We use our losses of $L_{FN} = 20$ and $L_{FP} = 1$
- For **ccp_alpha** = 0, 0.01, 0.02, 0.03, ..., 1 :
 - We will do cross-validation with the value of **ccp_alpha**.
 - We will record the average loss value over all folds. Call it **AvgLoss**.
- Finally, we will choose that value of **ccp_alpha** that achieves the lowest value of **AvgLoss**.
- Then fit a (final) tree on the full training data with this value of **ccp_alpha**.

Cross-Validation to select the best choice of `ccp_alpha`



Cross-Validation Results from R

Tested values Average performance



Select the best value

	ccp_alpha	AvgLoss (L=20)
0	0.00	0.542710
1	0.01	0.392636
2	0.02	0.376477
3	0.03	0.376477
4	0.04	0.376477
5	0.05	0.376477
6	0.06	0.484674
7	0.07	0.496364
8	0.08	0.496364
9	0.09	0.496364
10	0.10	0.496364
11	0.11	0.496364
12	0.12	0.496364
13	0.13	0.496364
14	0.14	0.496364
15	0.15	0.742734
16	0.16	0.913596
17	0.17	0.945212
18	0.18	0.945212
19	0.19	0.945212
20	0.20	0.945212
21	0.21	0.945212
22	0.22	0.945212
23	0.23	0.945212
24	0.24	0.945212

Python Code for Cross Validation

1. Define the estimator with the loss matrix:

```
dt_cv = DecisionTreeClassifier(  
    min_samples_leaf=5,  
    class_weight={0: 1, 1: L}  
)
```

2. Define the scoring function:

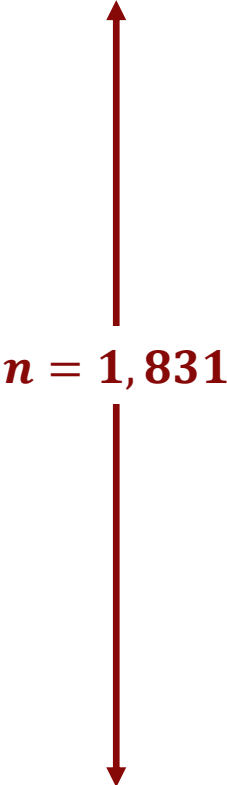
```
loss_scorer = make_scorer(lambda y_true, y_pred: avg_loss(y_true,  
    y_pred, L), greater_is_better=False)
```

3. Do cross validation on ccp_alpha:

```
cv.trees = GridSearchCV(  
    dt_cv,  
    param_grid = {'ccp_alpha': np.linspace(0, 1, 101)},  
    cv=StratifiedKFold(10),  
    scoring=loss_scorer,  
    n_jobs=-1  
)
```

Using the model to make predictions on the Test set

k : # of independent variables ($k = 6$); n : # of observations ($n = 1,831$)



	Violator	Male	Age	TimeServed	Class	Multiple	InCity	Prediction
2	1	1	26.0	5.95	D	1	0	1
3	0	1	24.9	2.25	D	1	0	0
4	0	1	52.1	29.22	A	0	0	0
9	0	1	31.6	1.17	E	0	0	0
11	0	1	53.9	21.61	A	0	1	0
20	0	1	36.4	1.63	D	0	1	0
27	0	1	60.7	1.83	C	0	1	0
35	0	1	30.3	1.83	B	1	0	0
46	0	1	88.1	26.66	A	1	0	0
52	0	1	47.6	20.57	A	1	0	0
58	0	1	41.8	16.93	B	1	1	1
59	0	1	57.0	1.01	E	0	0	0
61	0	1	33.1	2.08	D	0	0	0
63	0	1	43.7	0.86	E	0	0	0
64	0	1	23.0	1.07	D	0	0	0
68	0	1	39.2	0.92	E	0	0	0
...	...	...	...	...	...	...	...	...
6080	0	0	43.7	1.76	E	0	0	0
6082	0	1	45.9	0.83	E	0	0	0
6083	0	1	48.2	2.21	D	0	0	0
6092	0	1	57.1	0.75	E	0	0	0
6097	0	1	54.7	20.87	A	0	1	0

Performance Metrics for our CART model with $\text{ccp_alpha} = 0.02$, $L_{FN} = 20$, and $L_{FP} = 1$

$$\text{Accuracy} = \frac{\text{Number correct}}{\text{Total number}} = \frac{94+1184}{546+94+7+1184} = 0.698$$

$$\text{True Positive Rate (TPR)} = \frac{\text{True Positives}}{\text{All Positives}} = \frac{94}{94+7} = 0.931$$

$$\text{False Positive Rate (TPR)} = \frac{\text{False Positives}}{\text{All Negatives}} = \frac{546}{546+1184} = 0.316$$

$$\text{Loss} = L_{FP} \times \# \text{ False Positives} + L_{FN} \times \# \text{ False Negatives} = 1 \times 546 + 20 \times 7 = 686$$

		Model	
Reality		No violation	Violation
	No Violation ($Y_i = 0$)	1,184	546
	Violation ($Y_i = 1$)	7	94

Performance Metrics for the “current practice” model

- “Current practice”: Predict that all parolees will not violate parole

$$\text{Accuracy} = \frac{\text{Number correct}}{\text{Total number}} = \frac{1,730 + 0}{1,730 + 101 + 0 + 0} = 0.945$$

$$\text{True Positive Rate (TPR)} = \frac{\text{True Positives}}{\text{All Positives}} = \frac{0}{101 + 0} = 0.000$$

$$\text{False Positive Rate (FPR)} = \frac{\text{False Positives}}{\text{All Negatives}} = \frac{0}{1,730 + 0} = 0.000$$

**2.8x higher
than CART**

$$\text{Loss} = L_{FP} \times \# \text{ False Positives} + L_{FN} \times \# \text{ False Negatives} = 1 \times 0 + 20 \times 101 = 2,020$$

		Model	
Reality		No violation	Violation
	No Violation ($Y_i = 0$)	1,730	0
	Violation ($Y_i = 1$)	101	0

Performance Metrics for the “current practice” model

- “Current practice”: Predict that all parolees will not violate parole

$$\text{Accuracy} = \frac{\text{Number correct}}{\text{Total number}} = \frac{1730+0}{1730+101+0+0} = 0.945$$

$$\text{True Positive Rate (TPR)} = \frac{\text{True Positives}}{\text{All Positives}} = \frac{0}{101+0} = 0.000$$

$$\text{False Positive Rate (TPR)} = \frac{\text{False Positives}}{\text{All Negatives}} = \frac{0}{1730+0} = 0.000$$

**2.9x higher
than CART**

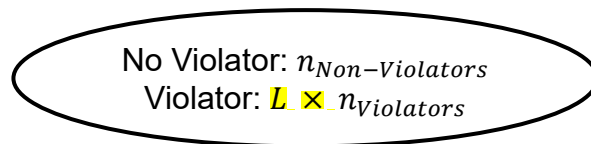
$$\text{Loss} = L_{FP} \times \# \text{ False Positives} + L_{FN} \times \# \text{ False Negatives} = 1 \times 0 + 20 \times 101 = 2,020$$

		Model	
		No violation	Violation
Reality	No Violation ($Y_i = 0$)	1,730	0
	Violation ($Y_i = 1$)	101	0

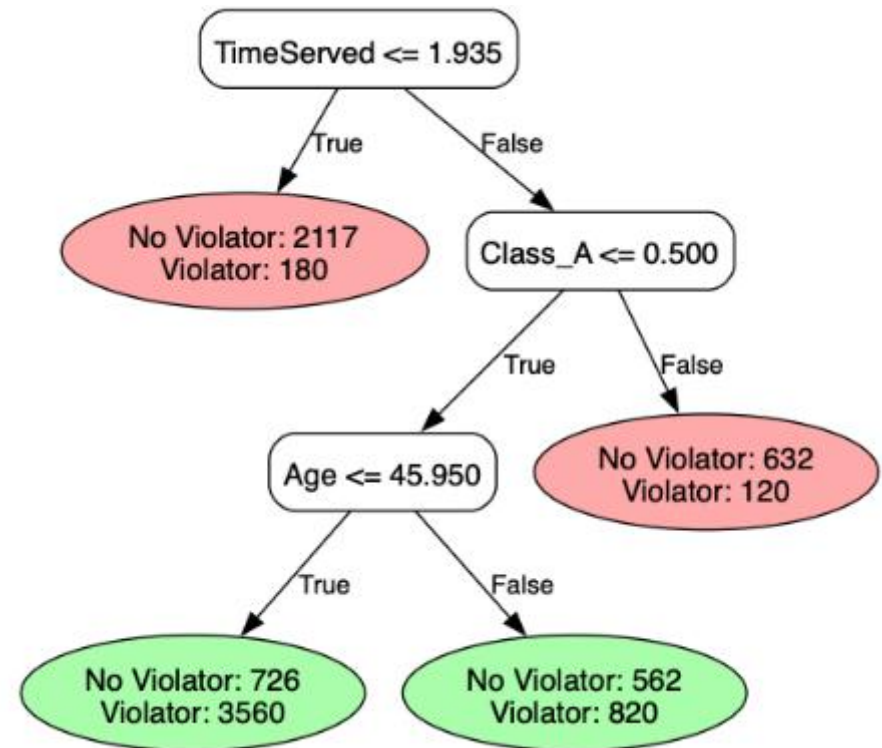
CART can assign probabilities to classes (instead of assigning classes)

CART is equivalent to assigning probabilities using the training set

- This tree was constructed to “fit” the $n = 4,271$ inmates in the training set
- Look at the composition of the 4 leaves: these are **loss-weighted** sample sizes



- We convert these leaf buckets to probabilities
- Now the leaf values are probabilities

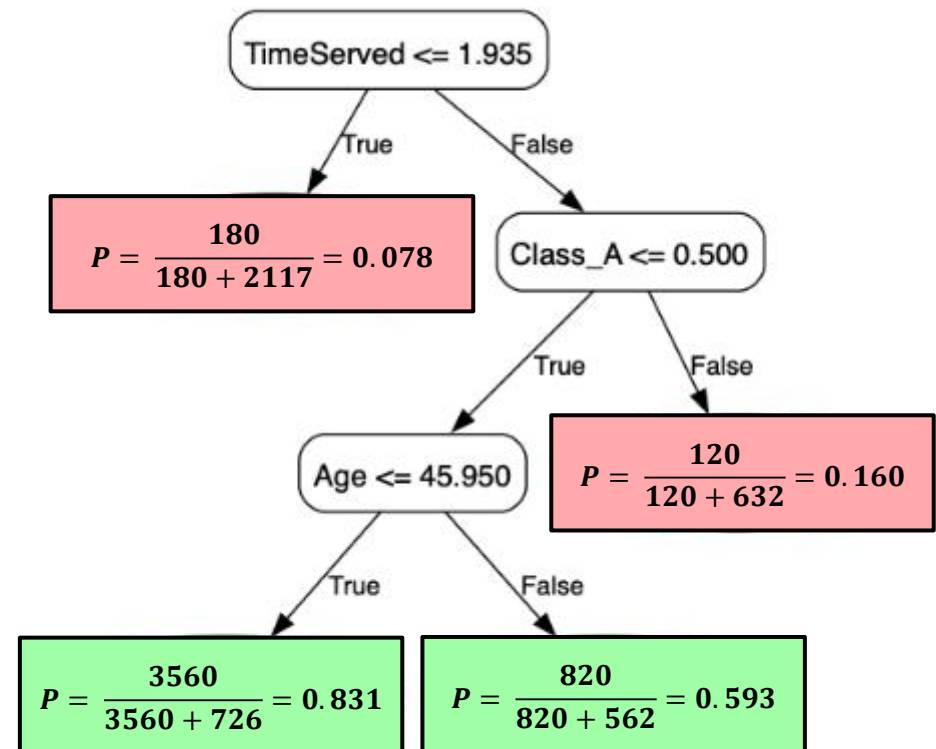


CART is equivalent to assigning probabilities using the training set

- This tree was constructed to “fit” the $n = 4,271$ inmates in the training set
- Look at the composition of the 4 leaves: these are loss-weighted sample sizes
- We can convert these leaf buckets to probabilities:

$$P = \frac{L \times n_{\text{Violators}}}{L \times n_{\text{Violators}} + n_{\text{Non-Violators}}}$$

- Now the leaf values are probabilities



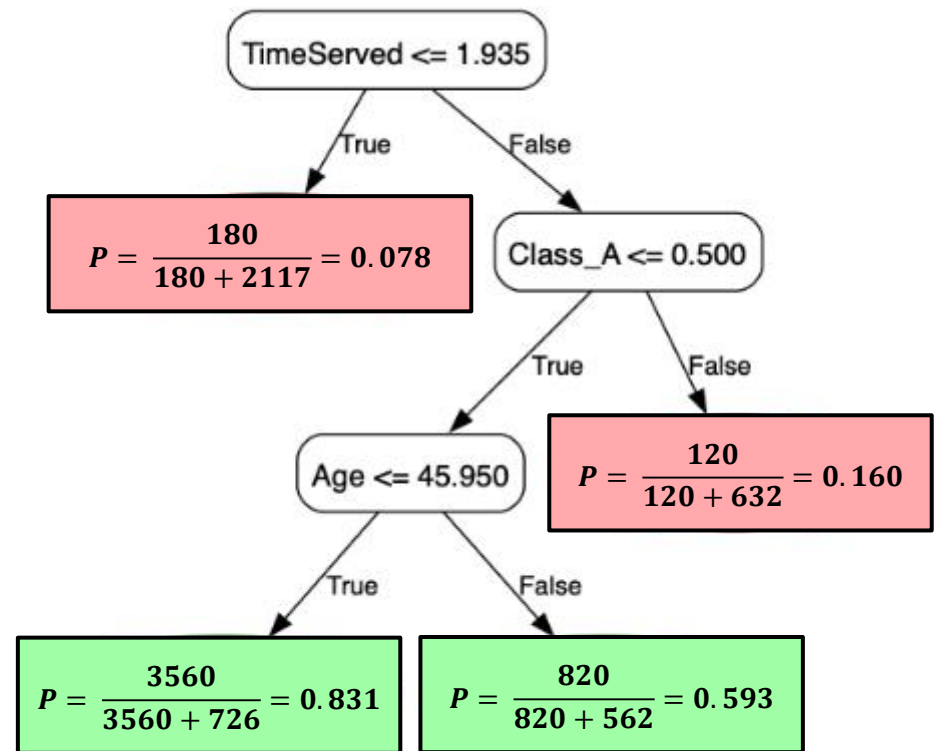
CART is equivalent to assigning probabilities using the training set

- `sklearn` assigns labels to leaves based on the majority loss-weighted sample sizes.
- A leaf is labeled “Violator” when:

$$L \times n_{\text{Violators}} > n_{\text{Non-Violators}}.$$

- This is equivalent to

$$P = \frac{L \times n_{\text{Violators}}}{L \times n_{\text{Violators}} + n_{\text{Non-Violators}}} > 0.5.$$



Let us Assess our “new” CART Probability Model on the Test Set

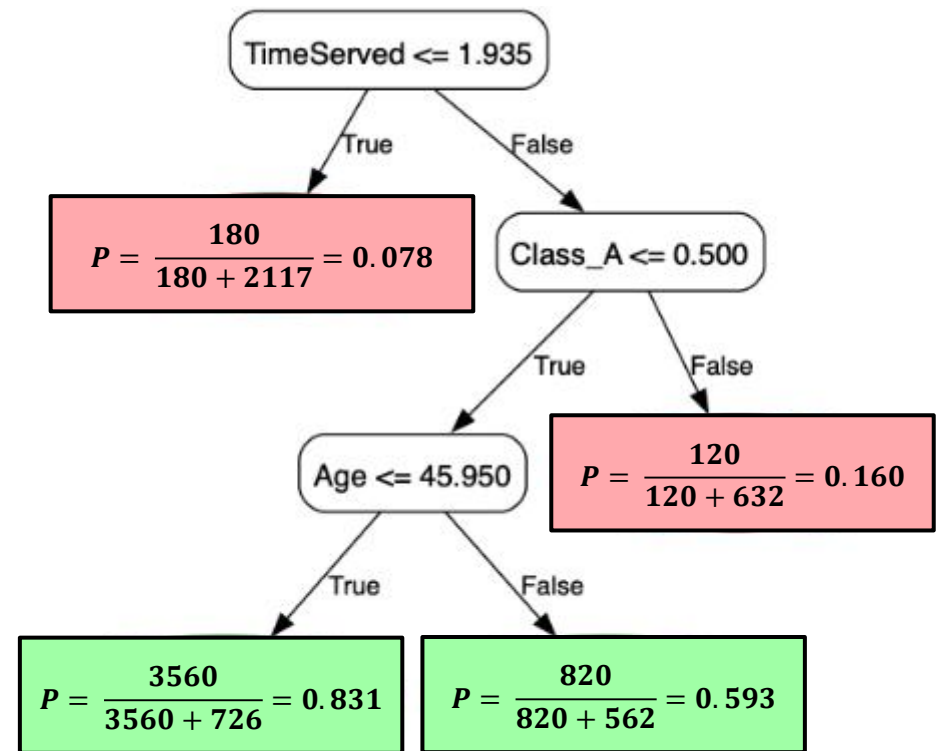
- We can use our “new” CART tree to assign a probability of parole violation to each of the $n = 1,831$ inmates in the Test set
- E.g.: test input falls into leaf:

No Violator: $n_{Non-Violators}$
Violator: $L \times n_{Violators}$

Then it's labeled “Violator”

$$\Leftrightarrow L \times n_{Violators} > n_{Non-Violators}$$

$$\Leftrightarrow P > p_{thres} = 0.5.$$



Setting p_{thres} at test time

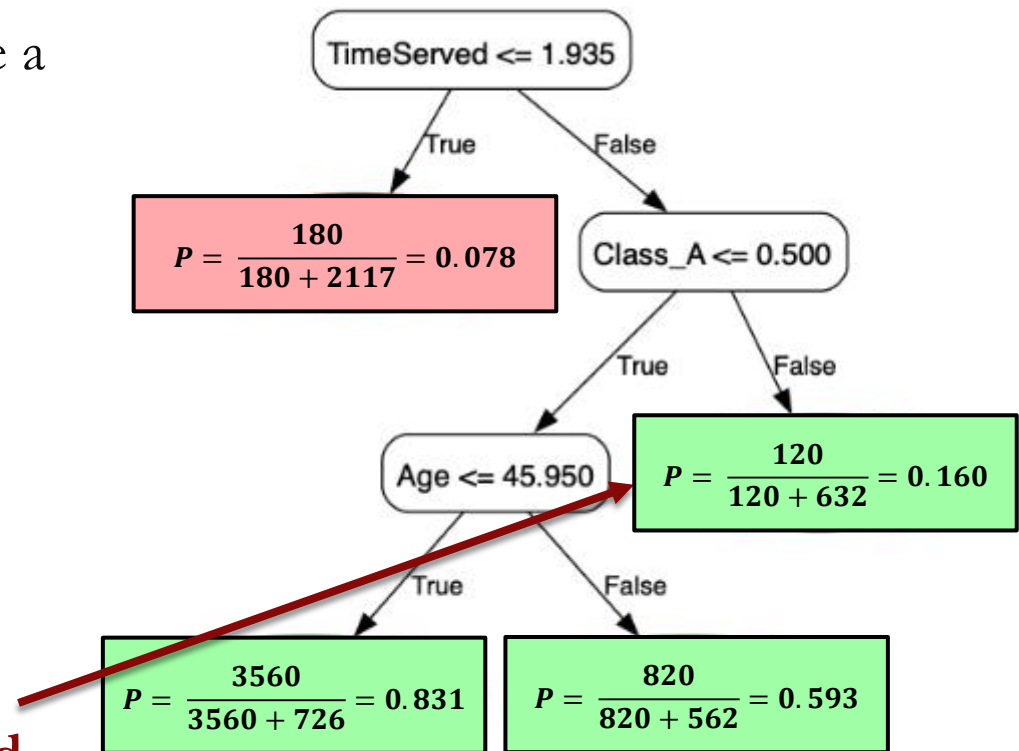
- Assume that we use $L = 20$ to construct the CART but use a different p_{thres} at test time.
- E.g.: set $p_{thres} = 0.1$. Test input falls into leaf:

No Violator: $n_{Non-Violators}$
Violator: $L \times n_{violators}$

Then it's labeled “Violator”

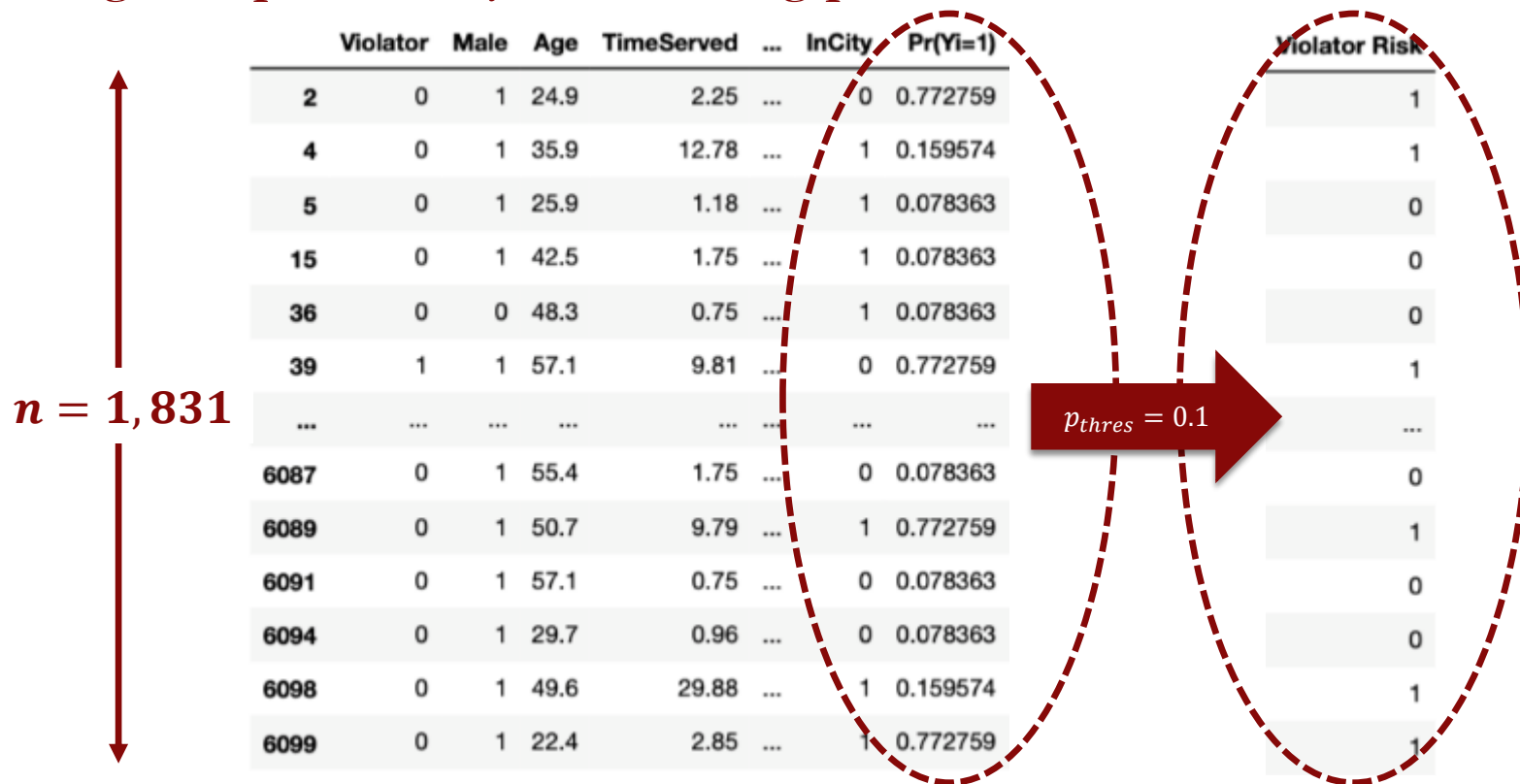
$$\Leftrightarrow P > p_{thres} = 0.1.$$

Note the change in predicted class for this leaf



Choose a threshold probability p_{thres} and process the Test set data into a confusion matrix for that cutoff value

Suppose we declare someone a parole violation risk if their loss-weighted probability of violating parole is at least $p_{thres} = 0.1 \dots$

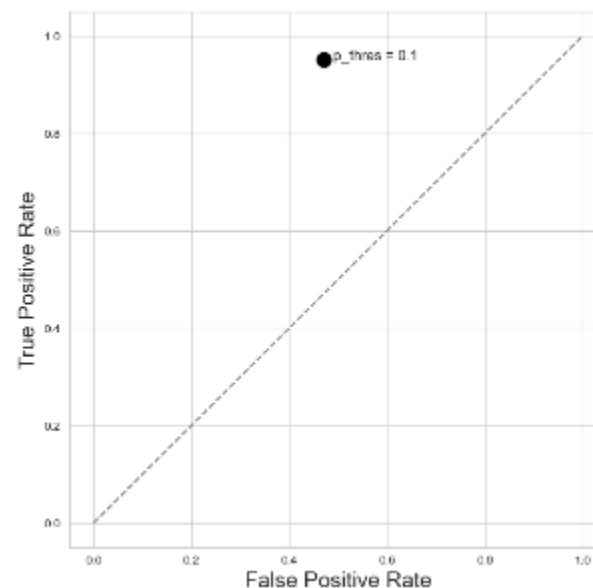


Confusion Matrix for cutoff probability $p_{thres} = 0.1$

		Model (cutoff probability $p_{thres} = 0.1$)	
Reality		Low violation risk $\Pr(Y_i = 1) < 0.1$	High violation risk $\Pr(Y_i = 1) > 0.1$
	No Violation ($Y_i = 0$)	917	813
	Violation ($Y_i = 1$)	5	96

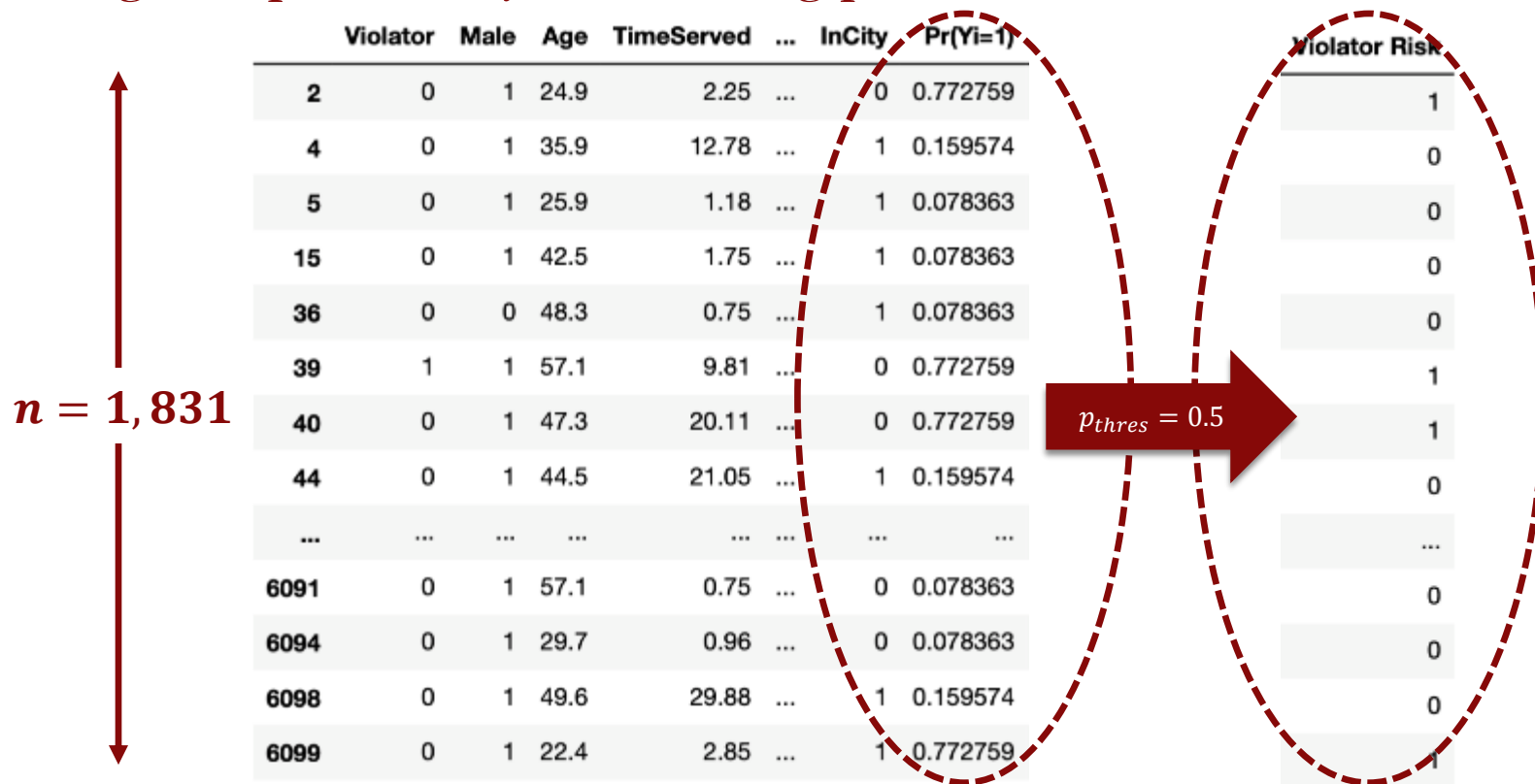
Records of all TPR and FPR values

Cutoff probability p_{thres}	TPR	FPR
0.1	0.951	0.470



Choose a threshold probability p_{thres} and process the Test set data into a confusion matrix for that cutoff value

Suppose we declare someone a parole violation risk if their loss-weighted probability of violating parole is at least $p_{thres} = 0.5...$

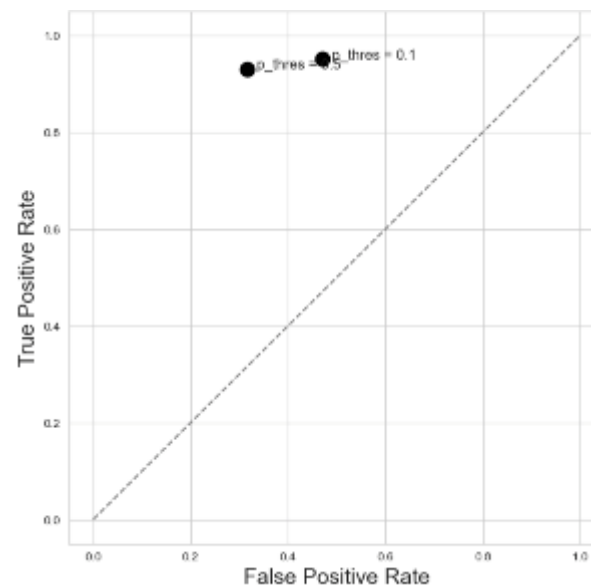


Confusion Matrix for cutoff probability $p_{thres} = 0.5$

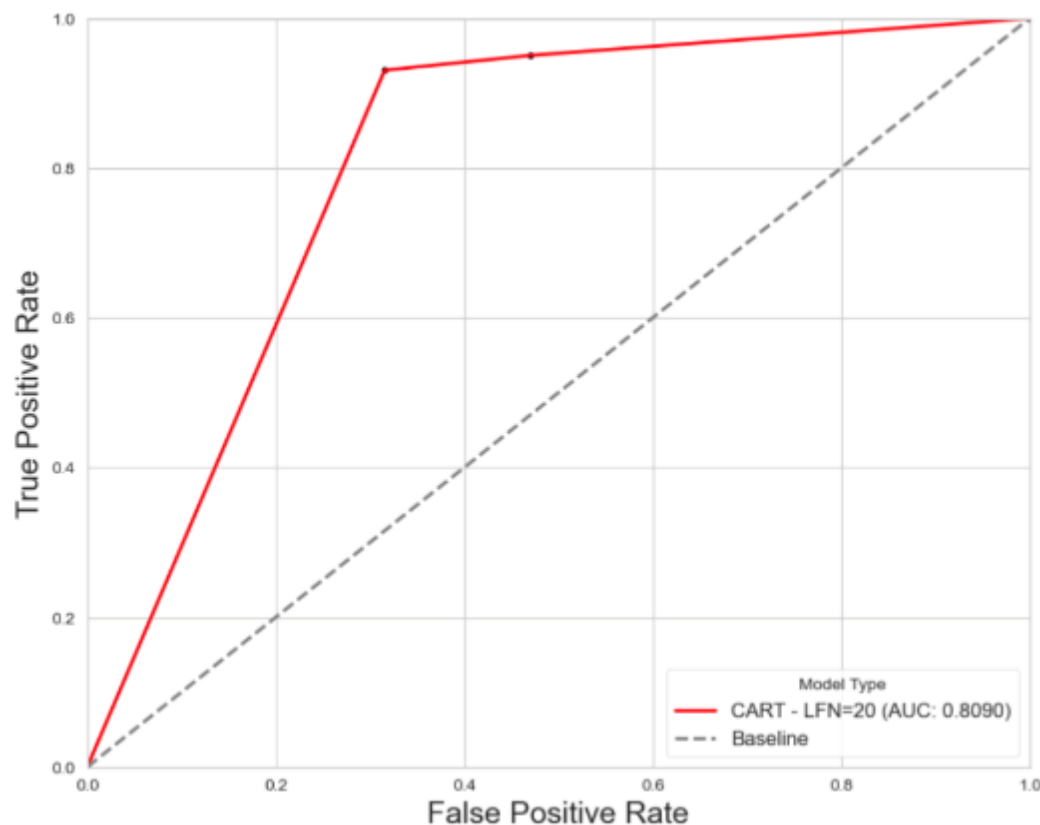
		Model (cutoff probability $p_{thres} = 0.5$)	
Reality		Low violation risk $\Pr(Y_i = 1) < 0.5$	High violation risk $\Pr(Y_i = 1) > 0.5$
	No Violation ($Y_i = 0$)	1184	546
	Violation ($Y_i = 1$)	7	94

Records of all TPR and FPR values

Cutoff probability p_{thres}	TPR	FPR
0.1	0.951	0.470
0.5	0.931	0.316

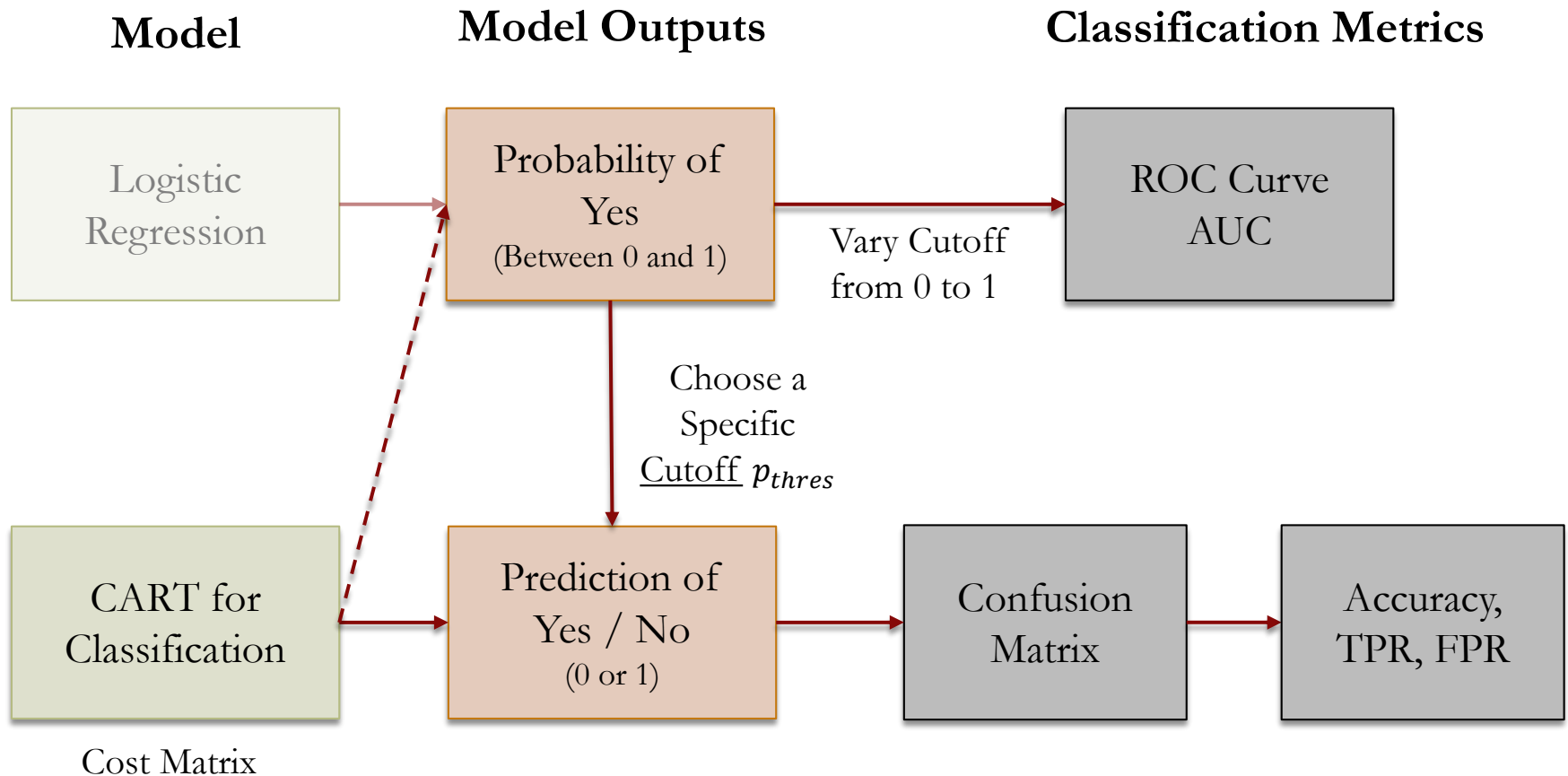


ROC Curve and AUC for the CART Model's Probability Tree with $L_{FN} = 20$



Model	AUC
CART $L_{FN} = 20$	0.810
Naïve Baseline	0.500

Aside: CART for Classification



Conclusion

Analytics for Parole Decisions

- 15+ states require formal risk assessment tools to inform Parole Boards. Texas has been using analytics the longest.
- 80%+ of all Parole Boards use analytics for decision support
 - **“Unaided, parole board members rely too much on their personal experiences and make inconsistent decisions”** – Donna Syteck, New Hampshire Parole Board
 - **“It is better to be guided by software than one’s gut”**– Olivia Craven, head of the Idaho Commission of Pardons and Parole
 - Depending on the state, models consider 50-100 different factors!
- Newer models use “personalized terms of parole” to adjust the terms of parole based on parolee characteristics
 - Improve likelihood of rehabilitation through personalized academic/vocational training, substance abuse rehabilitation programs, mental health programs, work programs, etc.

Methodology Summary

- CART: a tree-based method for both **regression** and **classification**
 - Interpretable
 - Data-driven
 - Flexible
 - Predictive power
- Trade-off between model complexity and predictive power
- k -fold cross validation: a powerful technique